

분산처리

GPU Architecture & CUDA Programming



Computer & Ai

Department of Computer Engineering

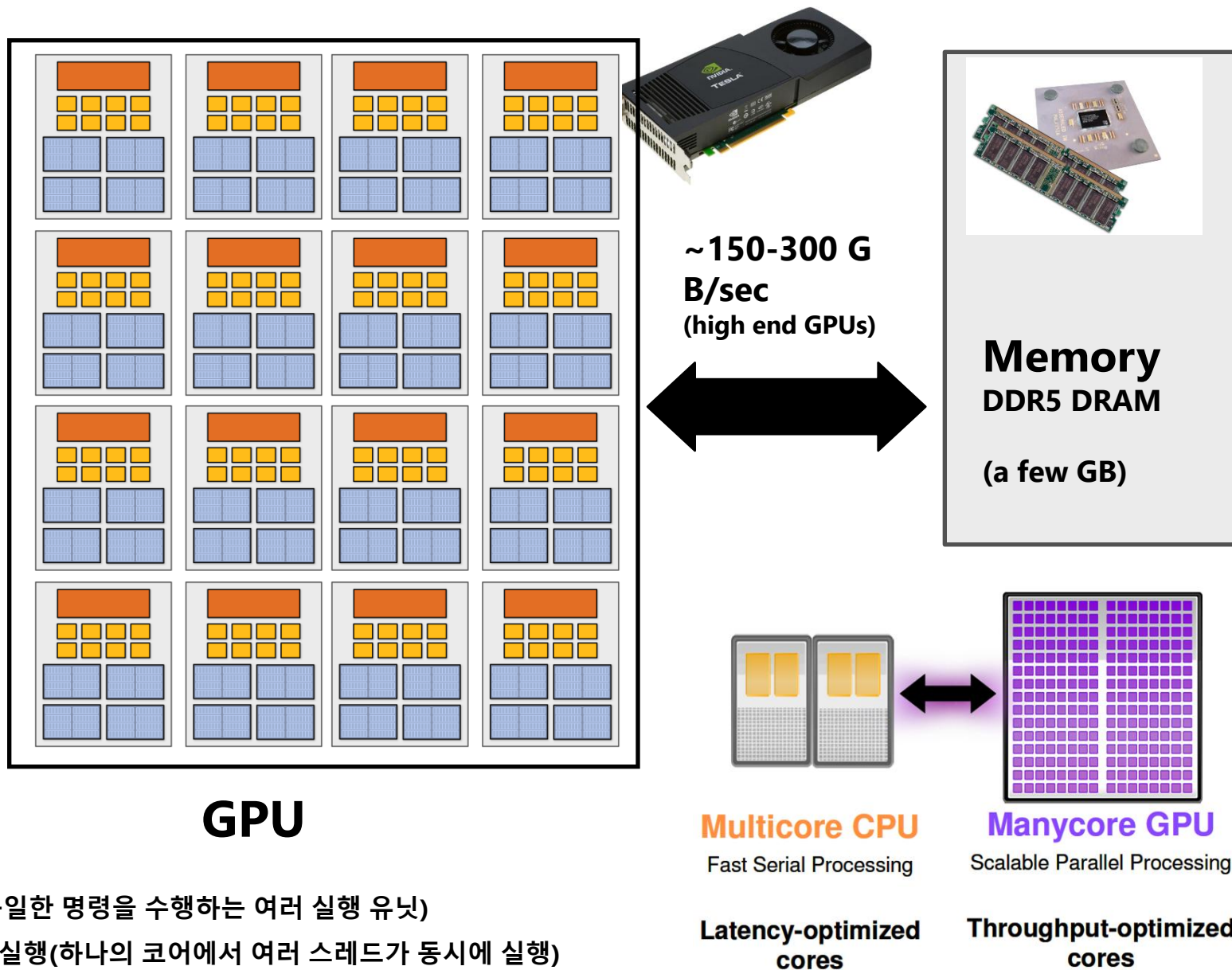
Review: how to run code on a CPU

History: 원래 3D 게임 가속을 위해 설계된 그래픽 프로세서가 다음과 같은 광범위한 애플리케이션을 위한 고도의 병렬 컴퓨팅 엔진으로 진화한 과정을 살펴보자:

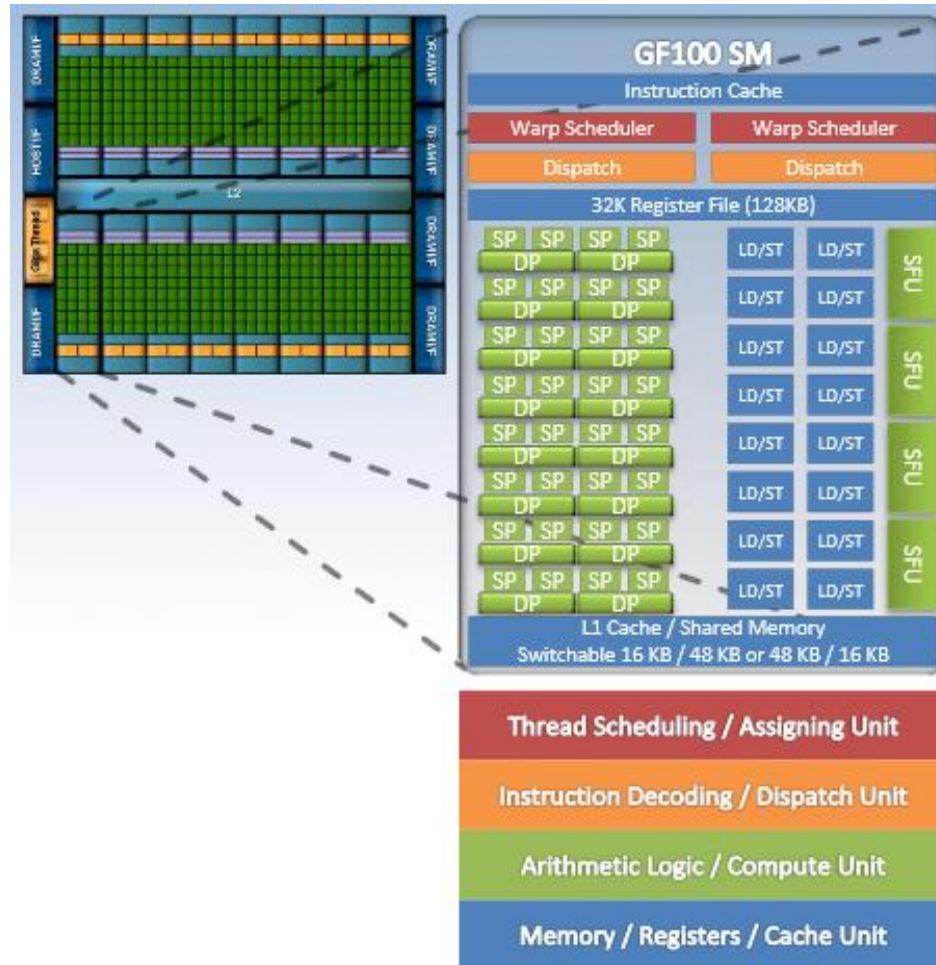
- deep learning
 - computer vision
 - scientific computing
- **CUDA 언어를 사용한 GPU 프로그래밍**
 - **GPU 아키텍처에 대해 자세히 살펴보기**



basic GPU architecture 이해



basic GPU architecture 이해



GPU가 본연의 설계된 기능: 3D 렌더링

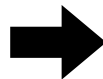
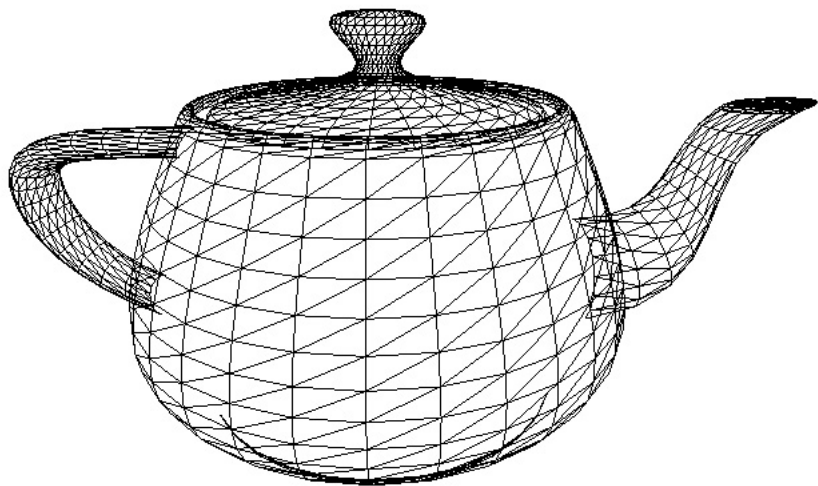


Image credit: Henrik Wann Jensen

Input: description of a scene

(장면을 만들기 위한 정보):

3D surface geometry (e.g., triangle mesh)

surface materials, lights, camera, etc.

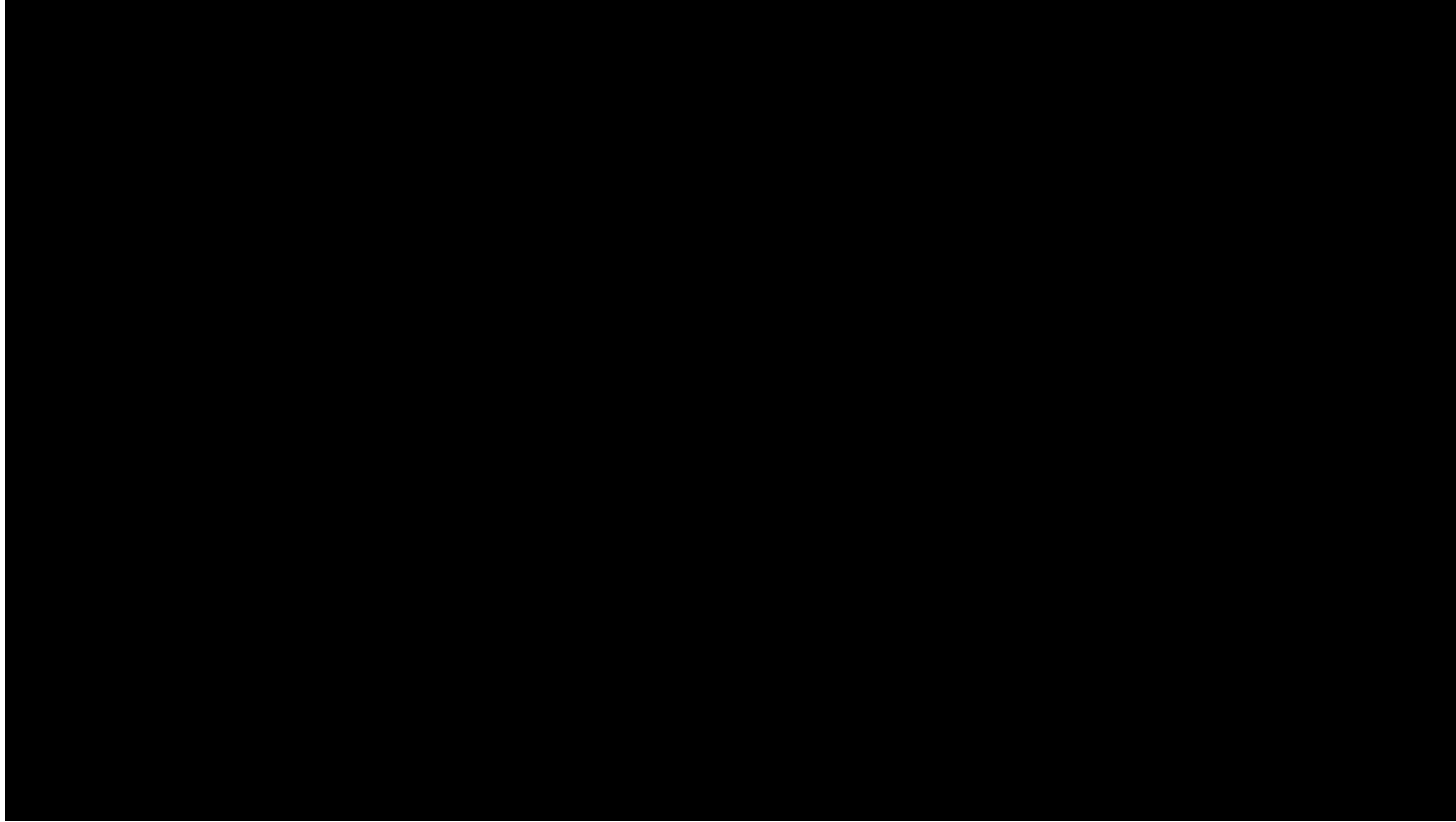
Output: image of the scene

렌더링 작업의 간단한 정의: 3D 메시의 각 트라이앵글이 이미지의 각 픽셀 모양에 어떻게 기여하는지 계산하는 것?

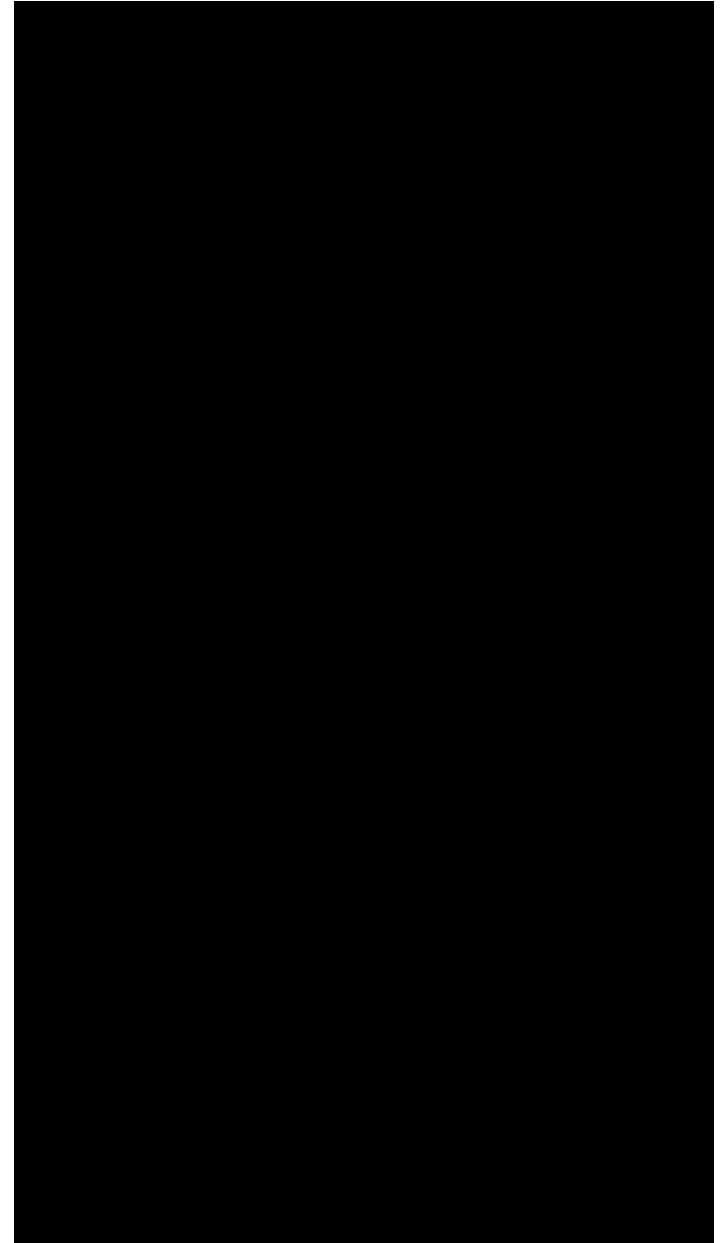
https://threejs.org/examples/?q=tea#webgl_geometry_teapot

GPU가 본연의 설계된 기능: 3D 렌더링

Real-time (30 fps) on a high-end GPU

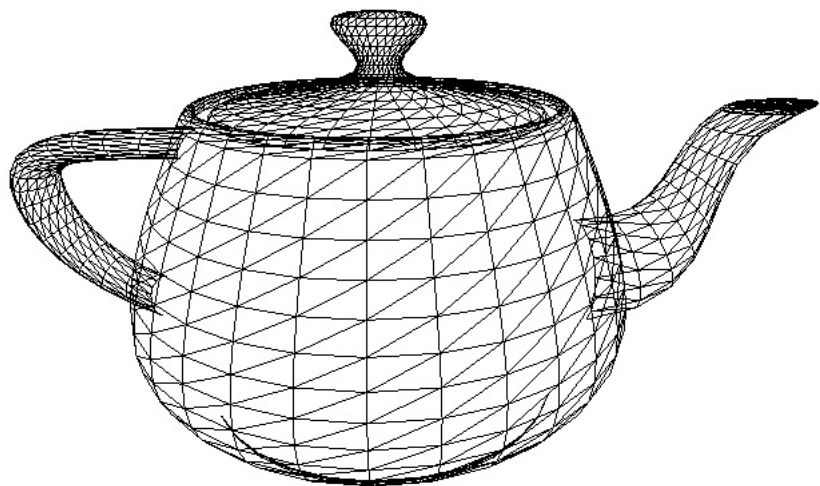


Unreal Engine Kite Demo (Epic Games 2015)



Real-time graphics primitives (entities)

표면을 3D 삼각형 메시로 표현



[three.js examples \(threejs.org\)](https://threejs.org/examples/)

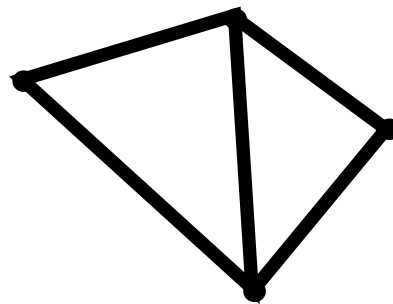
• 1

• 3

• 4

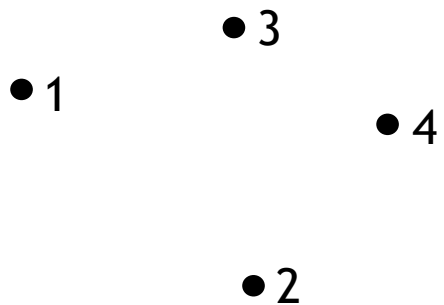
• 2

Vertices
(points in space)

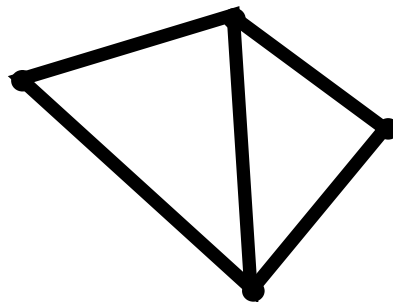


Primitives
(e.g., triangles, points, lines)

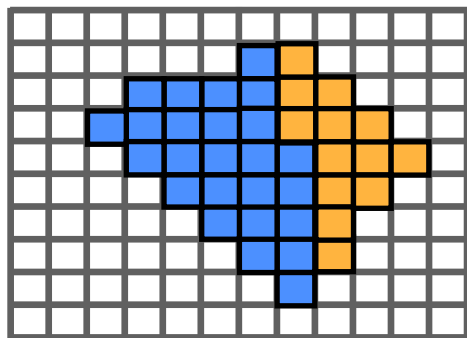
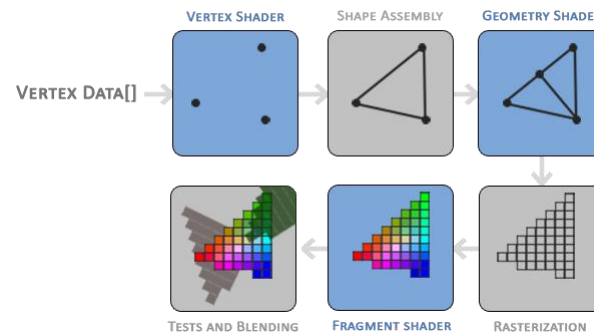
Real-time graphics primitives (entities)



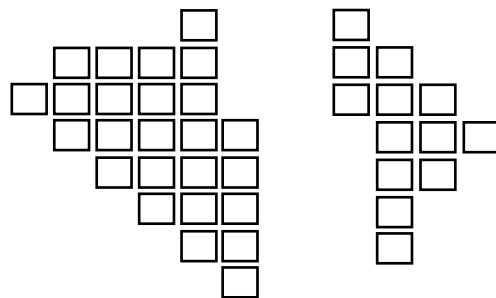
Vertices
(points in space)



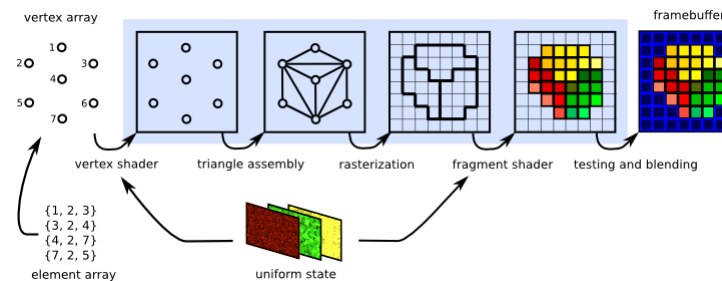
Primitives
(e.g., triangles, points, lines)



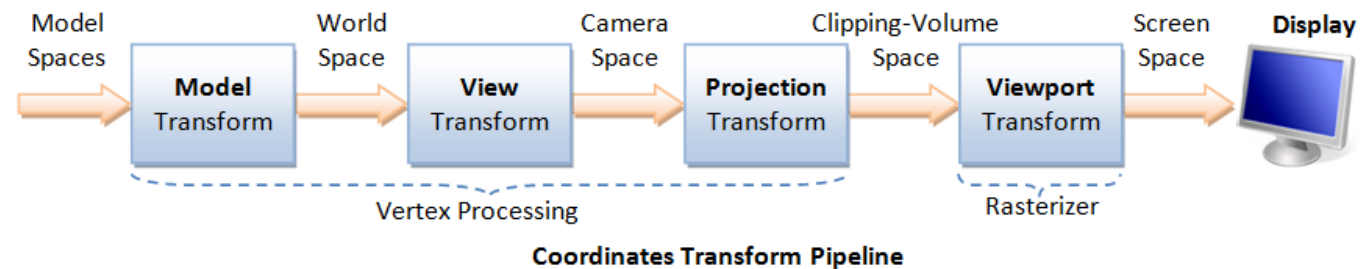
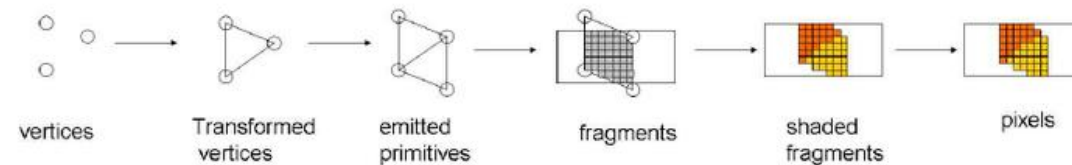
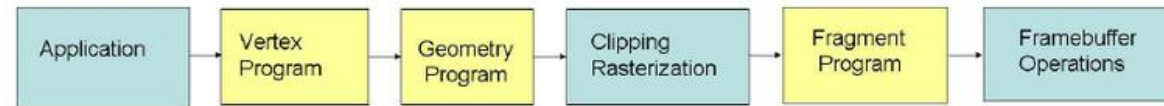
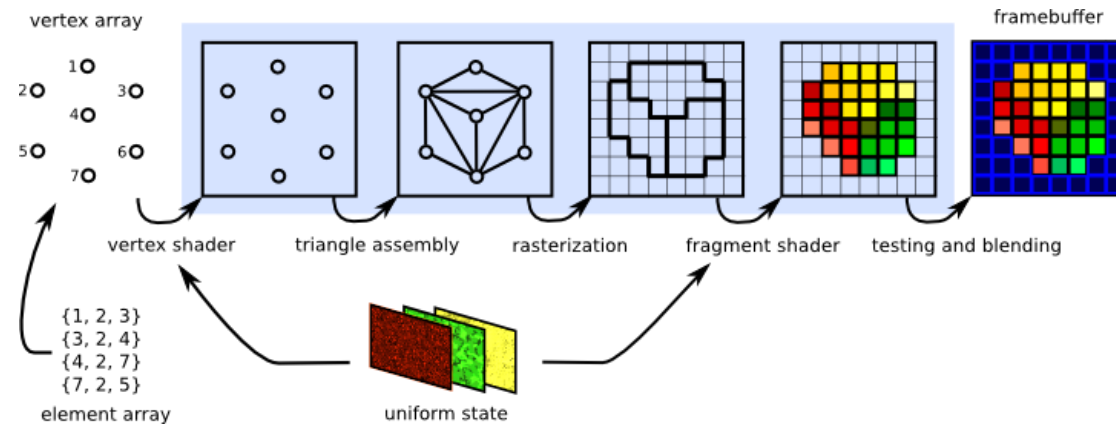
Pixels (in an image)



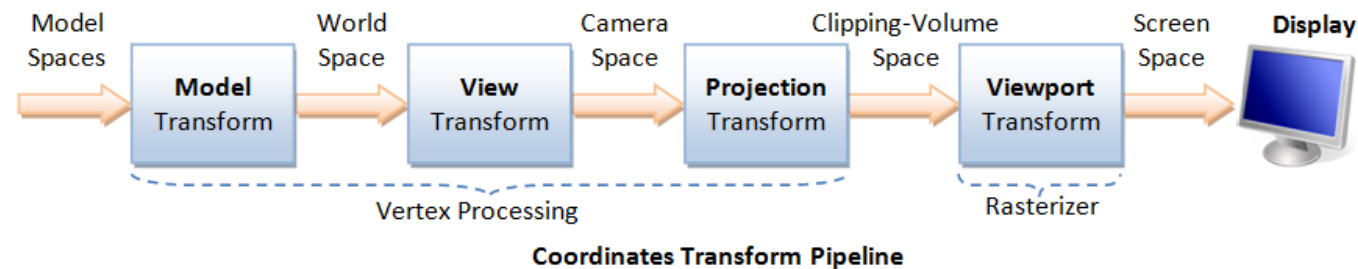
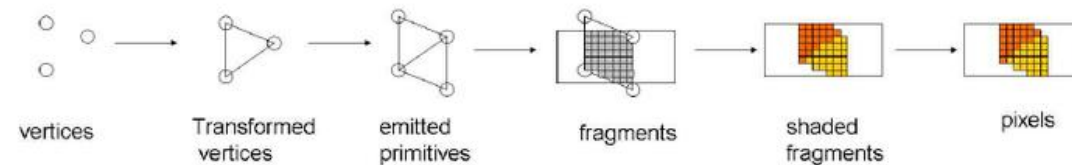
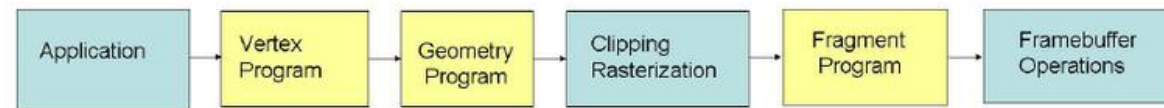
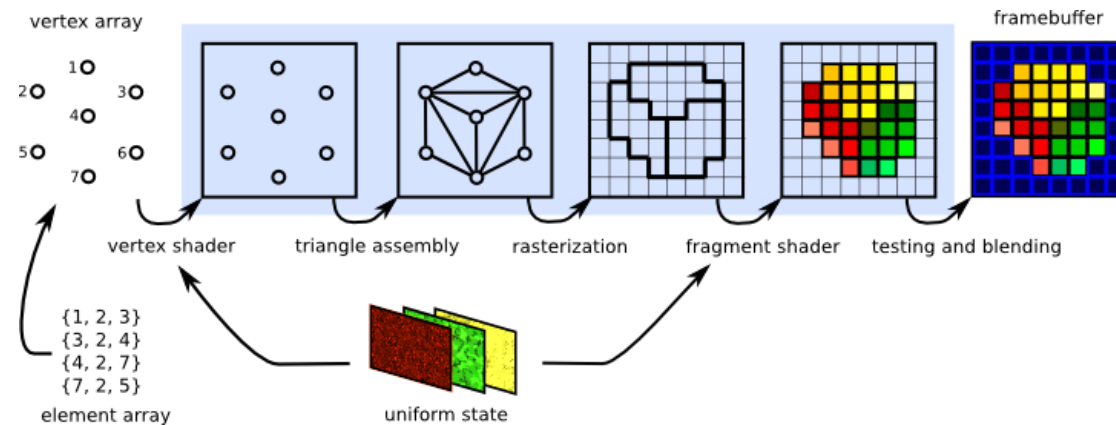
Fragments



Transformations and the Graphics Pipeline



Transformations and the Graphics Pipeline

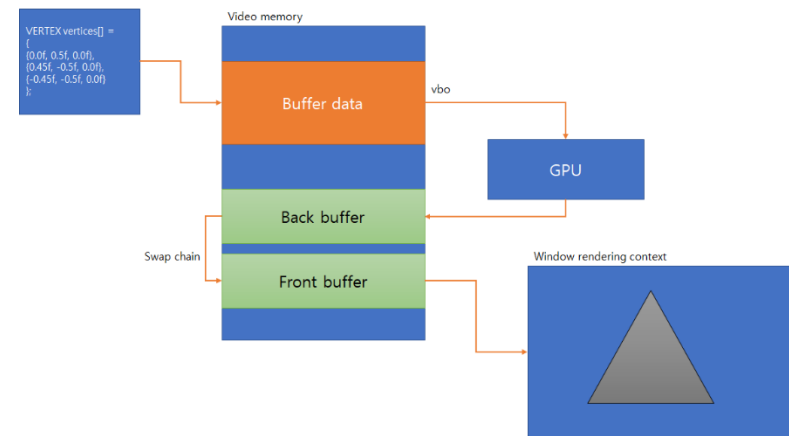
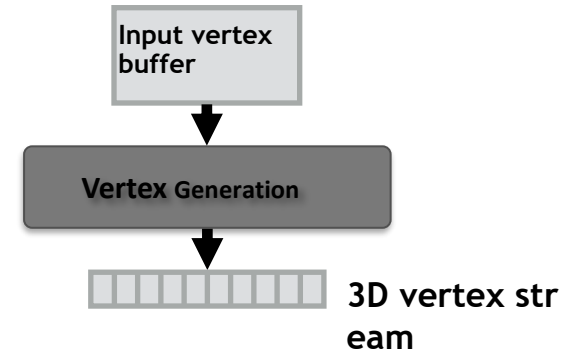


Rendering a picture

입력: 3D 공간의 정점 목록(및 프리미티브에 대한 연결성)

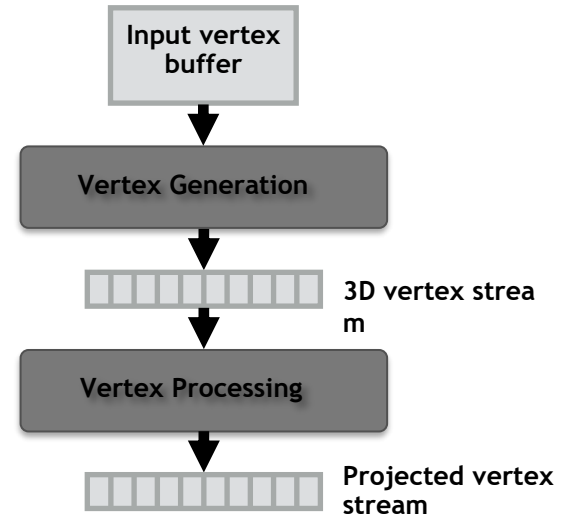
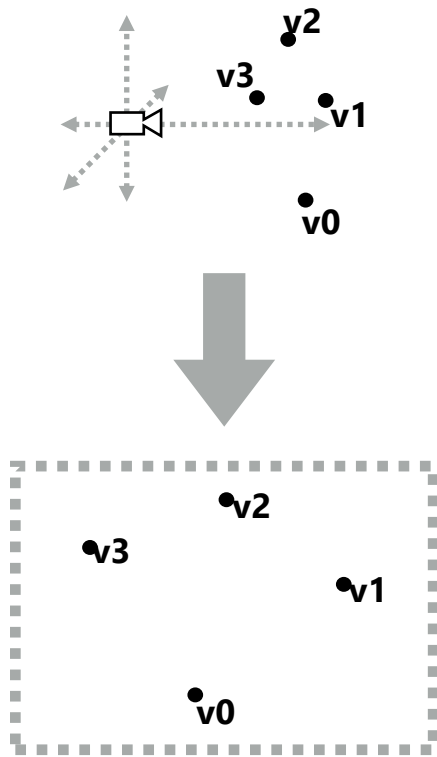
예: 정점 3개마다 삼각형을 정의.

```
list_of_positions = {  
    v0x, v0y, v0z,  
    v1x, v1y, v1x, triangle 0 = {v0, v1, v2}  
    v2x, v2y, v2z, triangle 1 = {v1, v2, v3}  
    v3x, v3y, v3x  
};
```



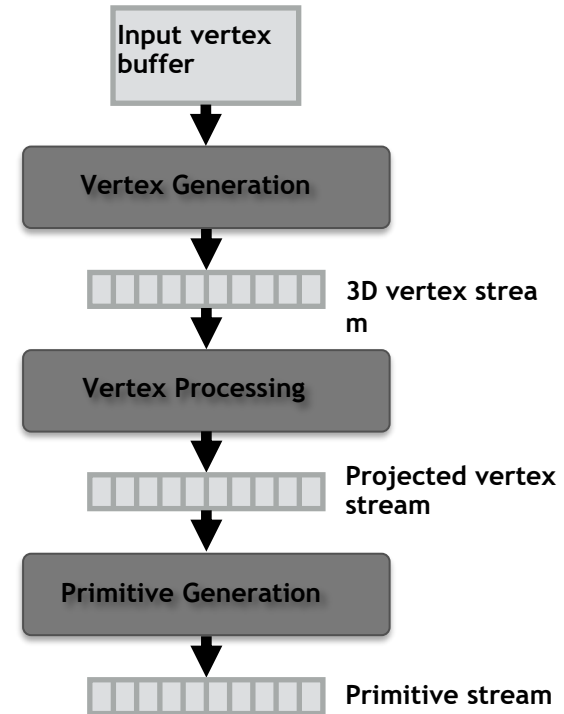
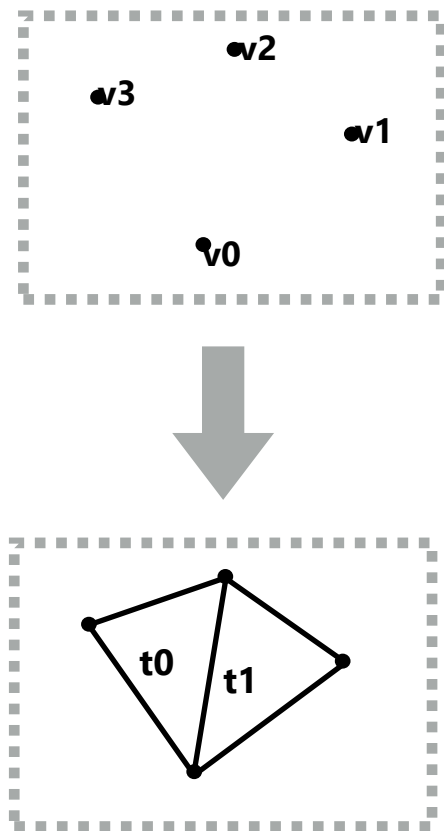
Rendering a picture

1단계: 썬 카메라 위치가 주어지면 화면
에서 정점이 어디에 있는지 계산.



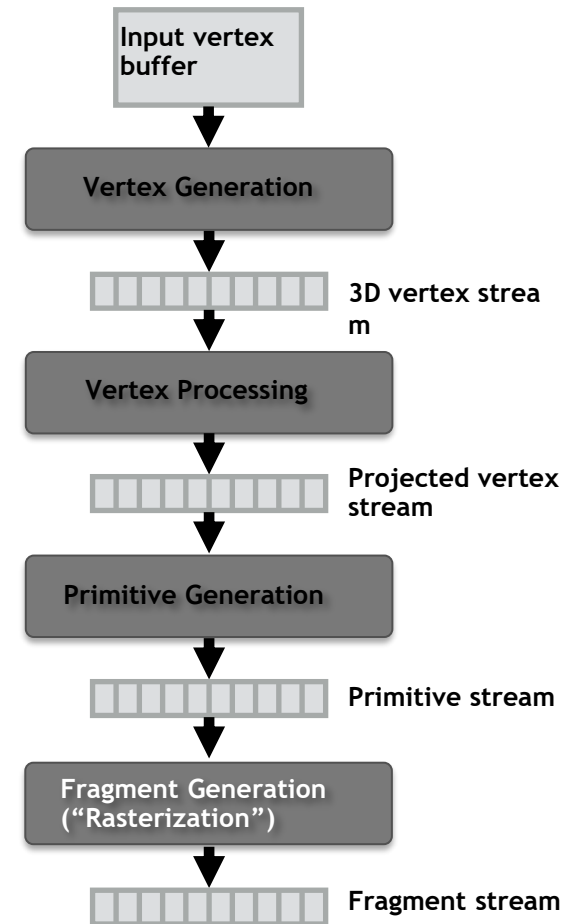
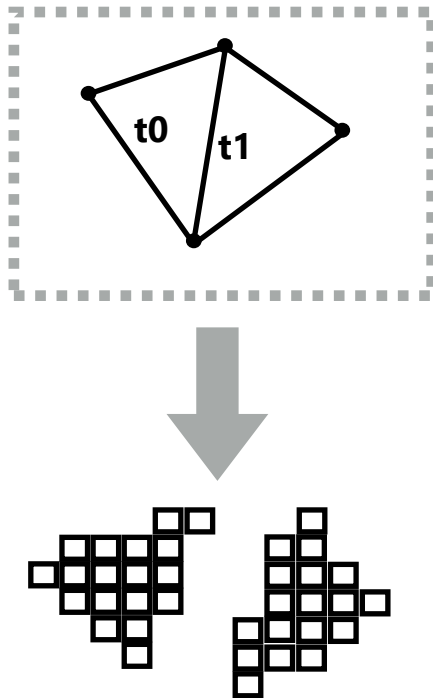
Rendering a picture

2단계: 정점을 프리미티브로 그룹화하기



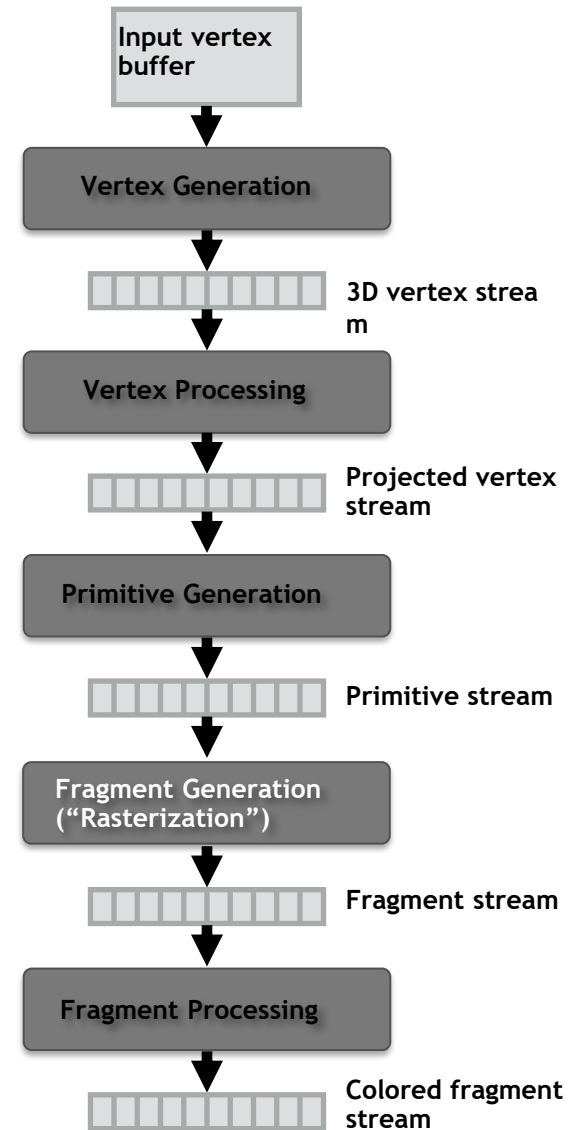
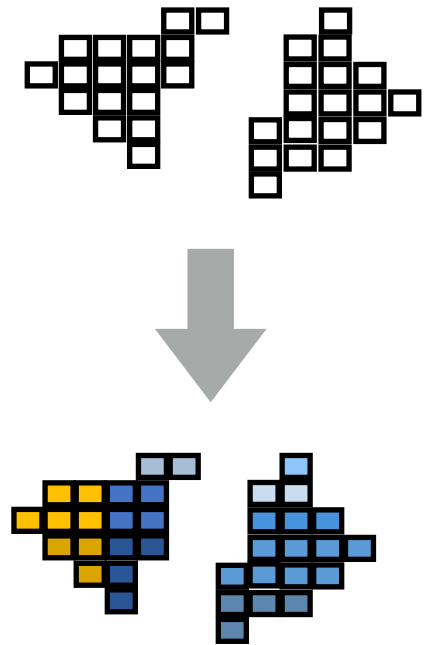
Rendering a picture

3단계: 프리미티브가 겹치는 각 픽셀에 대해 하나의 조각을 생성.



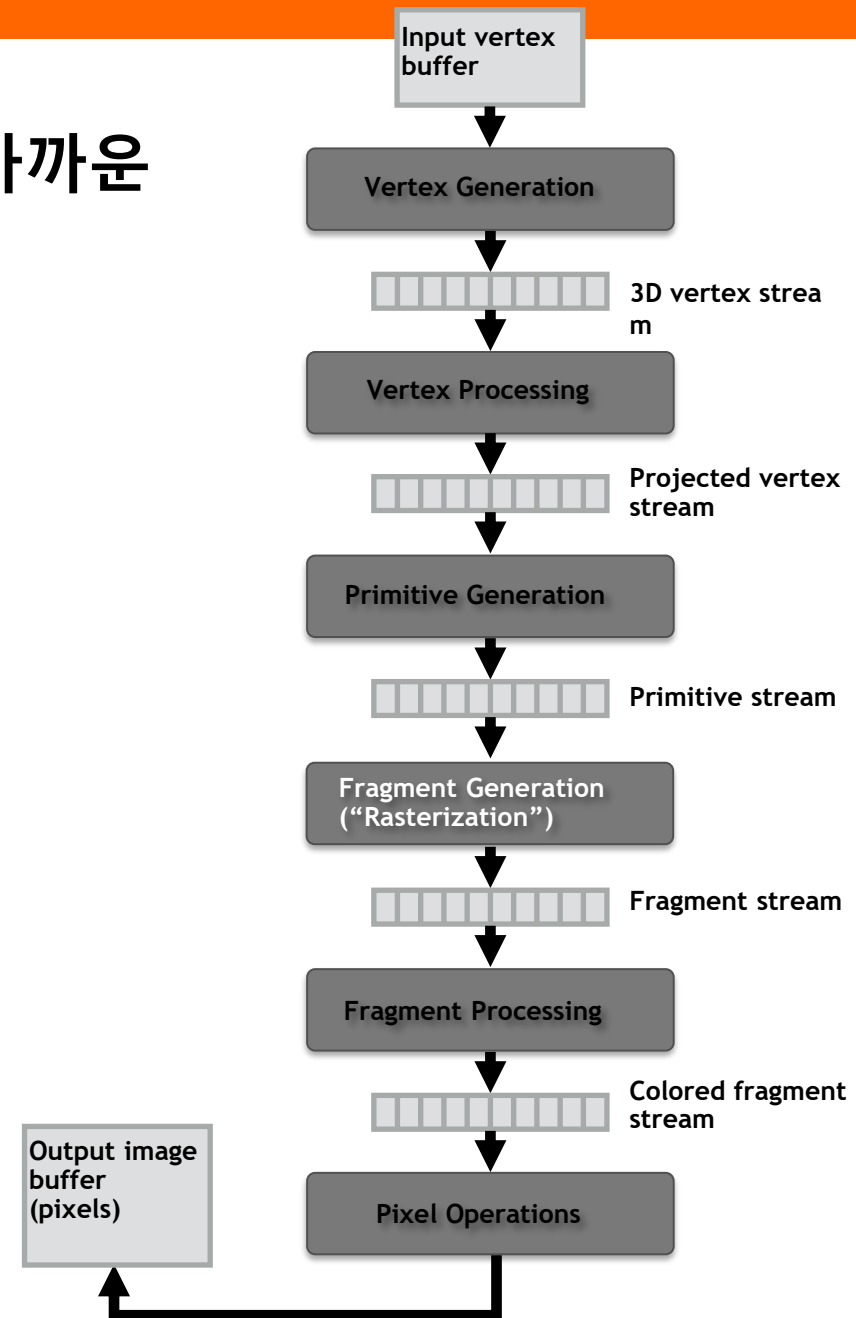
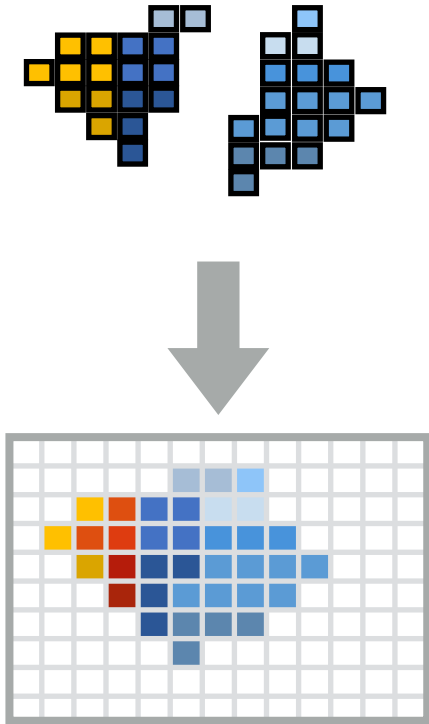
Rendering a picture

4단계: 각 조각에 대한 프리미티브 색상 계산(썬 라이팅 및 프리미티브 머티리얼 프로퍼티 기반)



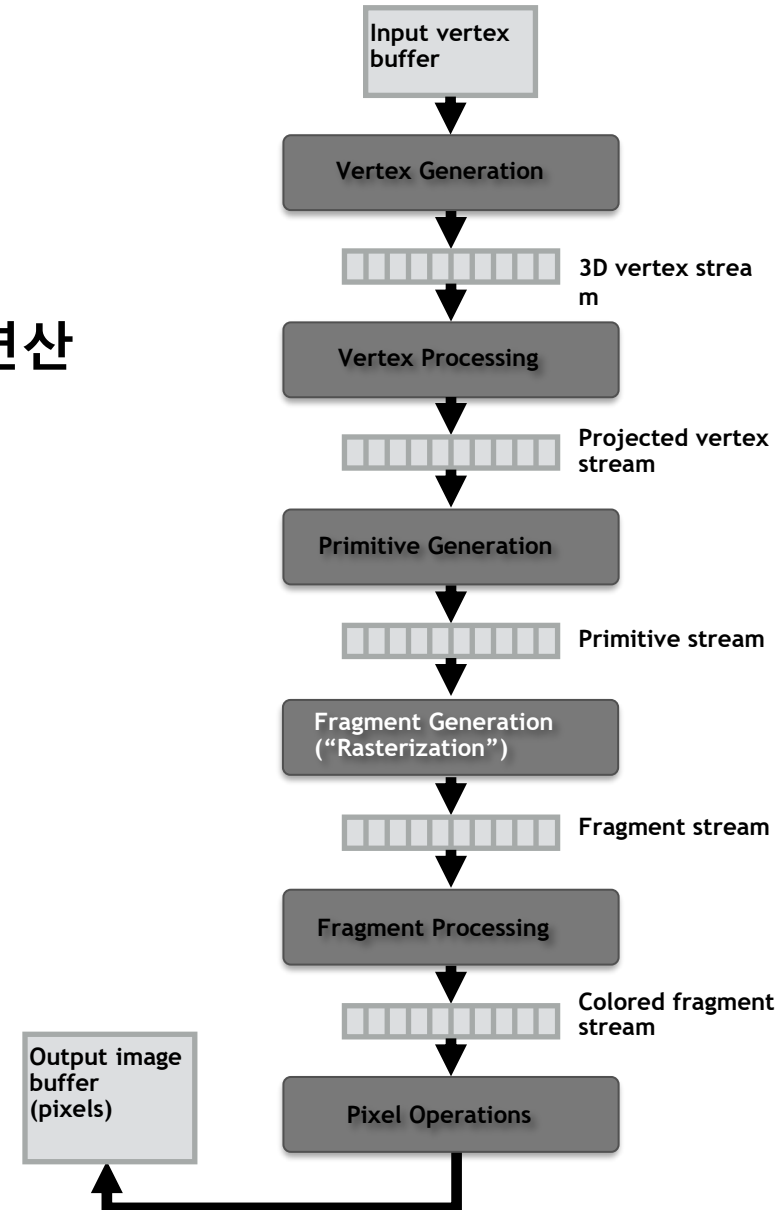
Rendering a picture

5단계: 출력 이미지에서 카메라에 "가장 가까운 조각"의 색상을 넣음.



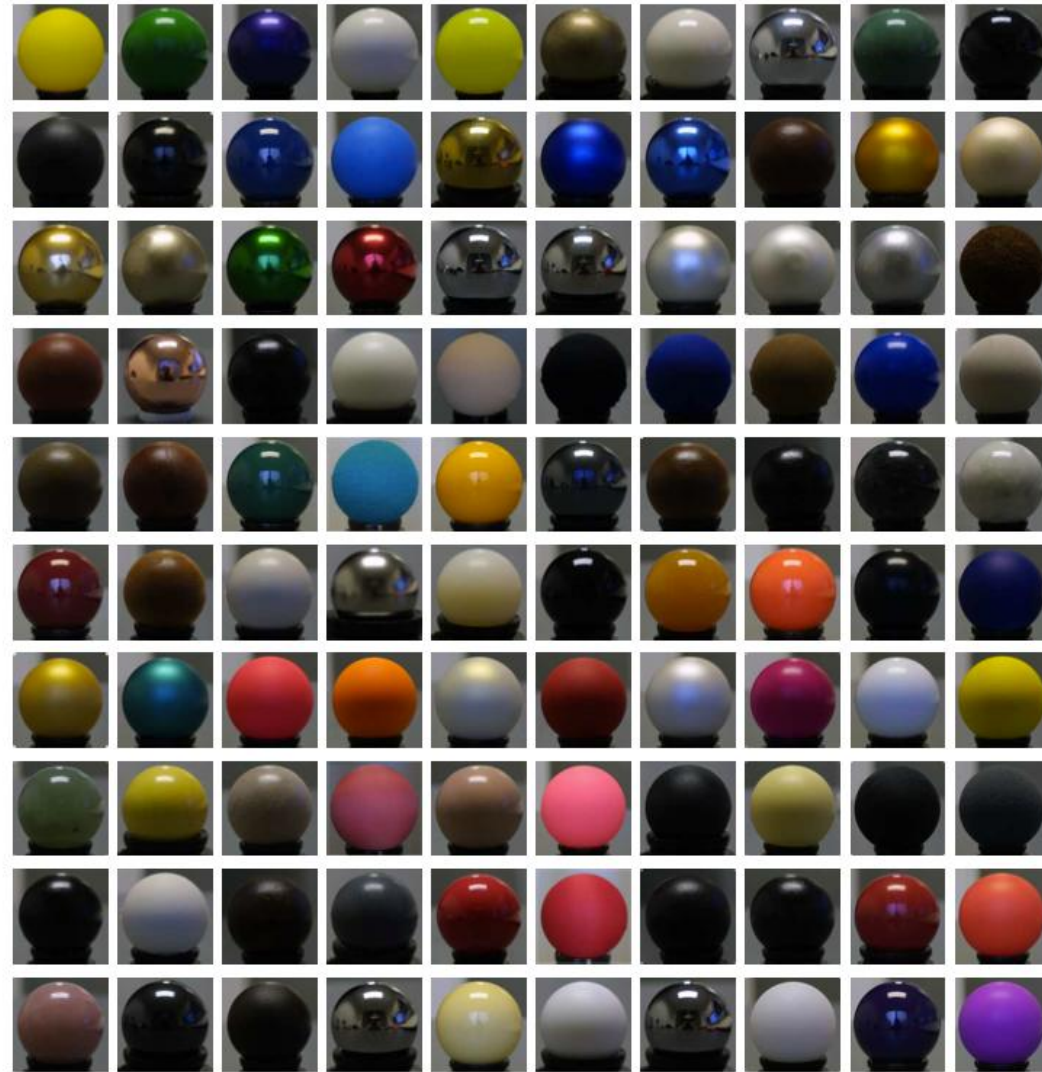
Real-time graphics pipeline

- 그림을 정점, 프리미티브, 조각, 픽셀에 대한 일련의 연산으로 렌더링하는 과정을 추상화 함.



프래그먼트 처리 계산은 real-world material의 빛 반사를 시뮬레이션함.

Example materials:



Images from Matusik et al. SIGGRAPH 2003

초기 그래픽 프로그래밍(OpenGL API)

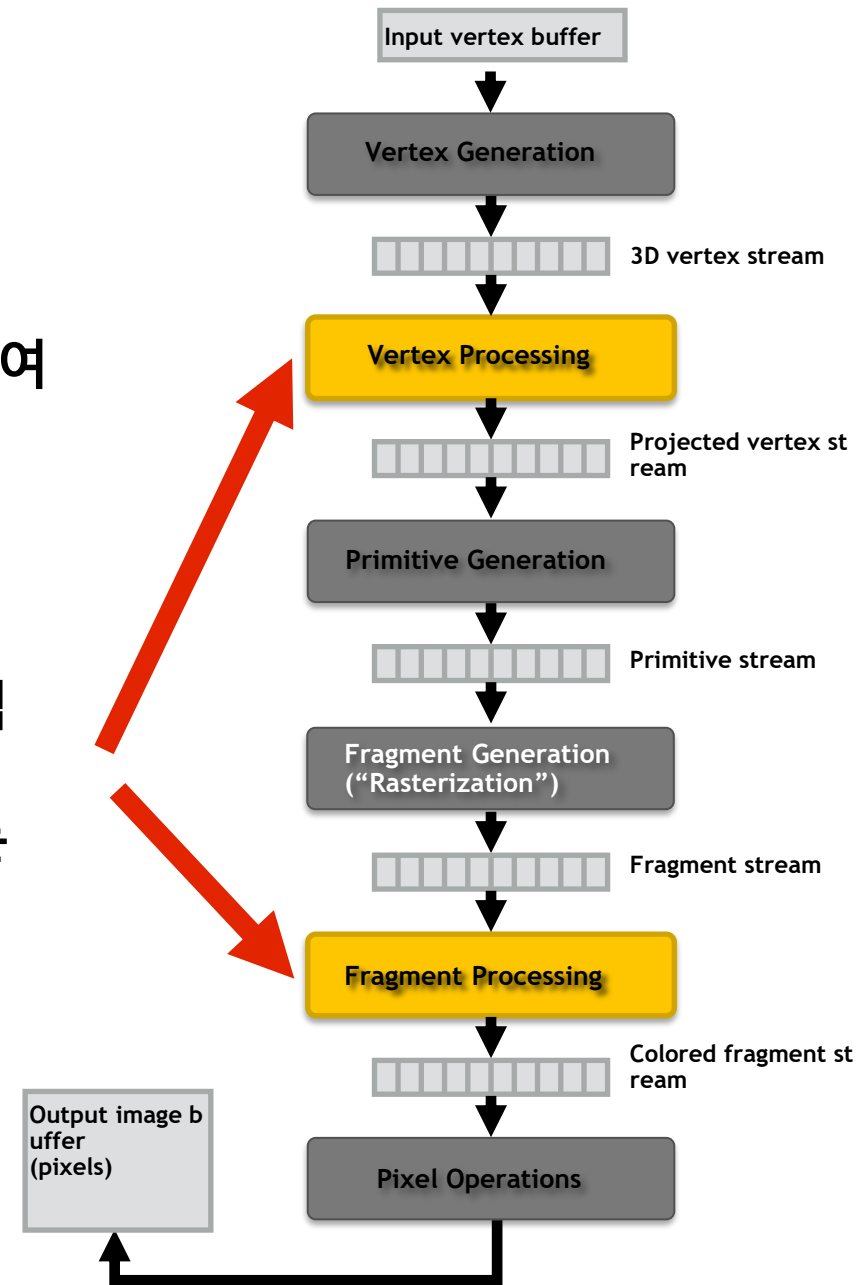
- 그래픽스 프로그래밍 API는 프로그래머가 썬 조명(Light) 및 머티리얼(Material)의 파라미터를 설정할 수 있는 메커니즘을 제공.
- `glLight(light_id, parameter_id, parameter_value)`
 - Examples of light parameters: color, position, direction
- `glMaterial(face, parameter_id, parameter_value)`
 - Examples of material parameters: color, shininess

세상의 다양한 소재와 조명!



Graphics shading languages

- 머티리얼과 조명을 프로그래밍 방식으로 지정하여 그래픽 파이프라인의 기능을 확장할 수 있음!
 - 다양한 머티리얼 지원
 - 다양한 조명 조건 지원
- 프로그래머는 특정 단계에 대한 파이프라인 로직을 정의하는 미니 프로그램("셰이더")을 제공.
- 파이프라인은 셰이더 기능을 입력 스트림의 모든 요소에 매핑.



Example fragment shader program *

Run once per fragment (per pixel covered by a triangle)

다음 OpenGL shading 언어(GLSL) 셰이더 프로그램:

Fragment processing 단계의 동작을 정의.

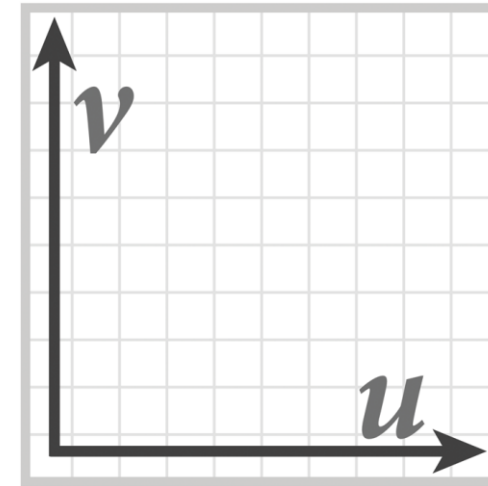
```
uniform sampler2D myTexture;
uniform float3 lightDir;
varying vec3 norm;
varying vec2 uv;

void myFragmentShader()
{
    vec3 kd = texture2D(myTexture, uv);
    kd *= clamp(dot(lightDir, norm), 0.0, 1.0);
    return vec4(kd, 1.0);
}
```

read-only global variables

per-fragment inputs

myTexture is a texture map



“fragment shader”
(a.k.a kernel function mapped
onto input fragment stream)

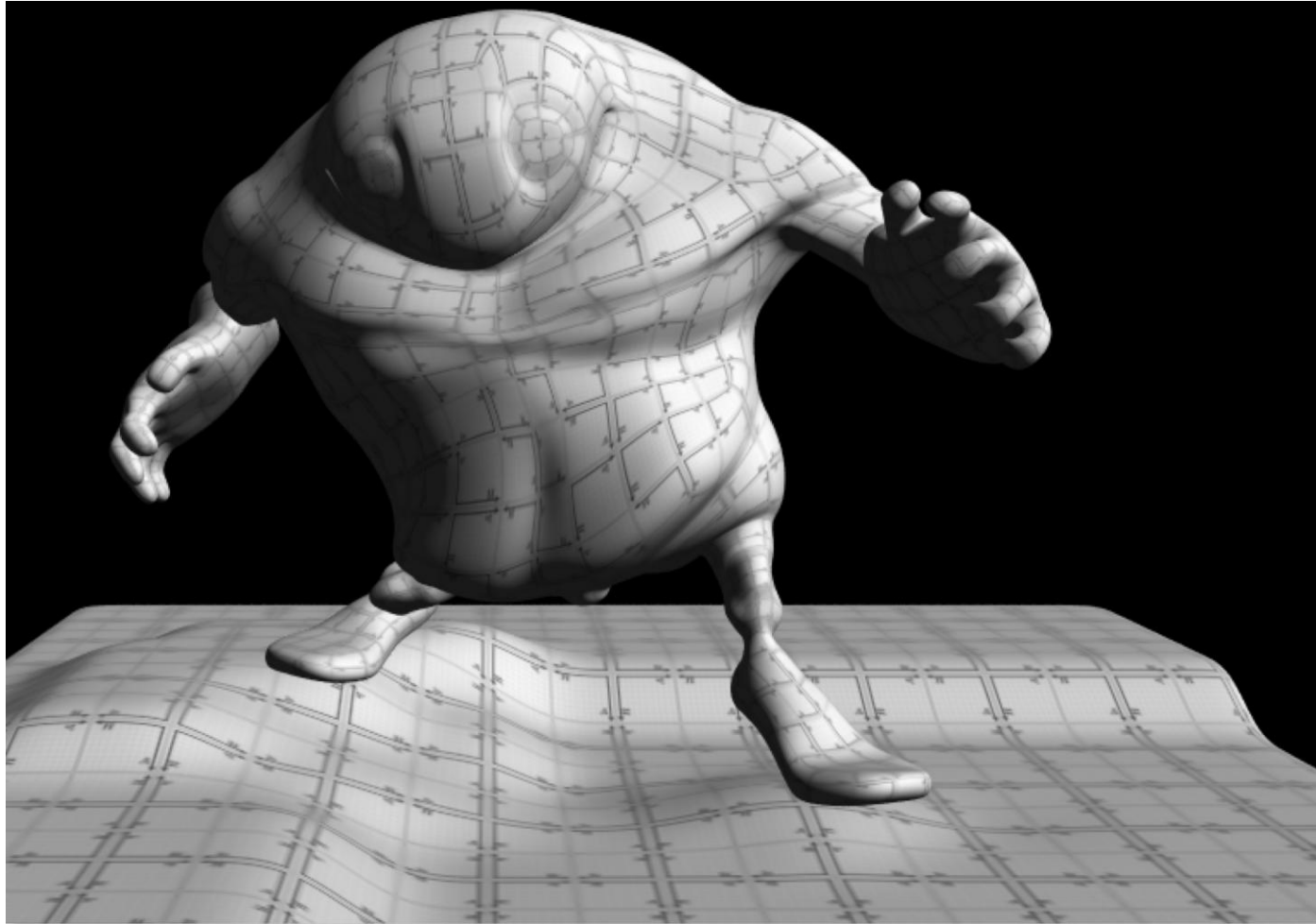
per-fragment output: RGBA surface color at pixel

* Syntax/details of this code not important to 15-418.

What is important is that it's a kernel function operating on a stream of inputs.

음영 처리된 결과

이미지에는 표면이 덮은 각 픽셀에 대한 myFragmentShader의 출력이 포함(여러 표면이 덮은 픽셀은 카메라에 가장 가까운 표면의 출력을 포함).

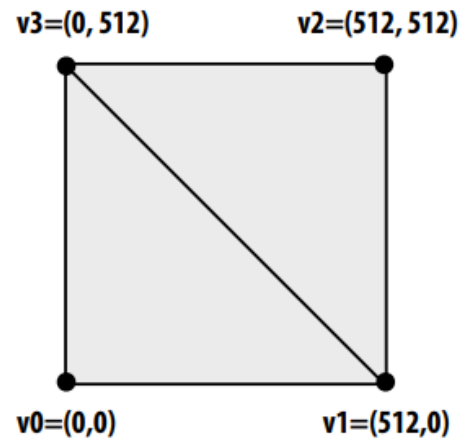


음영 처리된 결과

이미지에는 표면이 덮은 각 픽셀에 대한 `myFragmentShader`의 출력이 포함(여러 표면이 덮은 픽셀은 카메라에 가장 가까운 표면의 출력을 포함).

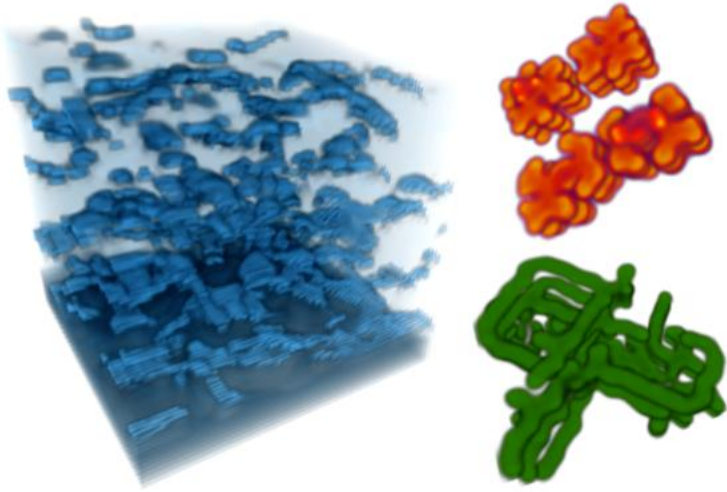
셰이더를 다른 연산에 사용할 수 없나요?

- OpenGL 출력 이미지 크기를 출력 배열 크기(예: 512 x 512)로 설정.
- 화면을 정확히 덮는 2개의 삼각형을 렌더링
(픽셀당 하나의 셰이더 계산 = 하나의 셰이더 계산 출력 이미지 요소).
- 이제 GPU를 데이터 병렬 프로그래밍 시스템처럼 사용할 수 있다.
- **Fragment shader function**는 512 x 512 요소 컬렉션에 매핑됩니다.

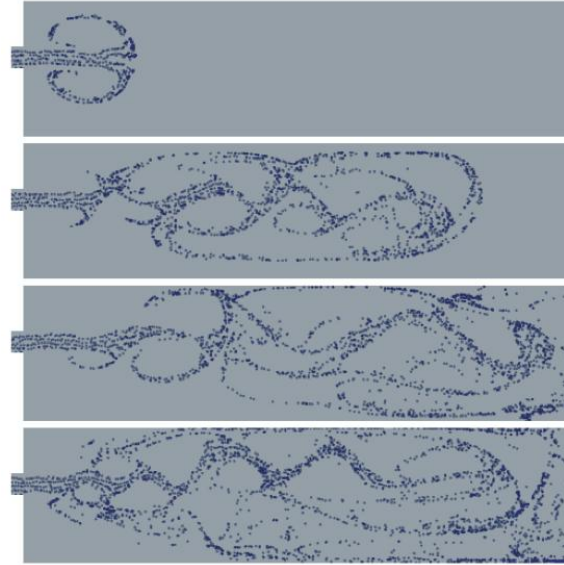


“GPGPU” 2002-2003

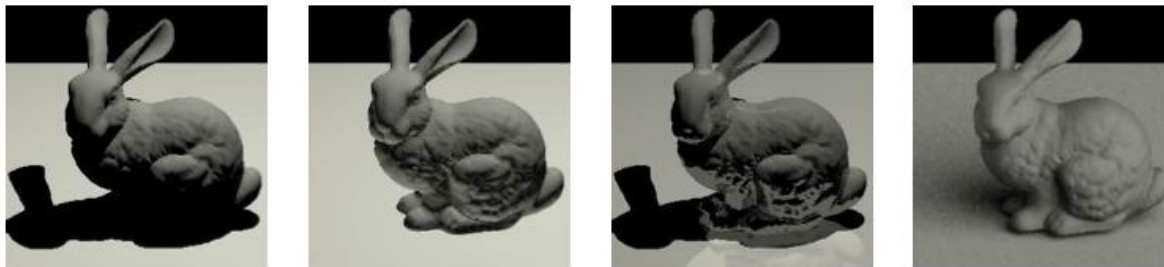
GPGPU = “general purpose” computation on GPUs



Coupled Map Lattice Simulation [Harris 02]



Sparse Matrix Solvers [Bolz 03]



Ray Tracing on Programmable Graphics Hardware [Purcell 02]

2001-2003년경 관측

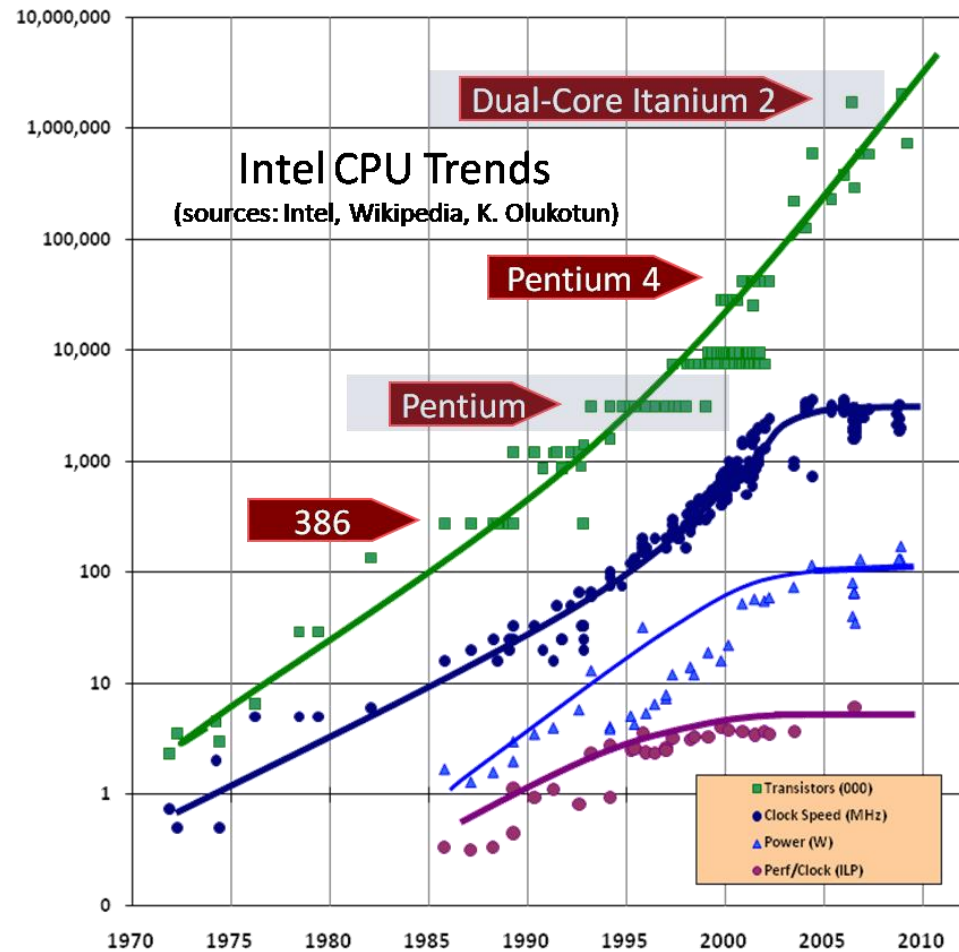
이러한 GPU는 대규모 데이터 모음(버텍스, 조각, 픽셀 스트림)에서 동일한 연산(셰이더 프로그램)을 수행하는 데 매우 빠른 프로세서입니다.

2001년: 프로그래밍 가능한
셰이딩이 가능한 최초의 단일 칩
GPU인 Nvidia GeForce 3 출시

하지만 많은 제약(루프 없음)

2002: ATI Radeon 9700, 루프 및 FP
수학을 갖춘 셰이더 지원

잠깐만요! 저한테는 데이터 병렬
처리와 비슷하게 들리는데요!
90년대 이국적인 슈퍼컴퓨터의
데이터 병렬 처리를 기억합니다.



Brook stream programming language (2004)

- **Stanford graphics lab research project** [Buck 2004]
- **Abstract GPU hardware as data-parallel processor**

```
kernel void scale(float amount, float a<>, out float b<>)  
{  
    b = amount * a;  
}  
  
float scale_amount;  
float input_stream<1000>; // stream declaration  
float output_stream<1000>; // stream declaration  
  
// omitting stream element initialization...  
  
// map kernel function onto streams  
scale(scale_amount, input_stream, output_stream);
```

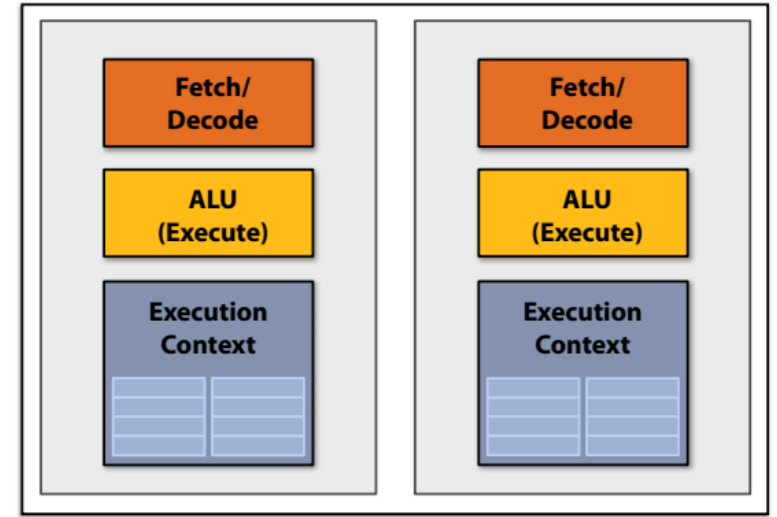
- Brook 컴파일러는 일반 스트림 프로그램을 당시의 GPU에서 실행할 수 있는 그래픽 명령어(예: drawTriangles)와 그래픽 셰이더 프로그램 세트로 변환함

CUDA Programming Language

GPU compute mode

Review: how to run code on a CPU

- 사용자가 멀티코어 CPU에서 프로그램을 실행하려고 한다고 가정해 보자
- ✓ OS가 프로그램 텍스트를 메모리에 로드.
- ✓ OS가 CPU 실행 컨텍스트 선택
- ✓ OS가 프로세서를 중단하고 실행 컨텍스트를 준비
(레지스터, 프로그램 카운터 등의 내용을 설정하여 실행 컨텍스트 준비).
- ✓ 실행!
- ✓ 프로세서는 실행 컨텍스트에서 유지된 환경 내에서 명령어 실행을 시작.



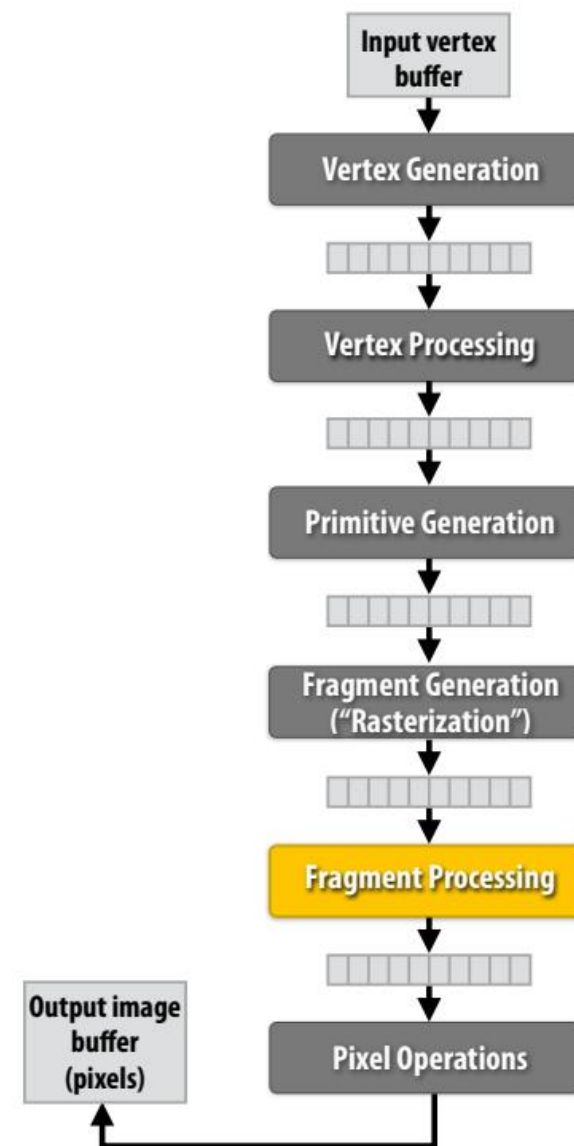
Multi-core CPU

How to run code on a GPU (prior to 2007)

● 사용자가 GPU를 사용하여 그림을 그리려고 한다고 가정해 보자..

- ✓ 애플리케이션(그래픽 드라이버를 통해)이 GPU 셰이더 프로그램 바이너리를 제공.
- ✓ 애플리케이션이 그래픽 파이프라인 파라미터를 설정.
(예: 출력 이미지 크기)
- ✓ 애플리케이션이 GPU에 버텍스 버퍼를 제공.
- ✓ 애플리케이션이 GPU에 "그리기" 명령을 보냄:
- ✓ `drawPrimitives(vertex_buffer)`

- 이것은 GPU 하드웨어에 대한 유일한 인터페이스였음.
- GPU 하드웨어는 그래픽 파이프라인 계산만 실행할 수 있었음.



NVIDIA Tesla architecture (2007)

➤ 첫 번째 대안, GPU 하드웨어에 대한 비그래픽 전용("컴퓨팅 모드") 인터페이스

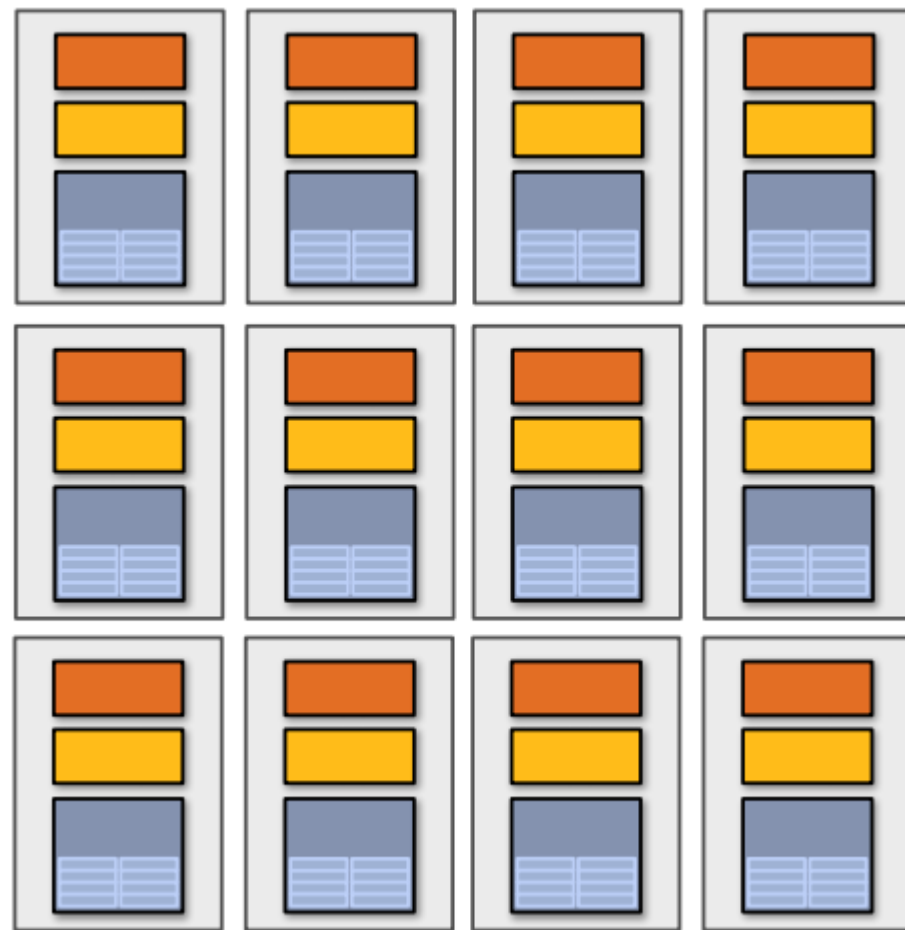
- 사용자가 GPU의 프로그래머블 코어에서 비그래픽 프로그램을 실행하고자 한다고 가정해 보자...

- ① 애플리케이션이 GPU 메모리에 버퍼를 할당하고 버퍼에서 데이터를 복사.
- ② 애플리케이션(그래픽 드라이버를 통해)이 GPU에 단일 커널 프로그램 바이너리 제공
- ③ 애플리케이션이 GPU에 SPMD 방식으로 커널을 실행하도록 지시함.

("이 커널의 N 인스턴스 실행")

launch(myKernel, N)

흥미롭게도 이것은 그래픽 연산 drawPrimitives()보다 훨씬 간단한 연산입니다.



CUDA programming language

- 2007년에 엔비디아 테슬라 아키텍처와 함께 도입.
- GPU에서 실행되는 프로그램을 표현하는 "C와 유사한" 언어.
- 비교적 낮은 수준: CUDA의 추상화는 최신 GPU의 기능/성능 특성과 거의 일치.
(설계 목표: 낮은 추상화 거리 유지).
- ✓ 참고: OpenCL은 CUDA의 개방형 표준 버전.
- ✓ CUDA는 NVIDIA GPU에서만 실행.
- ✓ OpenCL은 여러 공급업체의 CPU 및 GPU에서 실행.
- ✓ CUDA에 대해 말하는 거의 모든 내용은 OpenCL에도 적용됨

CUDA programming language

1. CUDA 프로그래밍 추상화.
2. 최신 GPU에서 CUDA 구현.
3. GPU 아키텍처에 대한 자세한 내용.

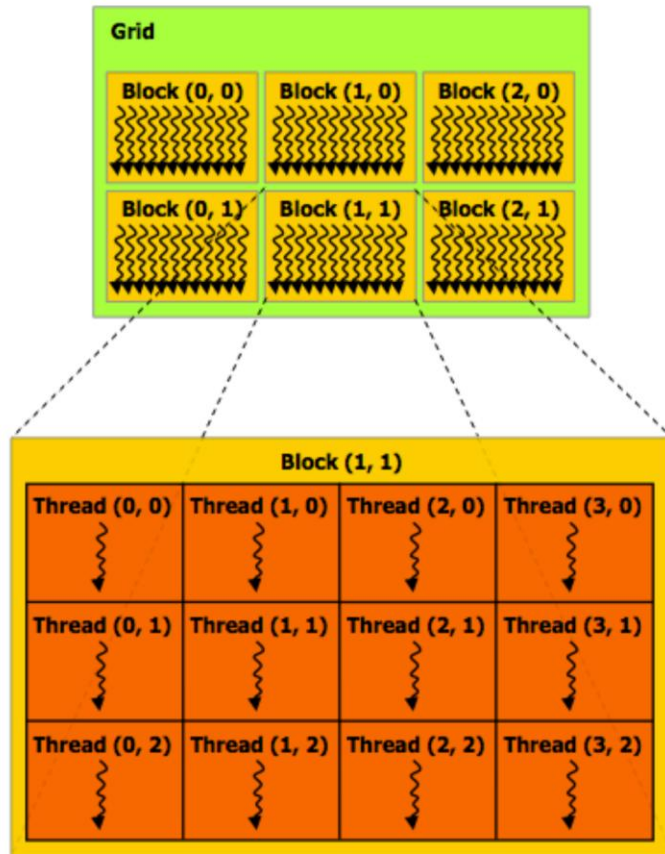
- 이 강의를 통해 고려해야 할 사항
 - ✓ CUDA는 데이터 병렬 프로그래밍 모델인가요?
 - ✓ CUDA는 공유 주소 공간 모델의 예시인가요?
 - ✓ 아니면 메시지 전달 모델의 예시일까요?
 - ✓ ISPC 인스턴스와 작업을 비유할 수 있을까요? pthread는 어떤가요?

설명 (다시 시작합니다)

- **CUDA 용어를 사용하여 CUDA 추상화를 설명**
- **특히 CUDA 스레드라는 용어를 사용할 때 주의.**
 - **CUDA 스레드는 논리적 제어 스레드에 해당한다는 점에서 pthread와 유사한 추상화를 제공하지만 CUDA 스레드의 구현은 매우 다름.**

CUDA 프로그램은 concurrent threads의 계층 구조로 구성

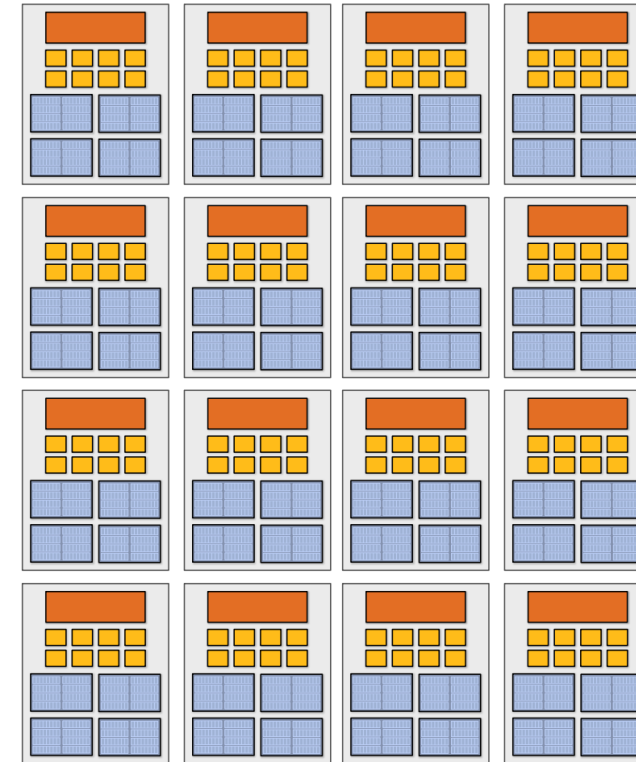
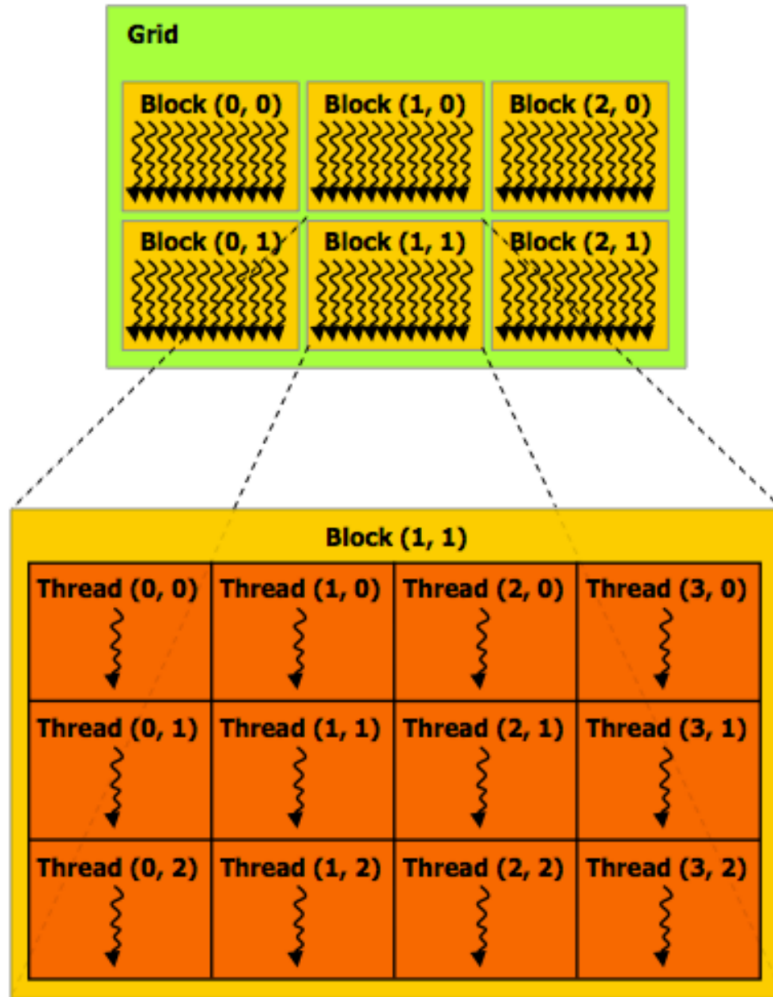
- 스레드 ID는 최대 3차원까지 가능(아래 2D 예시).
- 다차원 스레드 ID는 자연적으로 N-D인 문제에 편리함.



Regular application thread running on CPU (the "host")

```
const int Nx = 12;  
const int Ny = 6;  
  
dim3 threadsPerBlock(4, 3);  
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);  
  
// assume A, B, C are allocated Nx x Ny float arrays  
  
// this call will launch 72 CUDA threads:  
// 6 thread blocks of 12 threads each  
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

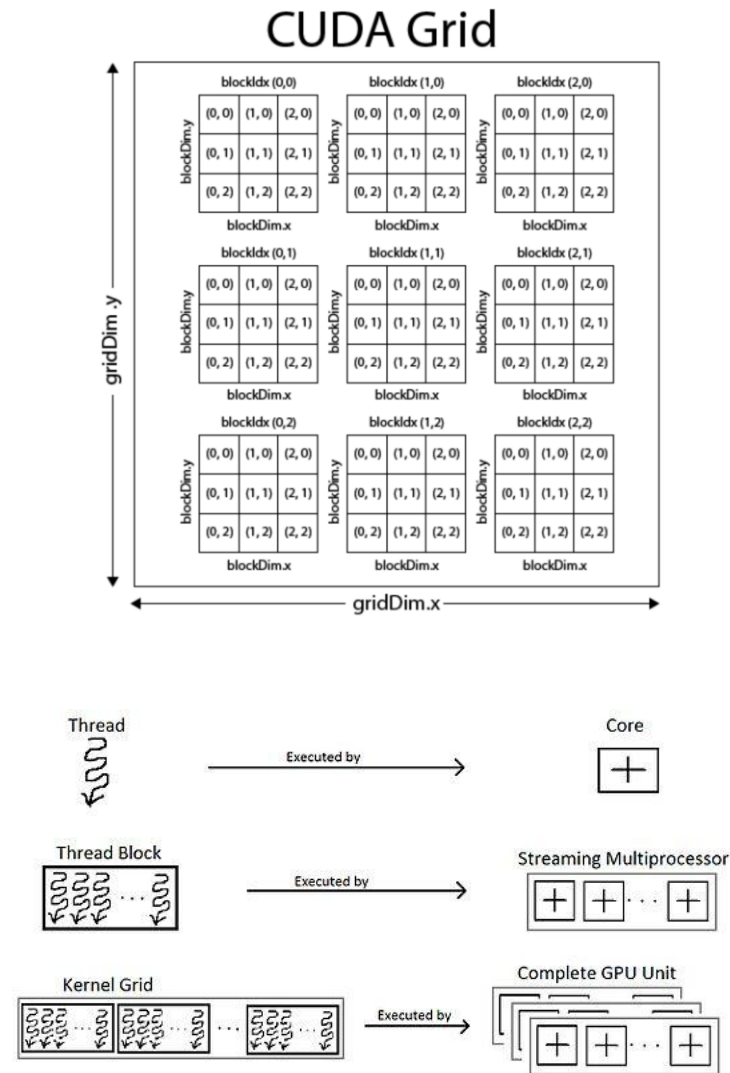
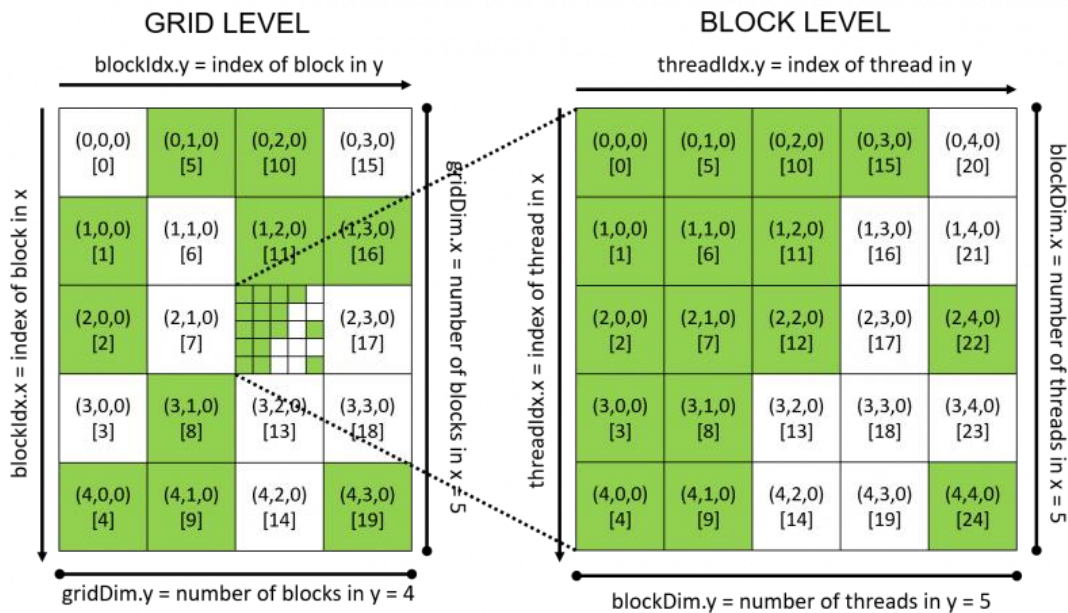
CUDA blocks map to GPU cores (streaming multiprocessors)



GPU

Grid, Block, and Thread

- **gridDim**: 그리드의 차원
- **blockIdx**: 그리드 내 block index
- **blockDim**: block의 차원
- **threadIdx**: block내 thread index



Basic CUDA syntax

“호스트” 코드 :

CPU에서 일반 C/C++ 애플리케이션의
일부로 실행되는직렬 실행

많은 CUDA 스레드의 대량 실행
“CUDA 스레드 블록 그리드 실행”
모든 스레드가 종료되면 호출이 반환.

Regular application thread running on CPU (the “host”)

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/
threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will launch 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

SPMD execution of device kernel function:

“CUDA device” code: kernel function (**__global__** denotes a CUDA kernel
function) runs on GPU

각 스레드는 블록 내 자신의 위치(threadIdx)와 그리드 내 블록의
위치(blockIdx)에서 전체 그리드 스레드 ID를 계산(유일성)

CUDA kernel definition

```
// kernel definition (runs on GPU)
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + B[j][i];
}
```

Basic CUDA syntax

```
#include <stdio.h>
```

```
_device__ void device_strcpy(char *dst, const char *src) {  
    while (*dst++ = *src++);  
}
```

```
_global__ void kernel(char *A) {  
    device_strcpy(A, "Hello, World!");  
}
```

```
int main() {  
    char *d_hello;  
    char hello[32];  
  
    cudaMalloc((void**)&d_hello, 32);  
  
    kernel<<<1,1>>>(d_hello);  
  
    cudaMemcpy(hello, d_hello, 32, cudaMemcpyDeviceToHost);  
  
    cudaFree(d_hello);  
  
    puts(hello);  
}
```


Basic CUDA syntax

호스트와 디바이스 코드의 명확한 분리

실행을 호스트 코드와 디바이스 코드로 분리하는 것은 프로그래머가 정적으로 수행.

“Host” code : serial execution on CPU

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAddDoubleB<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

“Device” code (SPMD execution on GPU)

```
__device__ float doubleValue(float x)
{
    return 2 * x;
}

// kernel definition
__global__ void matrixAddDoubleB(float A[Ny][Nx],
                                float B[Ny][Nx],
                                float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + doubleValue(B[j][i]);
}
```

호스트와 디바이스 코드의 명확한 분리

```
#include <stdio.h>
```

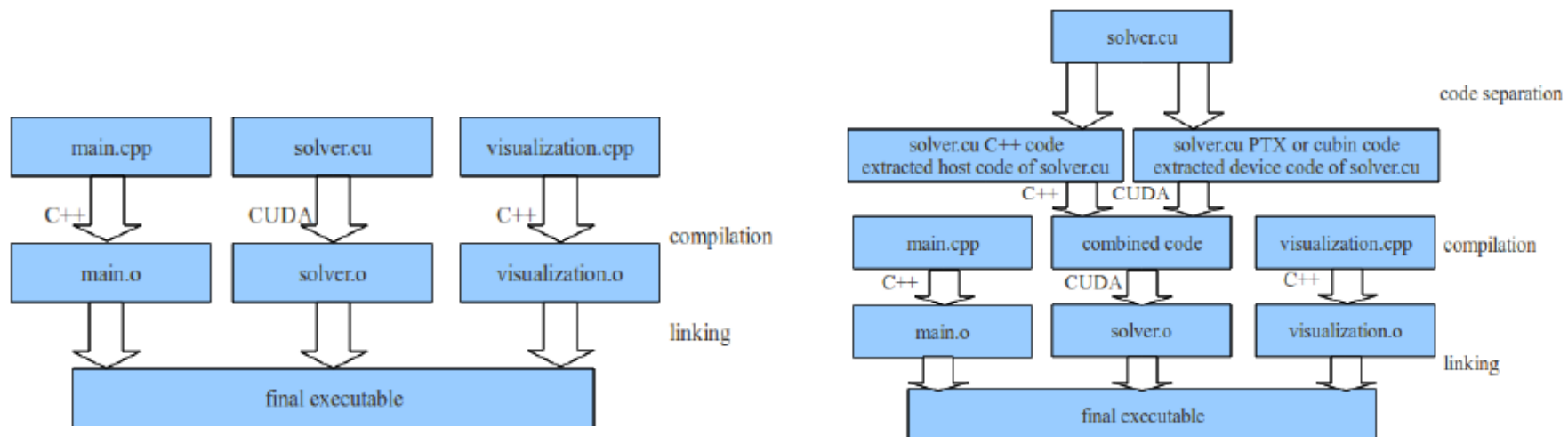
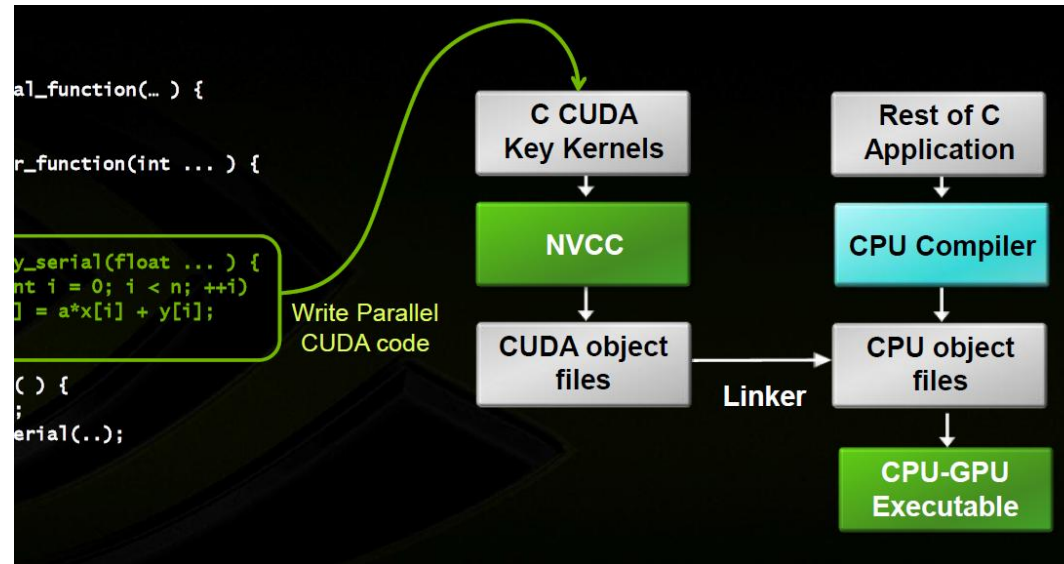
```
__device__ void device_strcpy(char *dst, const char *src) {  
    while (*dst++ = *src++);  
}
```

```
__global__ void kernel(char *A) {  
    device_strcpy(A, "Hello, World!");  
}
```

```
int main() {  
    char *d_hello;  
    char hello[32];  
  
    cudaMalloc((void**)&d_hello, 32);  
  
    kernel<<<1,1>>>(d_hello);  
  
    cudaMemcpy(hello, d_hello, 32, cudaMemcpyDeviceToHost);  
  
    cudaFree(d_hello);  
  
    puts(hello);  
}
```

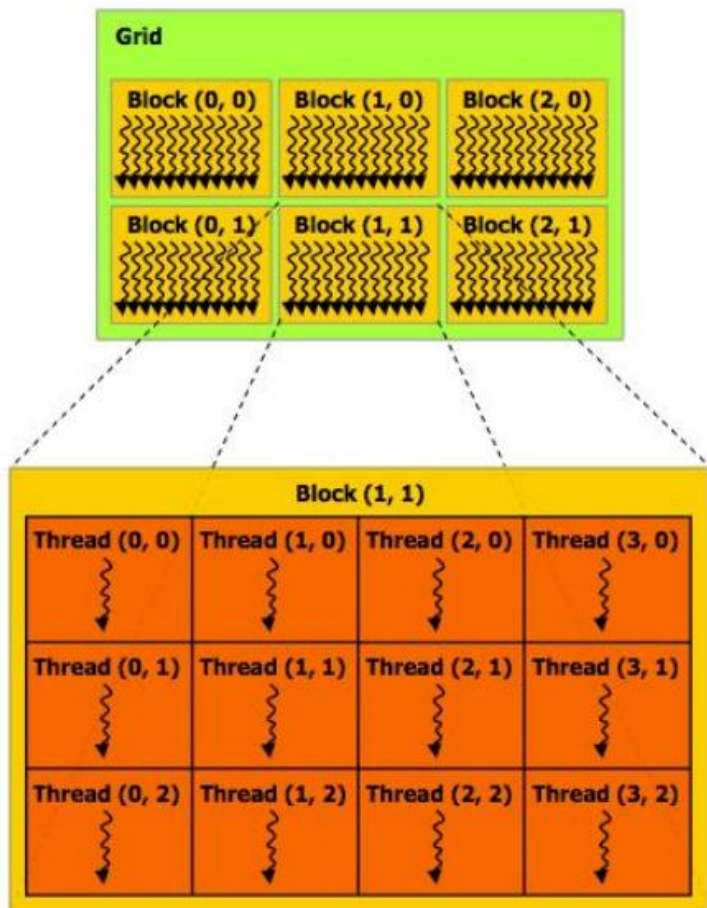
호스트와 디바이스 코드의 명확한 분리

실행을 호스트 코드와 디바이스 코드로 분리하는 것은 프로그래머가 정적으로 수행.



SPMD 스레드 수는 프로그램에 명시

- 커널 호출 횟수는 데이터 수집 크기에 따라 결정되지 않음.
- (kernel launch은 그래픽 셰이더 프로그래밍의 경우처럼 맵(커널, 컬렉션)이 아님).



Regular application thread running on CPU (the "host")

```
const int Nx = 11; // not a multiple of threadsPerBlock.x
const int Ny = 5;  // not a multiple of threadsPerBlock.y

dim3 threadsPerBlock(4, 3);
dim3 numBlocks((Nx+threadsPerBlock.x-1)/threadsPerBlock.x,
               (Ny+threadsPerBlock.y-1)/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

CUDA kernel definition

```
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // guard against out of bounds array access
    if (i < Nx && j < Ny)
        C[j][i] = A[j][i] + B[j][i];
}
```

CUDA execution model

Host
(serial execution)



Implementation: CPU

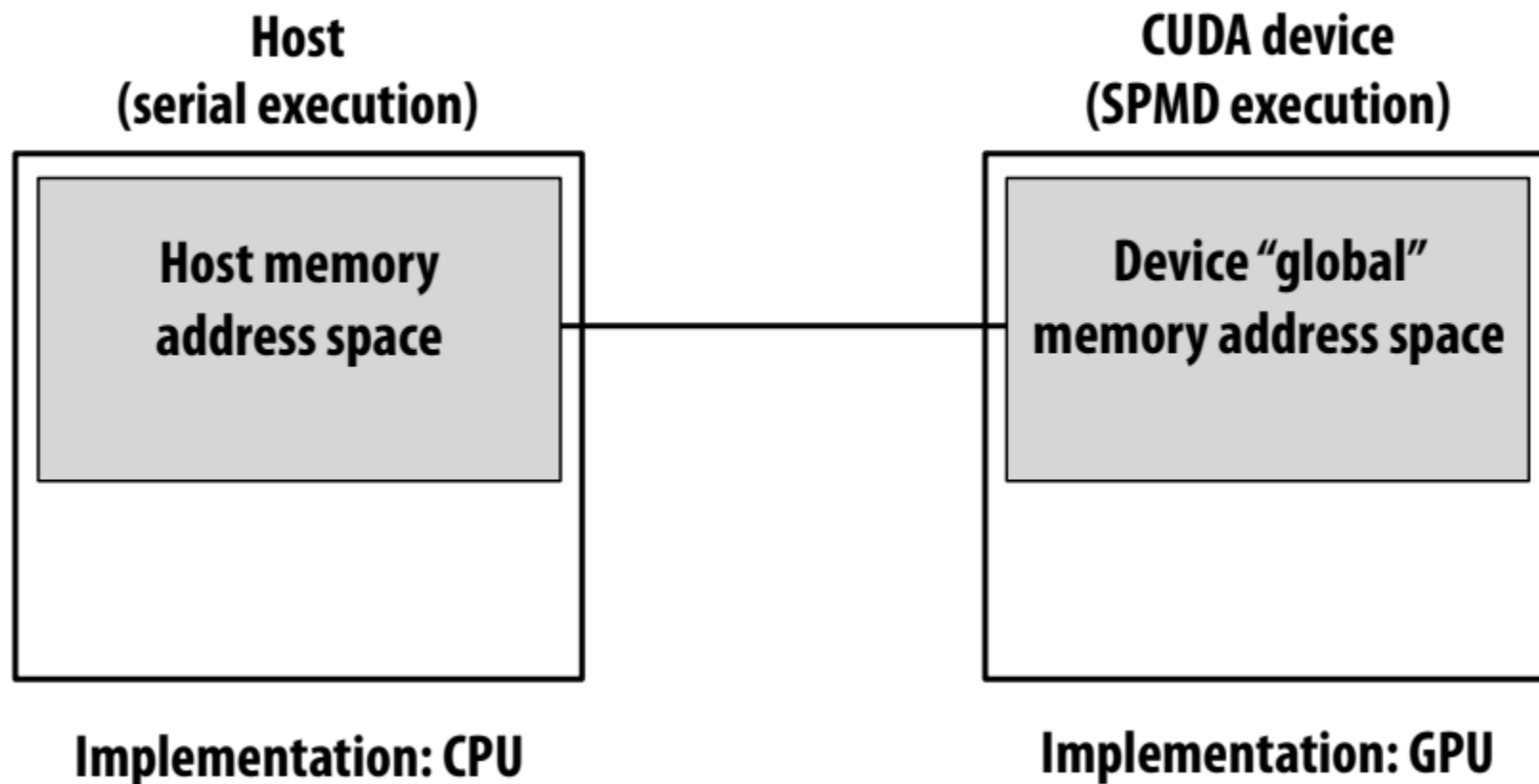
CUDA device
(SPMD execution)



Implementation: GPU

CUDA memory model

고유한 호스트 및 디바이스 주소 공간



memcpy primitive

Move data between address spaces

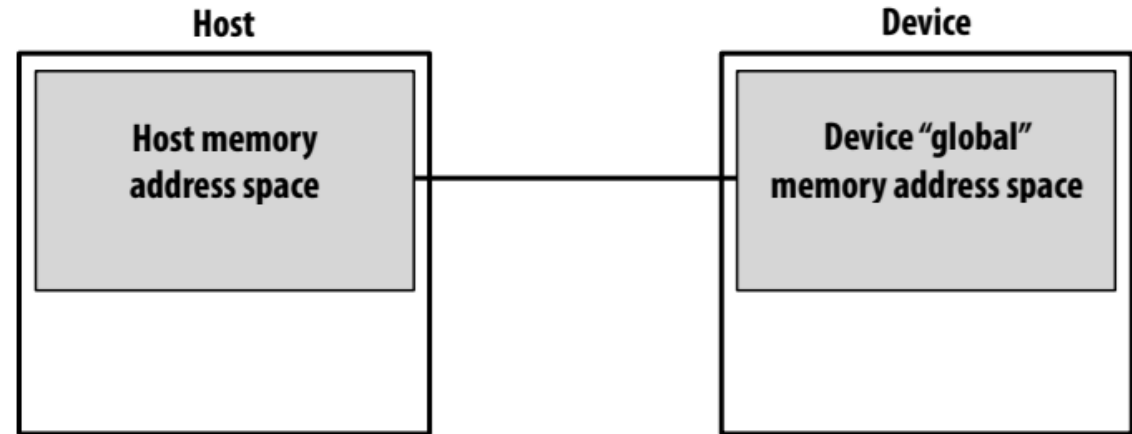
```
float* A = new float[N]; // allocate buffer in host mem

// populate host address space pointer A
for (int i=0; i<N; i++)
    A[i] = (float)i;

int bytes = sizeof(float) * N;
float* deviceA; // allocate buffer in
cudaMalloc(&deviceA, bytes); // device address space

// populate deviceA
cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);

// note: directly accessing deviceA[i] is an invalid
// operation here (cannot manipulate contents of deviceA
// directly from host, since deviceA is not a pointer
// into the host's address space)
```



cudaMemcpy를 떠올리면 어떤 것이 떠오르나요?

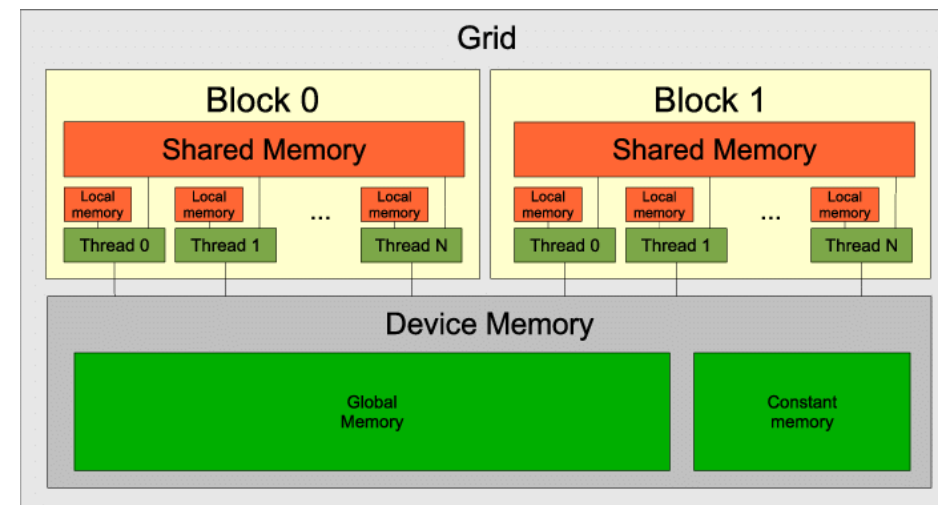
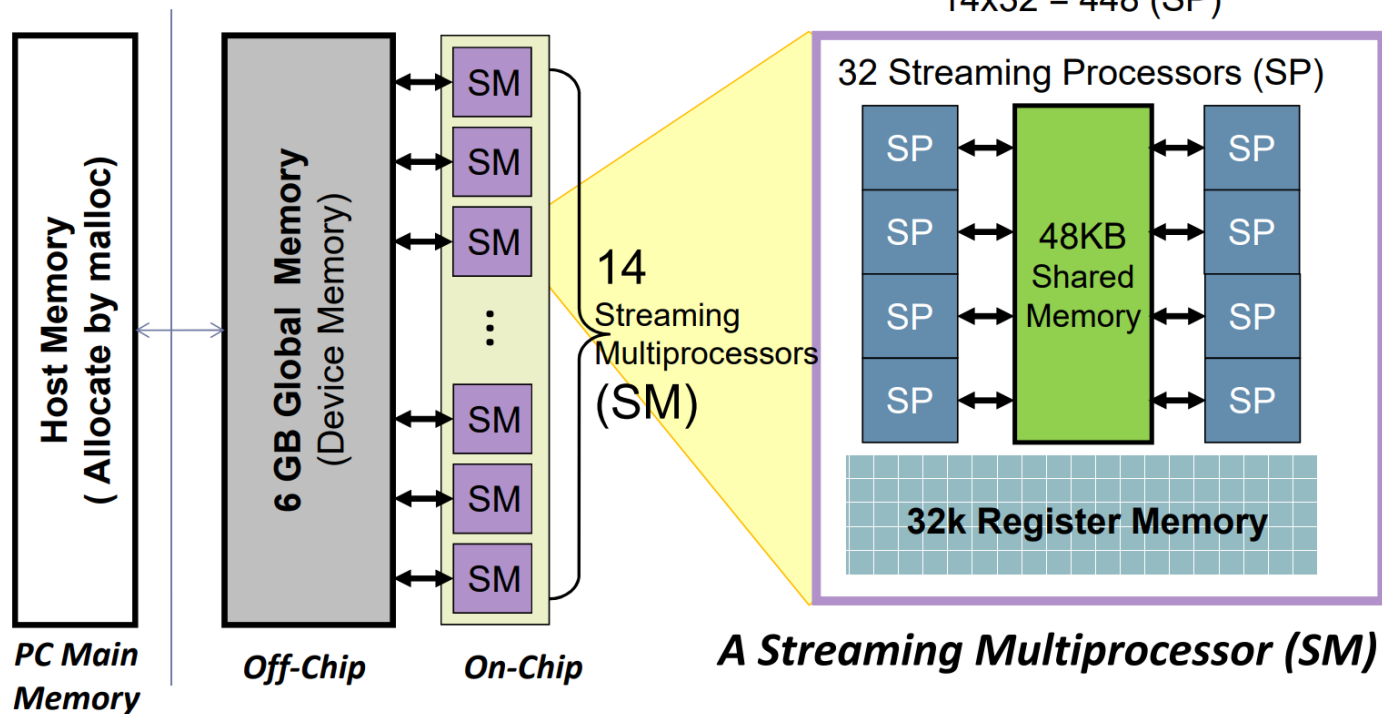
CUDA memory model

고유한 호스트 및 디바이스 주소 공간

GPU 메모리 구조

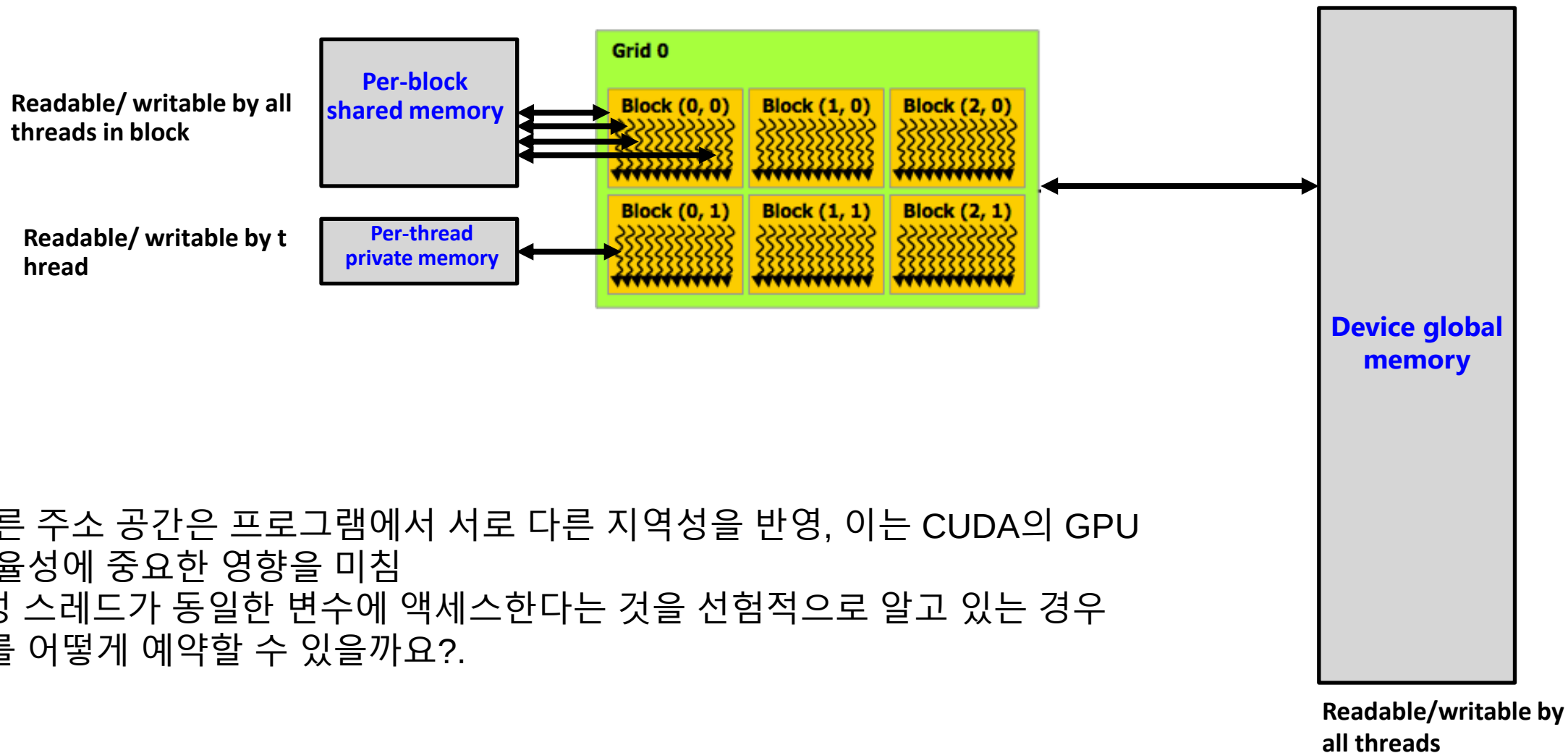
- ▶ Highly parallel, multithreaded, many-core processor

14x32 = 448 (SP)



CUDA device memory model

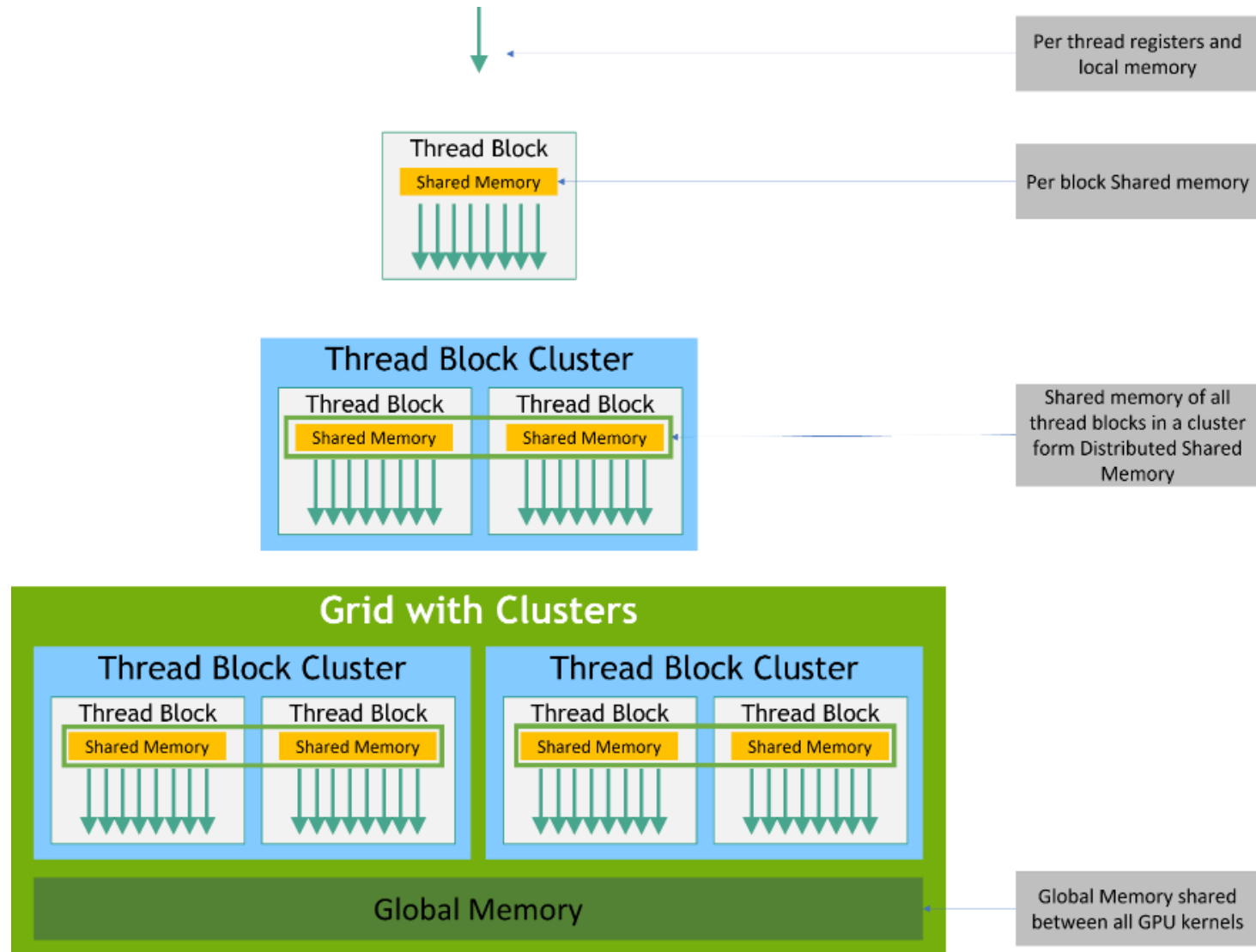
커널에 표시되는 세 가지 메모리 유형



- 서로 다른 주소 공간은 프로그램에서 서로 다른 지역성을 반영, 이는 CUDA의 GPU 구현 효율성에 중요한 영향을 미침
- (예: 특정 스레드가 동일한 변수에 액세스한다는 것을 선형적으로 알고 있는 경우 스레드를 어떻게 예약할 수 있을까요?).

CUDA device memory model

프로그래머가 메모리 계층구조를 직접 제어가능



CUDA device memory model 특징

◆ Per-Thread (local Memory)

•Registers

- On-chip memory
- 가장 빠르고, 가장 작은 메모리
- 보통 블록 1개가 8k-64k 32bit register를 사용한다.
- 각 SM 내부에 존재
- thread들이 register를 나누어서 가져간다.(register의 영역을 나누어서 사용) 따라서 context switching이 필요 없다.

•Local Memory

- Off-chip memory
- SM 내부에 없고, DRAM 영역 내부에 존재
- register에 다 올라가지 못하는 것들은 Local Memory에 올려서 동작
- Register에 비해 느리지만 더 많은 공간을 활용할 수 있다.(크게 제약이 없다.)

CUDA device memory model 특징

◆ Per-Block (shared Memory)

- Shared Memory(__shared__)
 - ✓ On-chip memory
 - ✓ SM 내부에 존재해 빠르지만 공간이 적다.
 - ✓ 블록 안의 모든 thread들이 공유한다.(모든 thread가 접근이 가능하다.)
 - ✓ thread 간의 communication 가능
 - ✓ SM 내부의 block들이 shared memory 공간을 나누어서 활용(block 개수 제한)

CUDA device memory model 특징

◆ Per-Gird (global Memory)

- Global Memory
 - ✓ 모든 thread가 활용할 수 있다.
 - ✓ off-chip memory
 - ✓ 가장 느리지만 가장 큰 공간을 가진다.
 - ✓ Host에서 접근이 가능하다.
- Constant Memory
 - ✓ 읽기 전용 메모리 : 읽을 수만 있다.
 - ✓ 선언은 global로 하고 host를 통해 초기화 한다.
 - ✓ 공간이 적지만 빠르다.
 - ✓ 이 영역을 위한 캐시가 존재한다.(캐시 영역 지원)
 - ✓ 기본적으로 DRAM에 저장되어 있지만 할당되어 있는 cache가 존재해서 자주 읽어오는 값들을 저장해 놓으면 빠르게 가져가 활용할 수 있다.(인자 값과 같은 것들..)
- Texture Memory
 - ✓ Constant memory와 비슷
 - ✓ 2차원 array를 다루는 것에 최적화
 - ✓ Hardware 필터링 제공
 - ✓ 기본적으로 graphic에 활용되므로 cuda에서 많이 사용하지는 않는다.

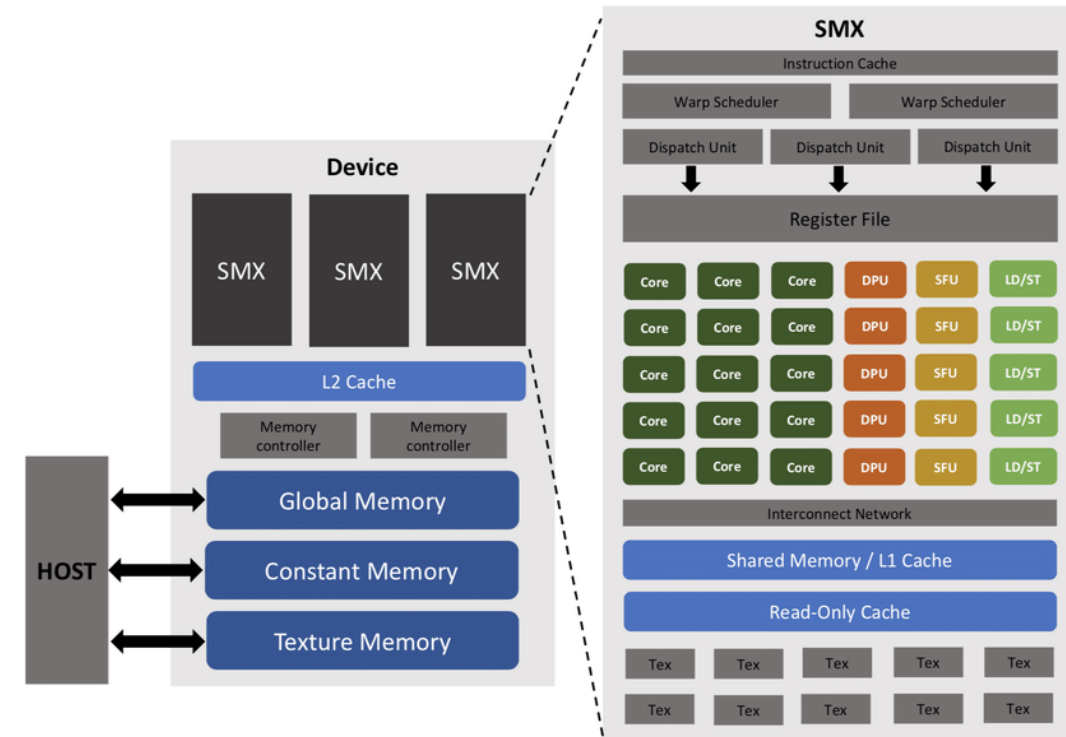
GPU Caches (Hardware Manage)

◆ L1 cache

- SM마다 존재한다.
- Shared memory와 같은 공간에 존재
- Shared memory 공간으로 활용하기도 한다.
- 따라서 L1 cache를 더 사용할 수도 사용하지 않을 수도 있다.

◆ L2 cache

- SM 내부에 있는 것이 아니라 바깥쪽에 존재하는 캐시
- Shared by all SM
- Read-only constant and texture caches



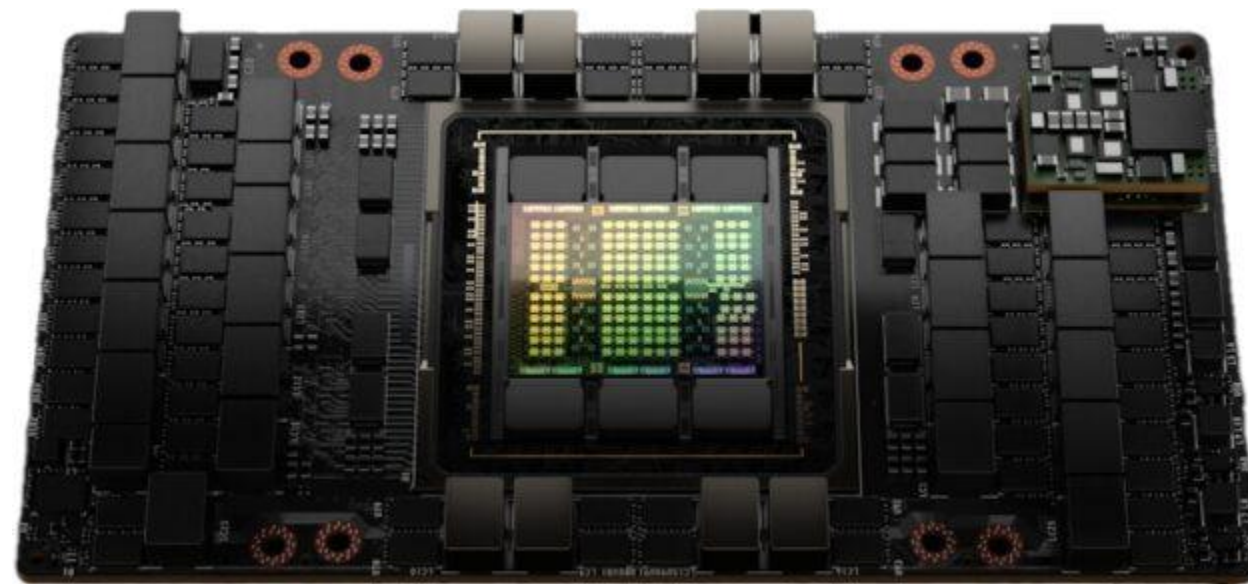
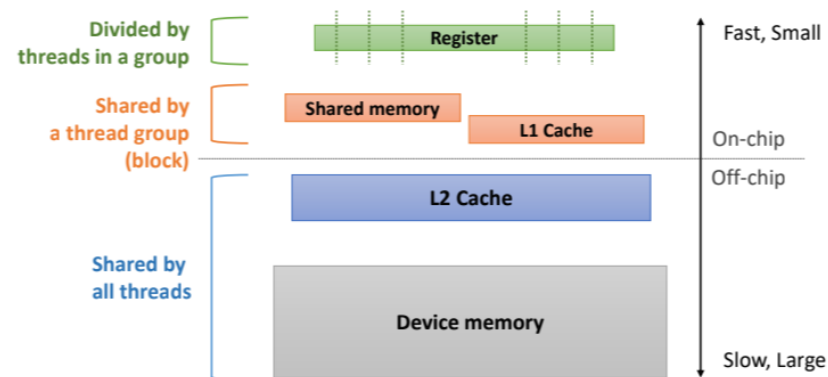
Physical Location of CUDA Memories

◆ On-chip memory(In each SM)

- ✓ Registers
- ✓ Shared Memory
- ✓ Constant cache, Texture cache

◆ Off-chip memory

- ✓ Local memory
- ✓ Global memory
- ✓ Constant memory and Texture memory
- ✓ L2 cache



Memory Model 성능에 대한 특징

- 일단 각각 Memory Model을 잘 이해하고 적당한 공간에 가져다가 사용해야 한다.
(Design your kernel and block)
- 무작정 많은 스레드를 사용한다고 성능이 좋아지는 것은 아니다.
- 결론적으로 Active Wrap과 Active Block이 많아야 성능이 좋아질 수 있다.

◆ Active Wrap

- 일부 thread가 요구하는 register 공간 할당받지 못할 수 있다.
 - ✓ $(\# \text{thread in a block}) * (\text{register per thread}) > \text{registers in a SM}$
- Active warp
 - ✓ wrap 내부의 모든 스레드들이 요구하는 register 공간을 가져야 한다.
- Active warp을 늘리기 위해 적당한 register 공간을 배분해야 한다.

◆ Active Block

- 하나의 블록에서 메모리 자원이 요구되는 것들을 모두 가지고 있는 것(?)
 - ✓ Register - all warps in a block are active wrap
 - ✓ Shared memory space
- SM 내의 Active blocks은 동시에 실행될 수 있다.
- Active Block을 늘리기 위해 적당한 Shared memory 공간을 배분해야 한다.

Occupancy(얼마나 가득 차 있는가)

◆ = (# of active warp) / (# of maximum warps)

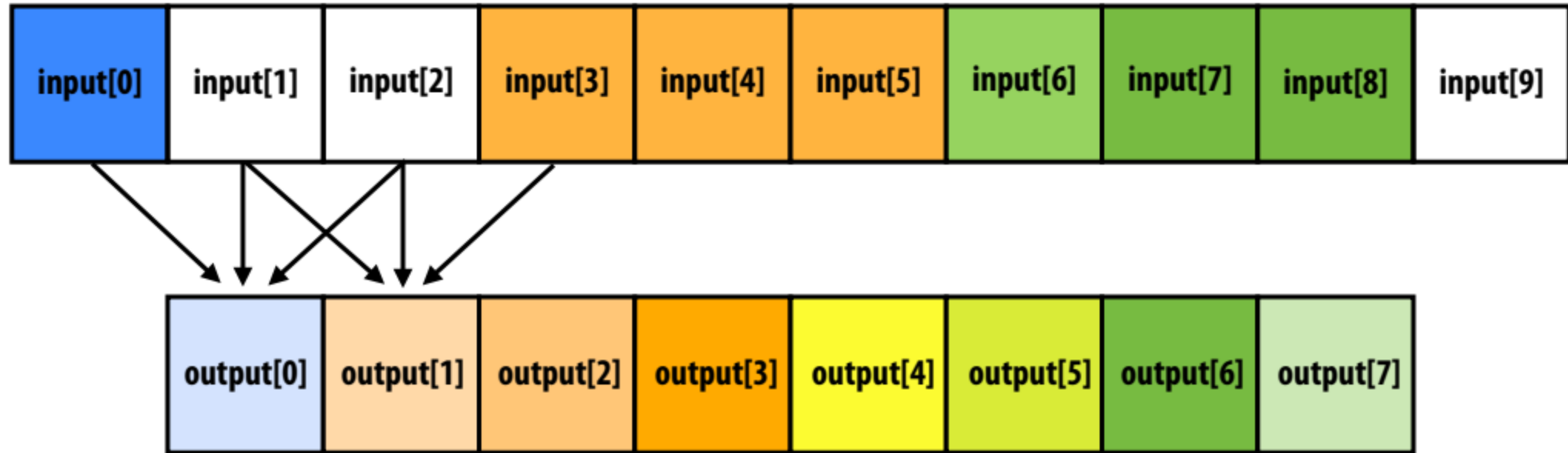
◆ 이론적으로 가능한 warp에 비해 active warp이 얼마나 존재하는가

◆ 많은 active block이 있을수록 높은 성능의 병렬 처리가 될 수 있는 확률이 높다.

◆ kernal과 thread layout을 최대한 많은 active warp을 낼 수 있도록 설계해야 한다.

- ✓ thread당 register의 수
- ✓ block당 thread의 수
- ✓ block당 shared memory 크기

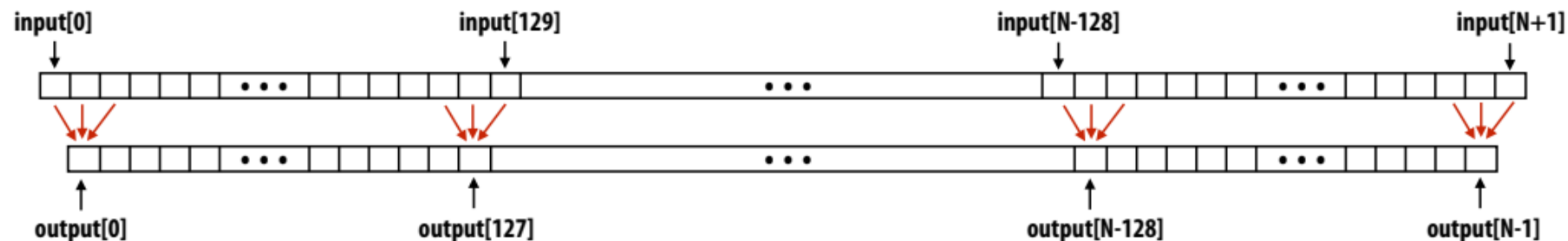
CUDA example: 1D convolution



```
output[i] = (input[i] + input[i+1] + input[i+2]) / 3.f;
```

1D convolution in CUDA (version 1)

출력 요소당 하나의 스레드



CUDA Kernel

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += input[index + i];

    output[index] = result / 3.f;
}
```

each thread computes
result for one element

each thread writes result
to global memory

Host code

```
int N = 1024 * 1024;
cudaMalloc(&devInput, sizeof(float) * (N+2) ); // allocate input array in device memory
cudaMalloc(&devOutput, sizeof(float) * N);      // allocate output array in device memory

// properly initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

1D convolution in CUDA (version 2)

One thread per output element: 메모리 계층별 input data in per-block shared memory

CUDA Kernel

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    __shared__ float support[THREADS_PER_BLK+2]; // per-block allocation
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK + threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result / 3.f;
}
```

모든 스레드가 블록의
"support 영역"을 글로벌
메모리에서 공유 메모리로
협력적으로 로드.

(total of 130 load instructions
instead of 3 * 128 load instructions)

barrier (all threads in block)

each thread computes
result for one element

write result to global
memory

Host code

```
int N = 1024 * 1024
cudaMalloc(&devInput, sizeof(float) * (N+2)); // allocate array in device memory
cudaMalloc(&devOutput, sizeof(float) * N); // allocate array in device memory

// property initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>>(N, devInput, devOutput);
```

CUDA synchronization constructs

- **__syncthreads()**

- Barrier: 블록의 모든 스레드가 이 지점에 도착할 때까지 대기

- **Atomic operations**

- e.g., `float atomicAdd(float* addr, float amount)`
- 전역 메모리와 공유 메모리 변수 모두에 대한 원자 연산

- **Host/device synchronization**

- 커널 반환 시 모든 스레드에 대한 암시적 배리어(barrier)

Summary: CUDA abstractions

- **Execution: thread hierarchy**

- 많은 스레드의 일괄 실행(정확하지 않은 표현.. 나중에 설명)
- 2단계 계층 구조: 스레드가 스레드 블록으로 그룹화됨

- **Distributed address space**

- 호스트와 디바이스 주소 공간 간에 복사할 수 있는 Built-in memcpy primitives
- 세 가지 유형의 디바이스 주소 공간
- 스레드당, 블록당('공유') 또는 프로그램당('전역').

- 스레드 블록의 스레드에 대한 **Barrier synchronization primitive.**

- 추가 동기화를 위한 원자 프리미티브(공유 및 전역 변수)

CUDA semantics

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    __shared__ float support[THREADS_PER_BLK+2]; // per-block allocation
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local var

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result / 3.f;
}

// host code ////////////////////////////////////////
int N = 1024 * 1024;
cudaMalloc(&devInput, N+2); // allocate array in device memory
cudaMalloc(&devOutput, N); // allocate array in device memory

// property initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>>(N, devInput, devOutput);
```

pthread_create() 또는 std::thread() 호출을 구현하는 것을 고려:

스레드 상태를 할당:

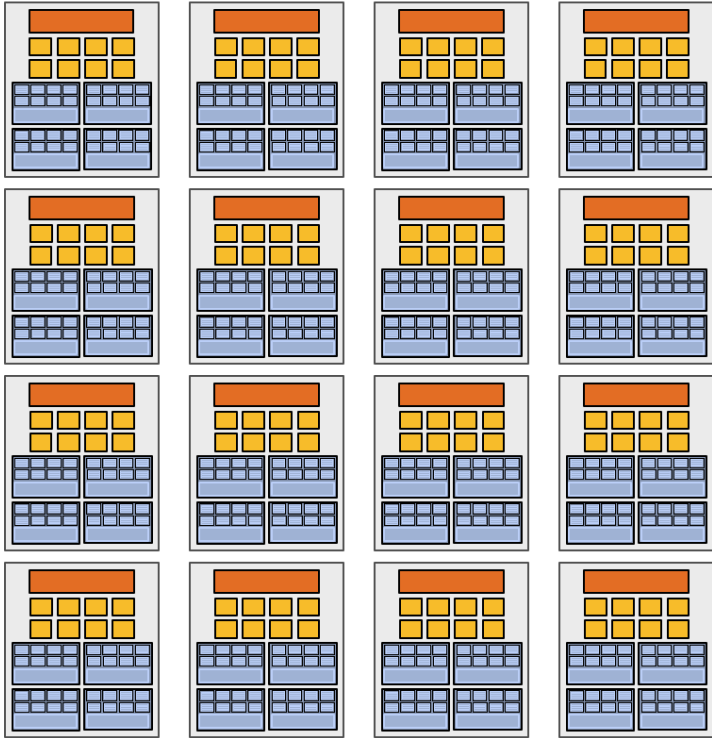
- 스레드를 위한 스택 공간
- OS가 스레드를 예약할 수 있도록 제어 블록을 할당.

이 CUDA 프로그램을 실행하면 로컬 변수/스택의 인스턴스가 1백만 개 생성되나요?

공유 변수의 8K 인스턴스(지원)

1백만 개 이상의 CUDA 스레드(8K 스레드 블록 이상)를 실행.

Assigning work



High-end GPU
(16+ cores)



Mid-range GPU
(6 cores)

- **CUDA 프로그램이 수정 없이 이 모든 GPU에서 실행되는 것이 바람직함**
- **참고: 제가 보여드린 CUDA 프로그램에는 num_cores라는 개념이 없음.**
- **(CUDA 스레드 실행은 데이터 병렬 모델 예제에서 forall loop와 비슷한 개념.)**

CUDA compilation

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    __shared__ float support[THREADS_PER_BLK+2]; // per block allocation
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local var

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result;
}
```

```
int N = 1024 * 1024;
cudaMalloc(&devInput, N+2); // allocate array in device memory
cudaMalloc(&devOutput, N); // allocate array in device memory

// property initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

A compiled CUDA device binary includes:

- Program text (instructions)
- Information about required resources:
 - 128 threads per block
 - B bytes of local data per thread
 - $128+2^7=130$ floats (520 bytes) of shared space per thread block

— launch 8K thread blocks

CUDA thread-block assignment

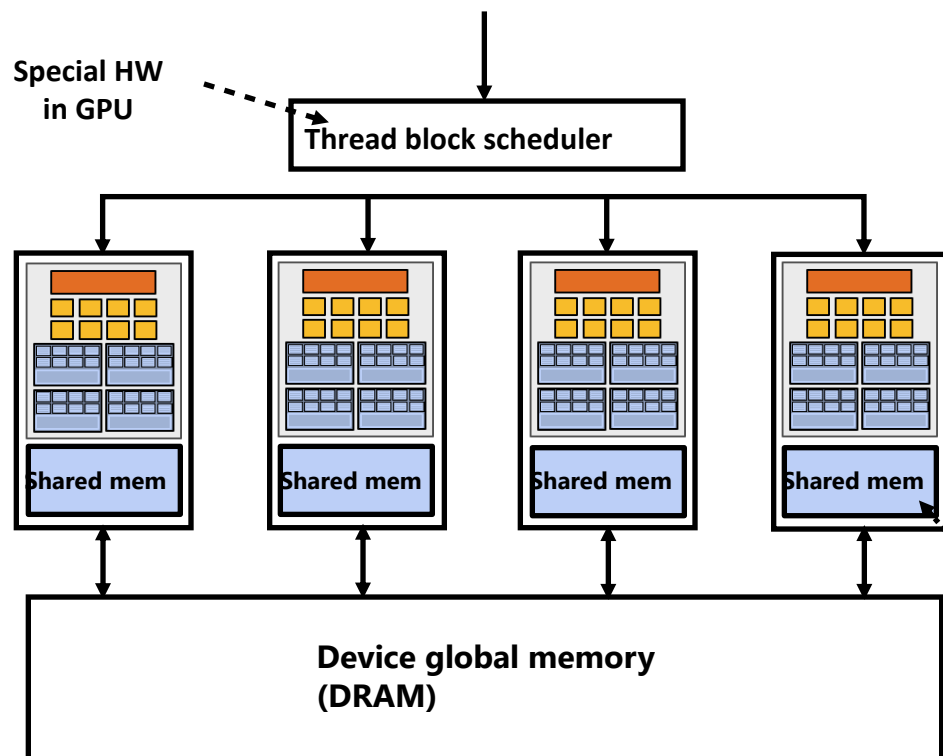


Grid of 8K(8000) convolve thread blocks (커널 실행 시 지정)

블록 리소스(Block resource) 요구 사항:
(컴파일된 커널 바이너리에 포함)

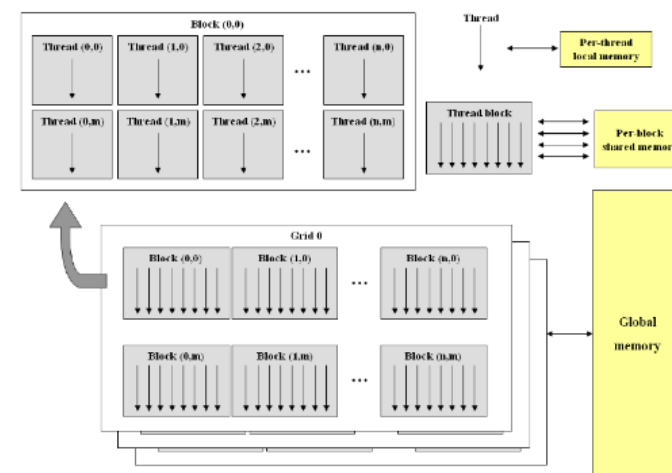
- 128 스레드
- 520바이트의 공유 메모리
- (128 x B) 바이트의 로컬 메모리

Kernel launch command from host
launch(blockDim, convolve)



CUDA의 주요 가정: thread blocks 실행은 어떤 순서로든 수행될 수 있음(블록 간 종속성 없음).

GPU 구현은 리소스 요구 사항을 존중하는 동적 스케줄링 정책을 사용하여 thread blocks ("작업")을 코어에 매핑.



Shared mem is fast
on-chip memory

Dynamic Scheduling 예제

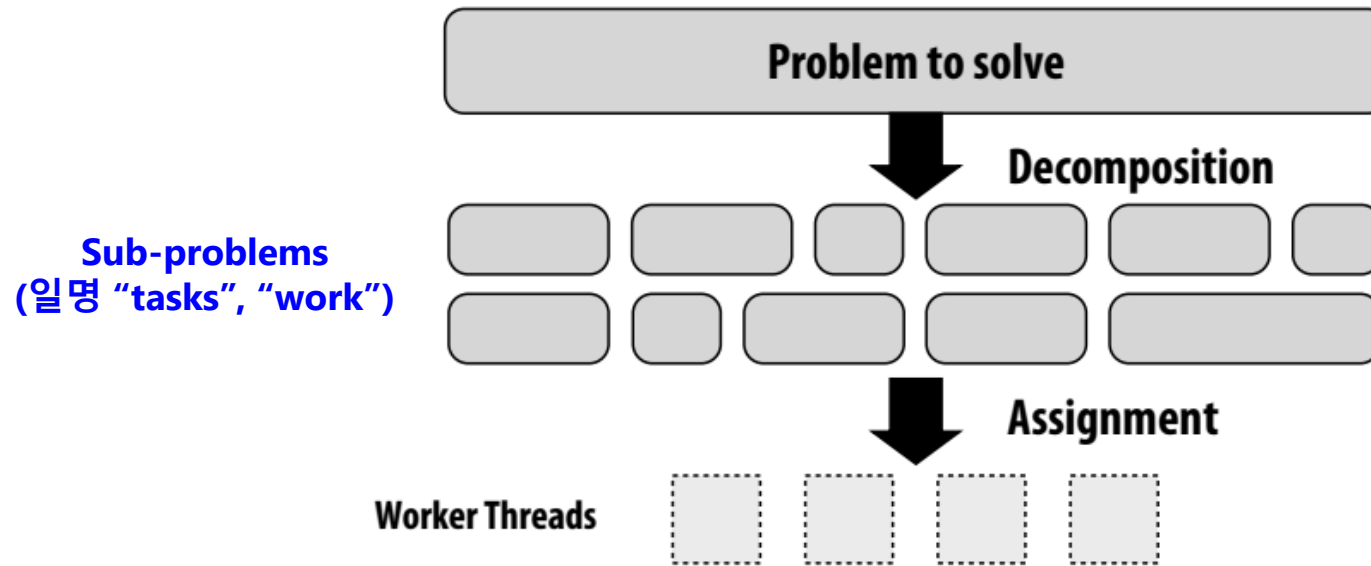
data flow와 exception behavior를 유지하는 범위 내에서 하드웨어적으로 instructions를 re-scheduling하여 stall을 줄이는 방법이다.

Dynamic Scheduling 예제

```
DIVD  F0, F2, F4  
ADDD  F10, F0, F8  
SUBD  F12, F8, F14
```

DIVD와 ADDD 사이에는 RAW dependence(F0)가 존재하고, SUBD는 위의 두 명령어와 아무런 dependence가 존재하지 않는다. 따라서 SUBD는 먼저 실행되어도 상관 없다. 즉, 이러한 명령어들은 out-of-order execution과 out-of-order completion이 가능하다.

일반적인 디자인 패턴의 또 다른 예시: worker "threads" POOL



모범 사례: 병렬 머신을 "채울" 만큼의 일꾼(Worker)만 만들고 그 이상은 만들지 말자:

- ✓ 병렬 실행 리소스당 하나의 일꾼(Worker)(예: CPU 코어, 코어 실행 컨텍스트)
- ✓ 코어당 N개의 일꾼(Worker)가 필요할 수 있음(여기서 N은 메모리/IO 지연 시간을 숨길 수 있을 만큼 충분히 큼).
- ✓ 각 일꾼(Worker)에 대한 리소스 사전 할당
- ✓ 일꾼(Worker) 스레드에 작업을 동적으로 할당(여러 작업에 대한 할당 재사용)

기타 예시

- ISPC의 작업 시작 구현
 - ✓ CPU에서 각 하이퍼 스레드마다 하나의 pthread를 생성. 나머지 프로그램 동안 스레드가 유지됨
- 웹 서버의 스레드 풀
 - ✓ 스레드 수는 미결 요청 수가 아닌 코어 수의 함수.
 - ✓ 웹 서버 시작 시 생성된 스레드는 작업이 도착할 때까지 대기.

NVIDIA V100 SM “sub-core”

 = SIMD fp32 functional unit,
control shared across 16 units
(16 x MUL-ADD per clock *)

 = SIMD int functional unit,
control shared across 16 units
(16 x MUL/ADD per clock *)

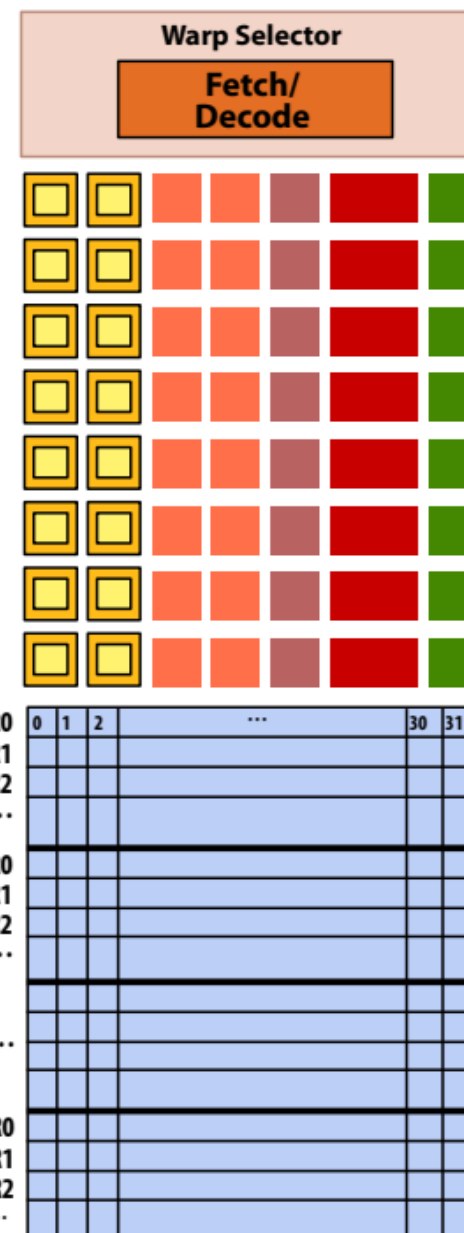
 = SIMD fp64 functional unit,
control shared across 8 units
(8 x MUL/ADD per clock **)

 = Tensor core unit

 = Load/store unit

* one 32-wide SIMD operation every two clocks

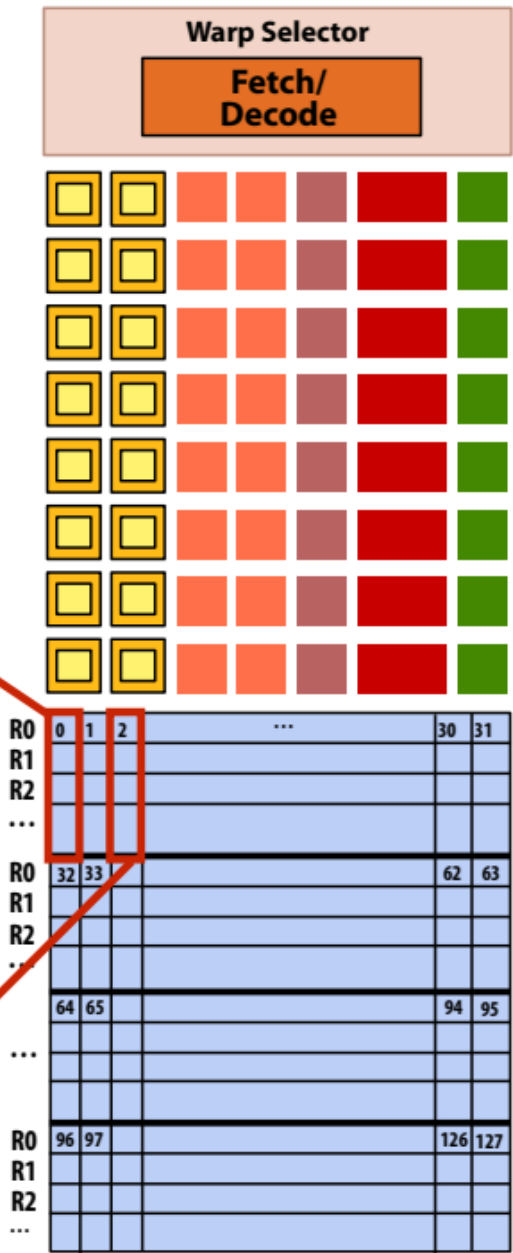
** one 32-wide SIMD operation every four clocks



NVIDIA V100 SM “sub-core”

Scalar registers for one CUDA thread: R0, R1, etc...

Scalar registers for another CUDA thread: R0, R1, etc...

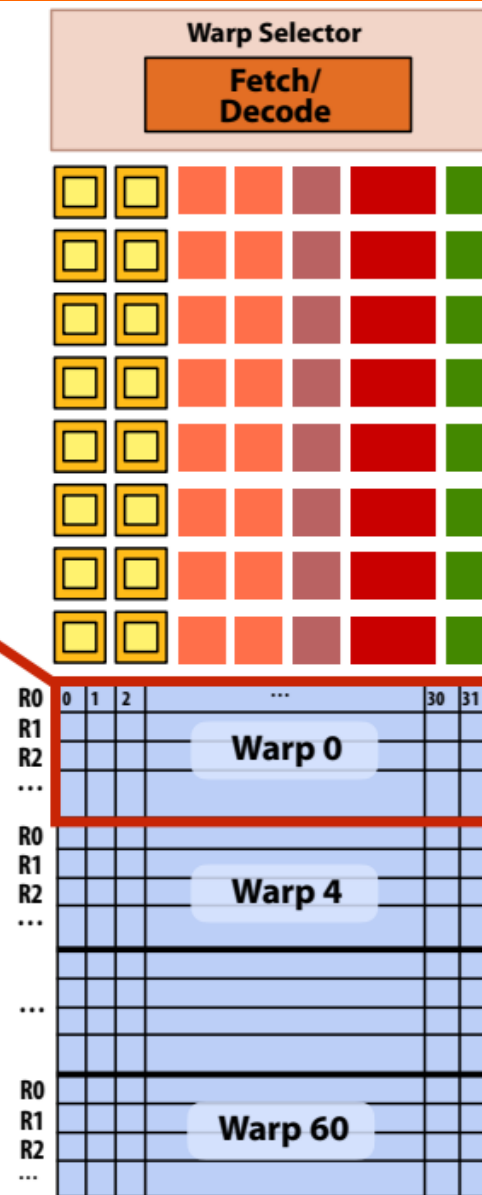


NVIDIA V100 SM "sub-core"

동일한 "워프"의 32개 스레드에 대한
스칼라 레지스터

스레드 블록의 32개 스레드 그룹을 워프라고 함.

- 스레드 블록에서 스레드 0-31은 동일한 워프에 속함(스레드 32-63 등도 마찬가지)
- 따라서 256개의 CUDA 스레드가 있는 스레드 블록은 8개의 워프에 매핑.
- V100의 각 서브코어는 최대 16개의 워프 실행을 스케줄링하고 인터리빙할 수 있음.



NVIDIA V100 SM "sub-core"

워프의 스레드는 SIMD 방식으로 실행.

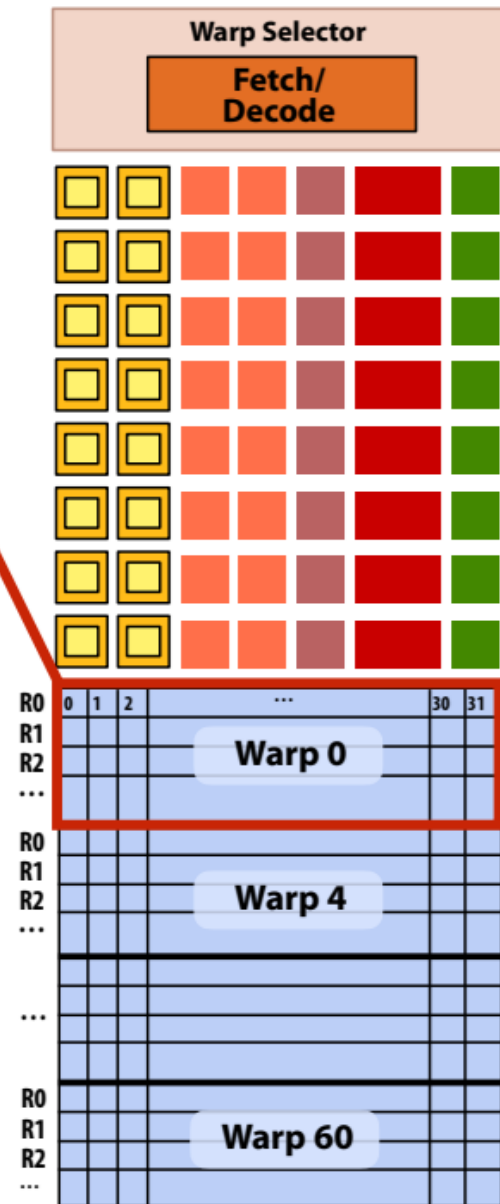
동일한 명령어를 공유하는 경우

- NVIDIA는 이를 SIMT(단일 명령어 다중 CUDA 스레드)라고 함.
- 32개의 CUDA 스레드가 동일한 명령어를 공유하지 않으면 서로 다른 실행으로 인해 성능이 저하될 수 있음.
- 이 매핑은 ISPC가 프로그램 인스턴스를 Gang*으로 실행하는 방식과 유사

워프는 CUDA의 일부가 아니지만 최신 NVIDIA GPU에서 중요한 CUDA 구현 세부 사항임.

* 하지만 GPU 하드웨어는 32개의 독립적인 CUDA 스레드가 하나의 명령을 공유하는지를 동적으로 확인하고, 이것이 사실이면 32개의 스레드를 모두 SIMD 방식으로 실행함.
CUDA 프로그램은 ISPC 갱처럼 SIMD 명령어로 컴파일되지 않음.

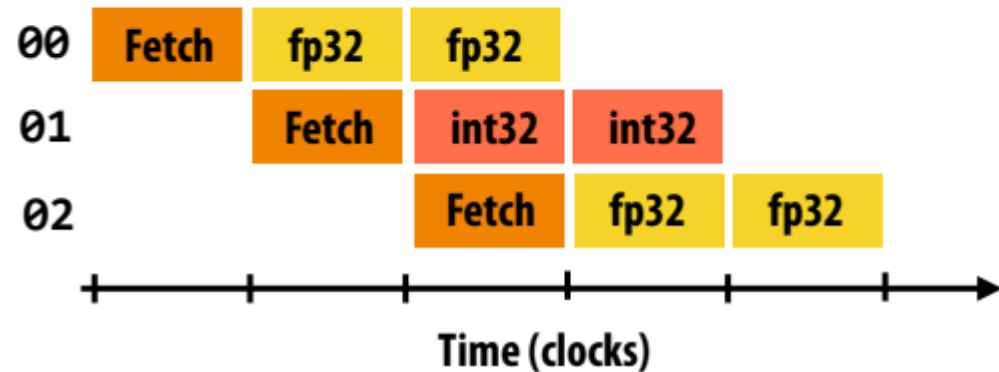
동일한 "워프"의 32개 스레드에 대한 스칼라 레지스터



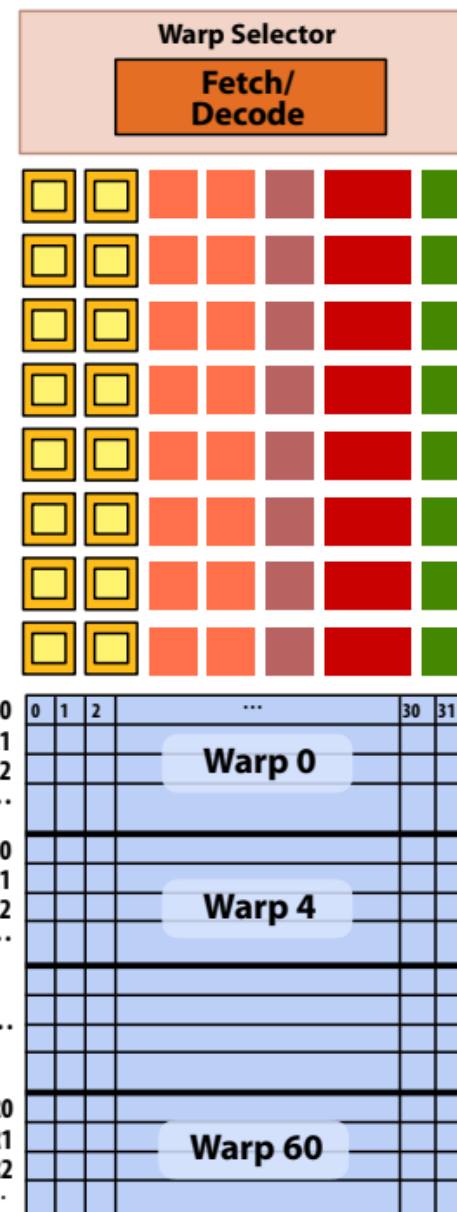
Instruction execution

워프에서 CUDA 스레드에 대한 명령 스트림...
(이 예제에서는 모든 명령이 독립적입니다.)

```
00  fp32  mul r0 r1 r2
01  int32  add r3 r4 r5
02  fp32  mul r6 r7 r8
...
```

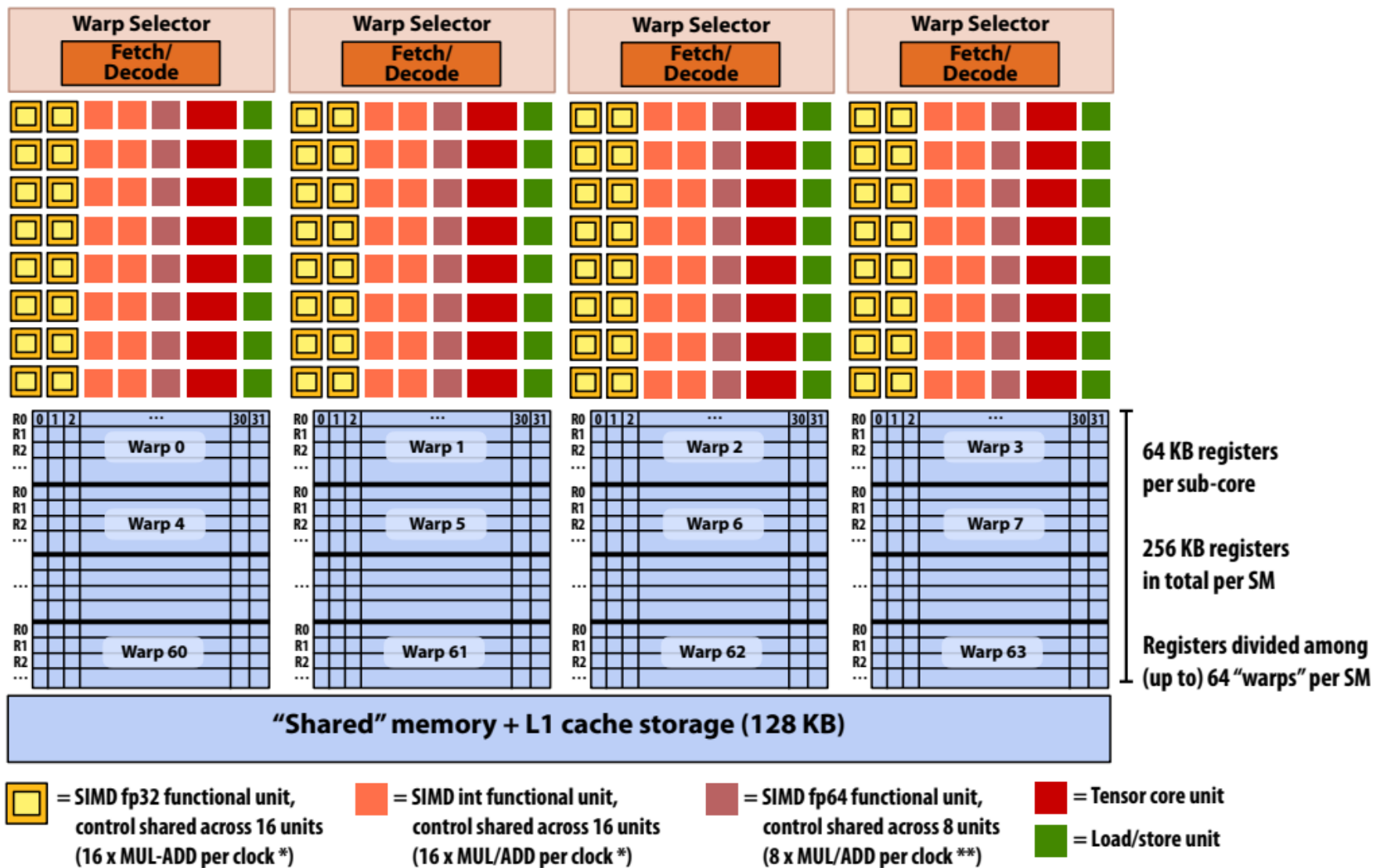


- CUDA 스레드의 전체 워프가 이 명령어 스트림을 실행한다는 점을 기억하자.
- 따라서 각 인스트럭션은 워프의 32개 CUDA 스레드 모두에서 실행됨.
- ALU가 16개이므로 전체 워프에 대한 인스트럭션을 실행하려면 두 개의 클럭이 필요함.



NVIDIA V100 GPU SM

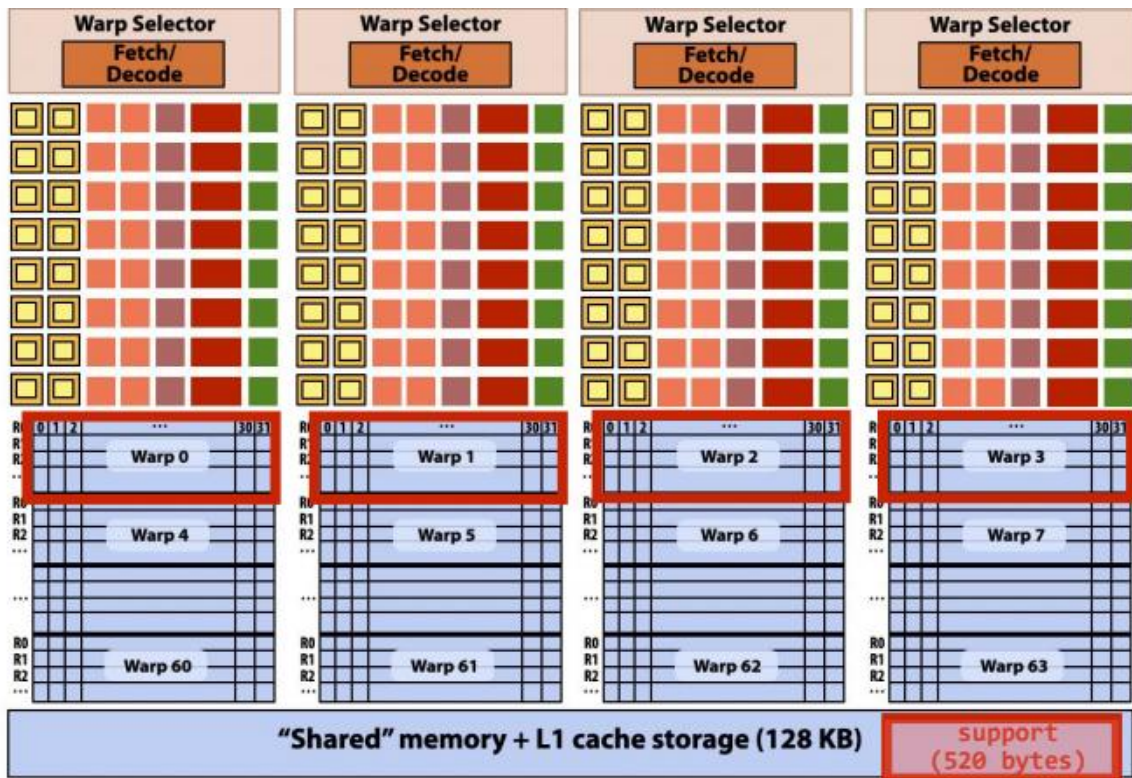
This is one NVIDIA V100 streaming multi-processor (SM) unit



* one 32-wide SIMD operation every 2 clocks

** one 32-wide SIMD operation every 4 clocks

Running a thread block on a V100 SM



A convolve thread block is executed by 4 warps
(4 warps x 32 threads/warp = 128 CUDA threads per block)

각 클럭마다 SM 코어 작동:

- 각 서브코어가 실행 가능한 하나의 워프(파티션의 16개 워프에서)를 선택
- 각 서브코어가 워프 내 CUDA 스레드의 다음 명령어를 실행(이 명령어는 발산에 따라 워프 내 CUDA 스레드의 전체 또는 하위 집합에 적용될 수 있음)

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input,
                        float* output)
{
    __shared__ float support[THREADS_PER_BLK+2];
    int index = blockIdx.x * blockDim.x +
                threadIdx.x;

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x]
            = input[index+THREADS_PER_BLK];
    }

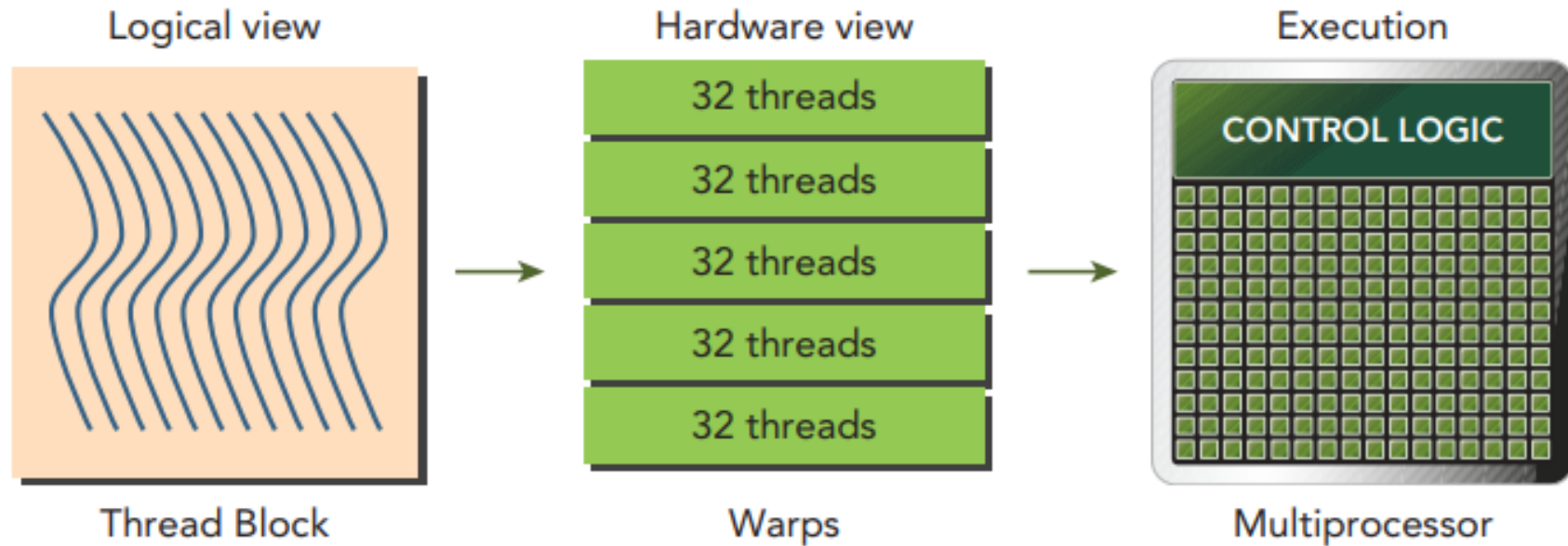
    __syncthreads();

    float result = 0.0f; // thread-local
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result;
}
```

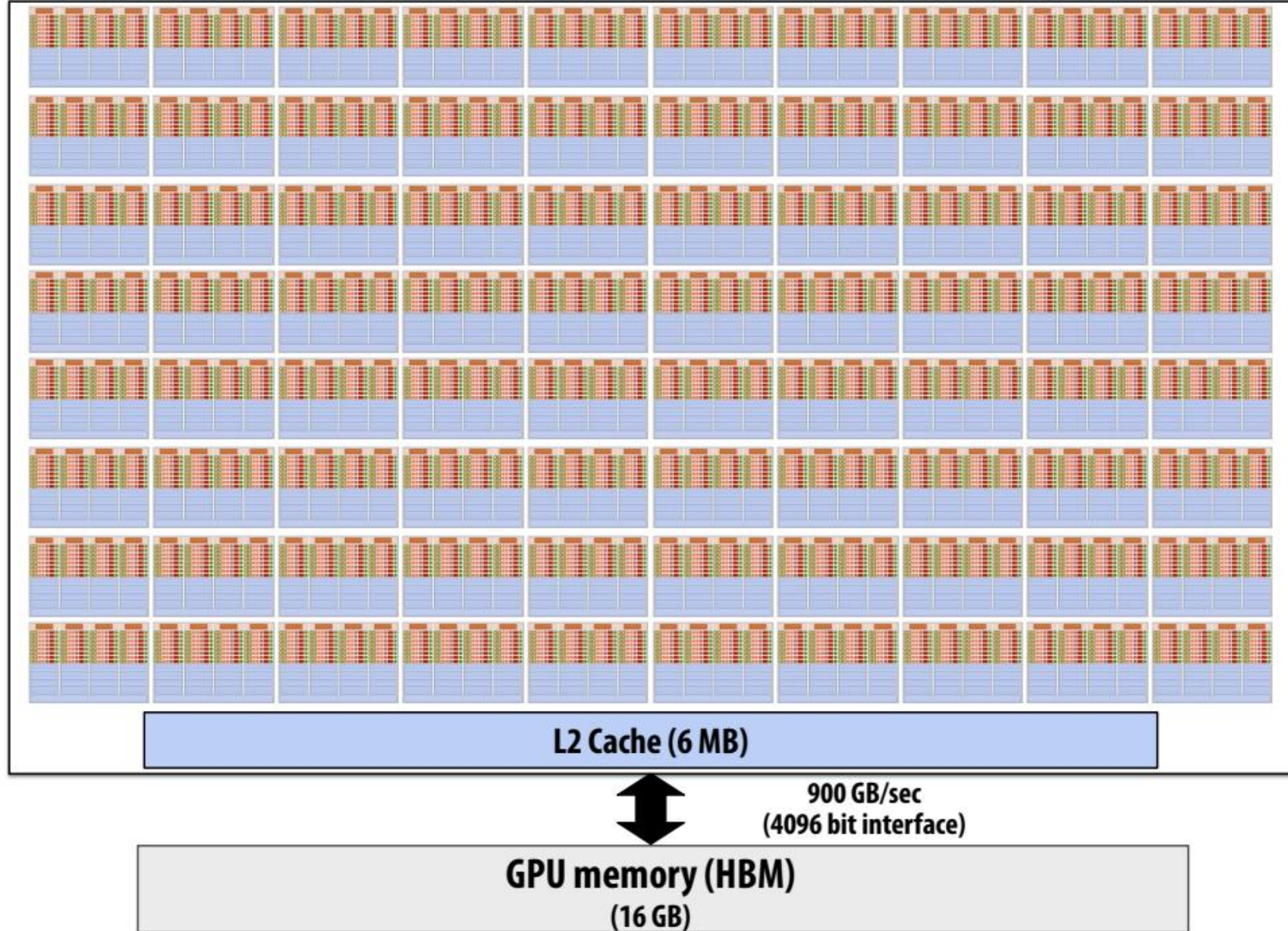
Warps and Thread Blocks

- Warp는 SM(Streaming Multi-processor)의 기본 실행 단위(unit of execution) 이다.
- 스레드 블록의 그리드를 실행하면, 그리드의 스레드 블록들은 SM들로 분배.
- 스레드 블록이 SM에 스케줄링되면 스레드 블록의 스레드들은 warp로 파티셔닝됨.
- 32개의 연속된 스레드들로 구성된 하나의 warp는 **SIMT**(Single Instruction Multiple Thread) 방식으로 실행.
- 즉, 모든 스레드는 동일한 명령어를 실행하고, 각 스레드는 할당된 private data에 대해 작업을 수행합니다.



- 스레드 블록은 1,2,3차원으로 구성될 수 있음. 하지만, 하드웨어 관점에서 살펴보면, 모든 스레드는 1차원으로 정렬됨.
- 각 스레드는 블록에서 unique ID를 가지고 있음. 1차원 스레드 블록에서, 스레드의 unique ID는 CUDA에 내장된 변수인 `threadIdx.x`에 저장되고, 연속된 `threadIdx.x`를 가진 스레드들이 warp로 그룹화됨.

NVIDIA V100 GPU (80 SMs)



Summary: geometry of the V100 GPU

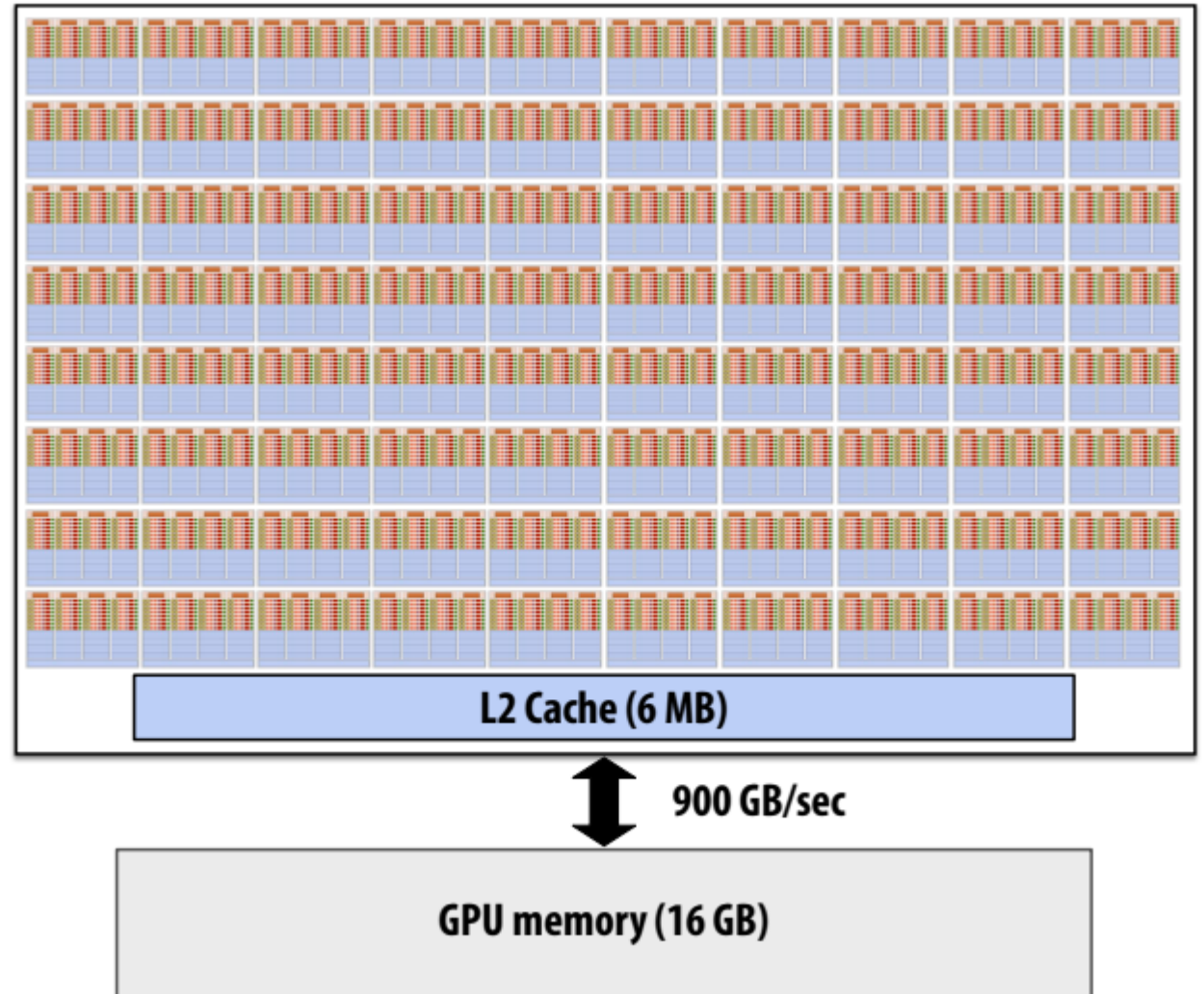
1.245 GHz clock

80 SM cores per chip

**$80 \times 4 \times 16 = 5,120$ fp32 mul-add ALUs
= 12.7 TFLOPs ***

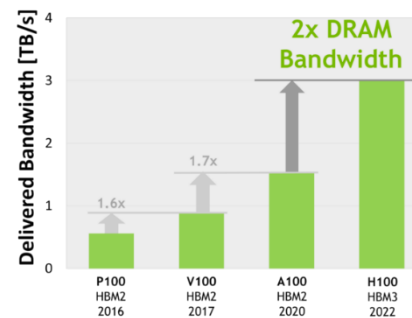
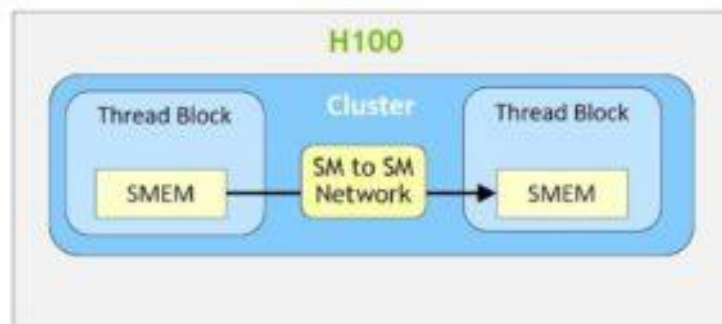
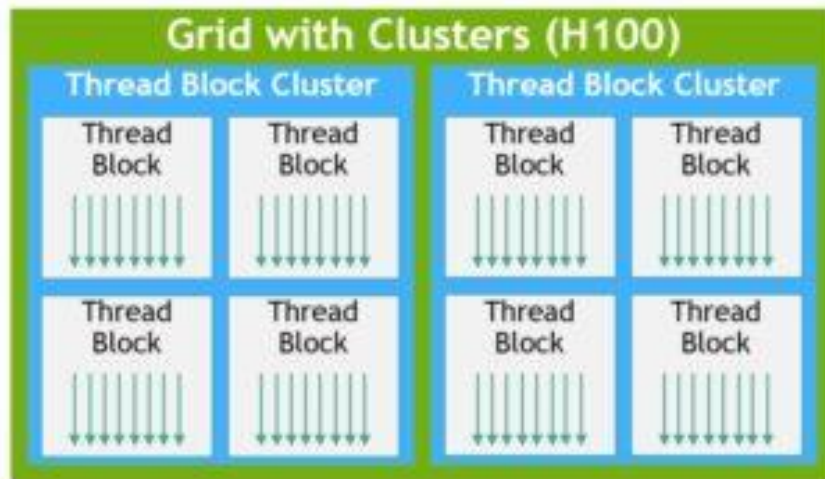
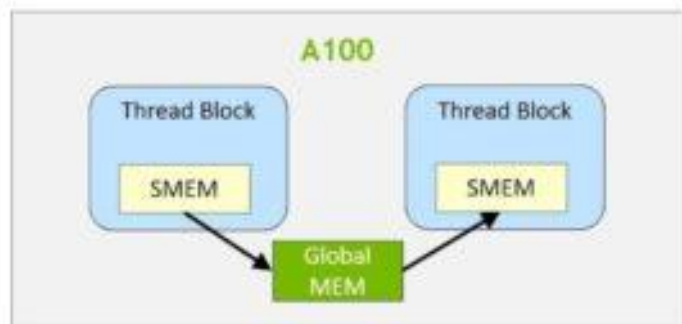
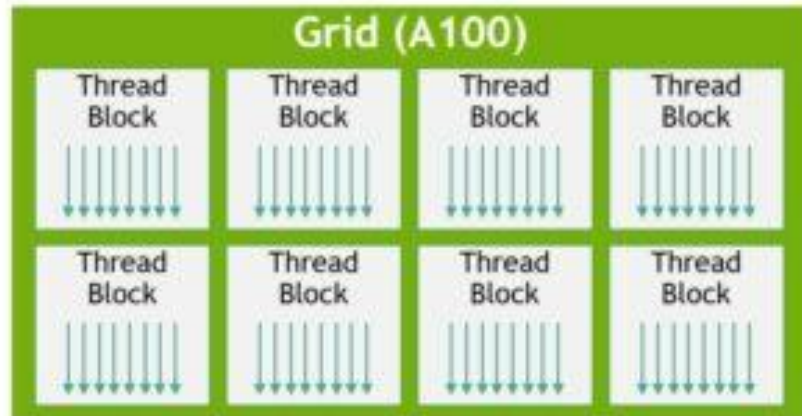
**Up to $80 \times 64 = 5120$ interleaved warps per chip
(163,840 CUDA threads/chip)**

*** mul-add counted as 2 flops:**



Running a CUDA program on a GPU

최신 GPU 아키텍처의 발전



Running the convolve kernel

convolve kernel's execution requirements:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

각 스레드 블록은 $130 \times \text{sizeof}(\text{float}) = 520$ 바이트의 공유 메모리를 할당.

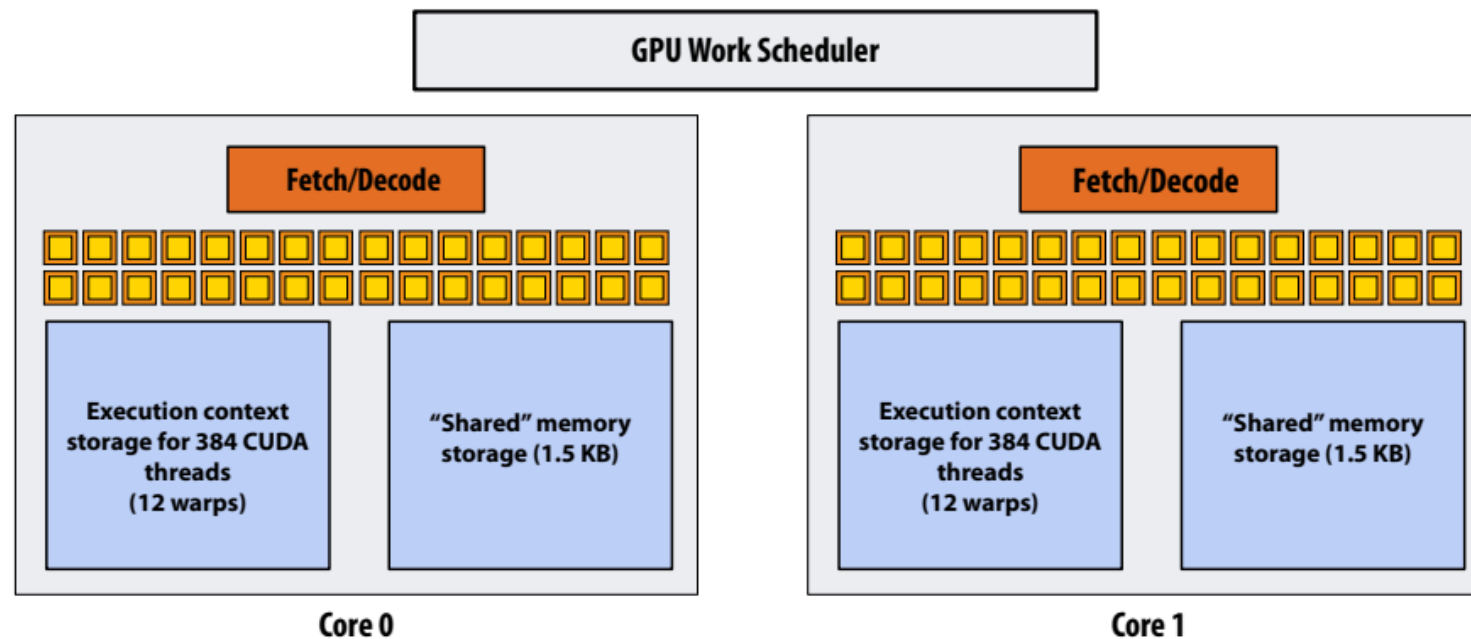
배열 크기 N 이 매우 커서 호스트 측 커널 실행 시 수천 개의 스레드 블록이 생성된다고 가정 보자

```
#define THREADS_PER_BLK 128
```

```
convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, input_array, output_array);
```

아래의 가상의 2코어 GPU에서 이 프로그램을 실행해 보자.

(참고: 가상의 코어는 앞서 강의에서 설명한 V100 SM 코어보다 실행 유닛 수가 적고, 더 적은 활성 워프를 지원하며, 공유 메모리도 더 적습니다.)



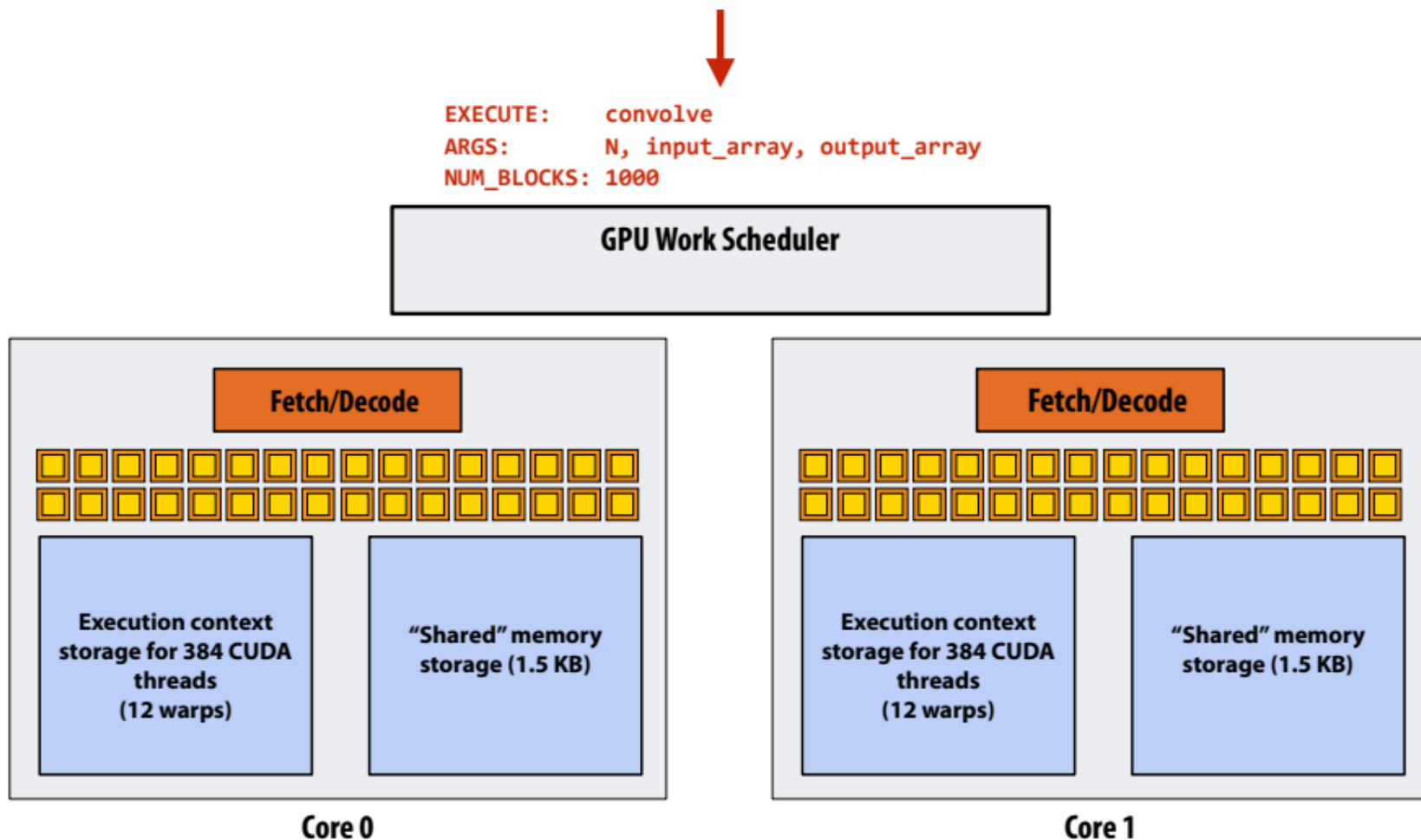
Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

Step 1: 호스트(HOST:CPU측)가 CUDA 디바이스(GPU)에 명령("이 커널 실행")을 전송.



Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

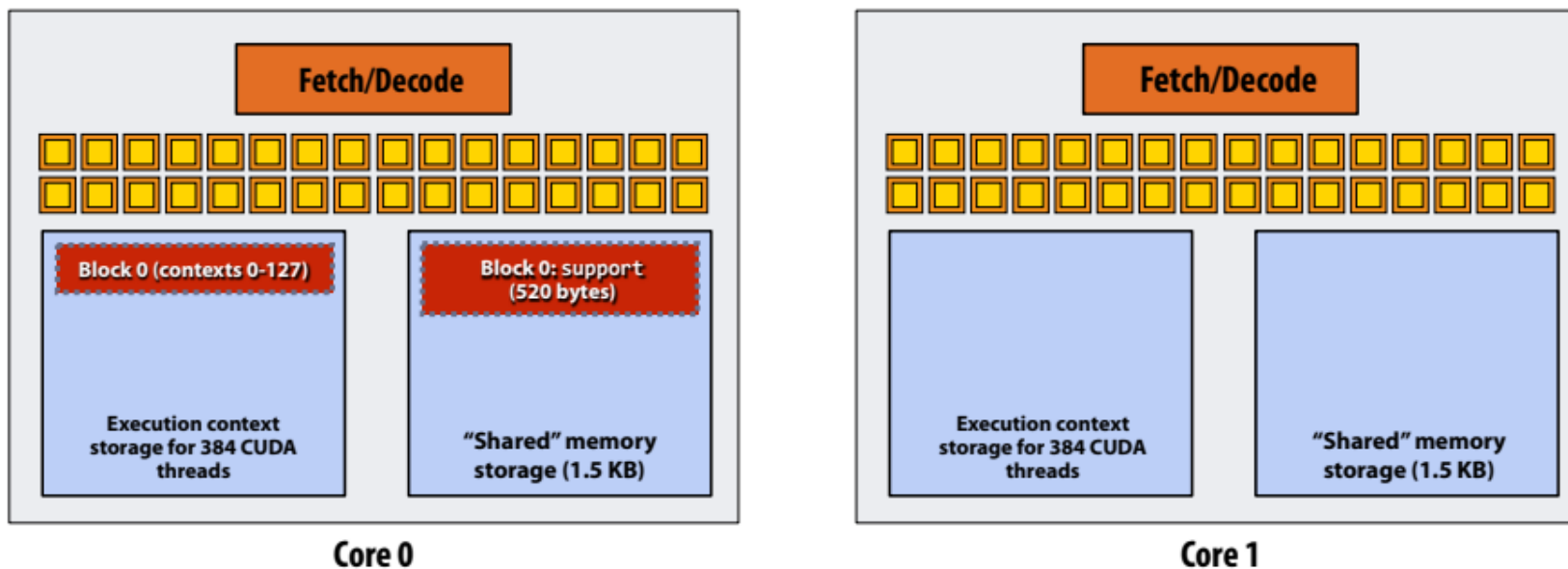
각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

Step 2: 스케줄러가 블록 0을 코어 0에 매핑
(128개의 스레드와 520바이트의 공유 스토리지에 대한 실행 컨텍스트 예약).



EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

NEXT = 1 GPU Work Scheduler
TOTAL = 1000



Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

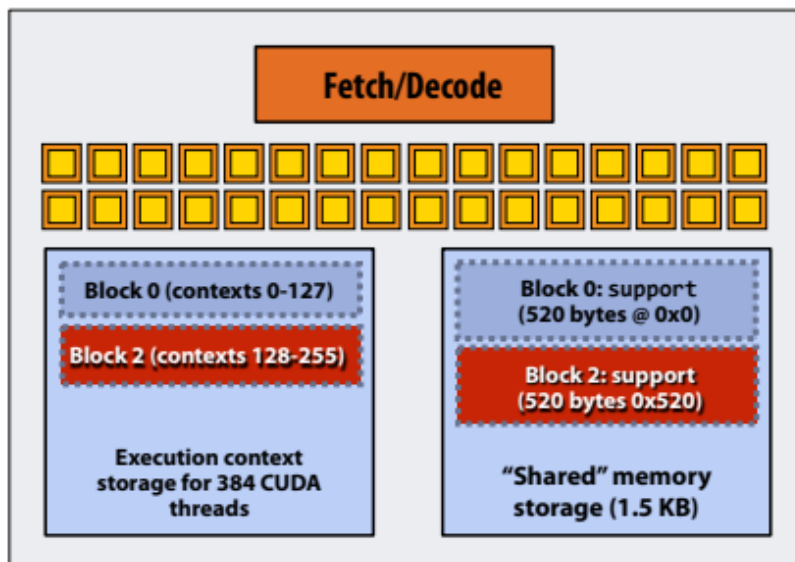
3단계: 스케줄러가 사용 가능한 실행 컨텍스트에 블록을 계속 매핑(인터리브 매핑 표시)



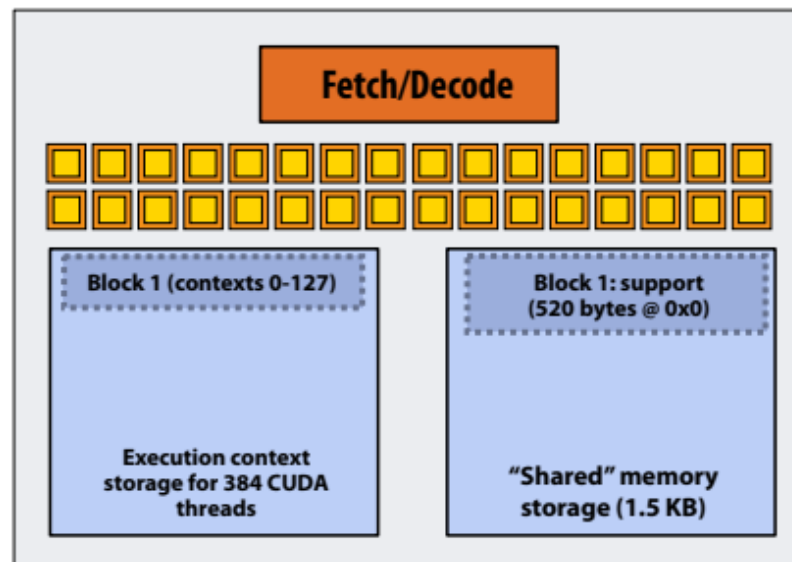
EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

NEXT = 3
TOTAL = 1000

GPU Work Scheduler



Core 0



Core 1

Running the CUDA kernel

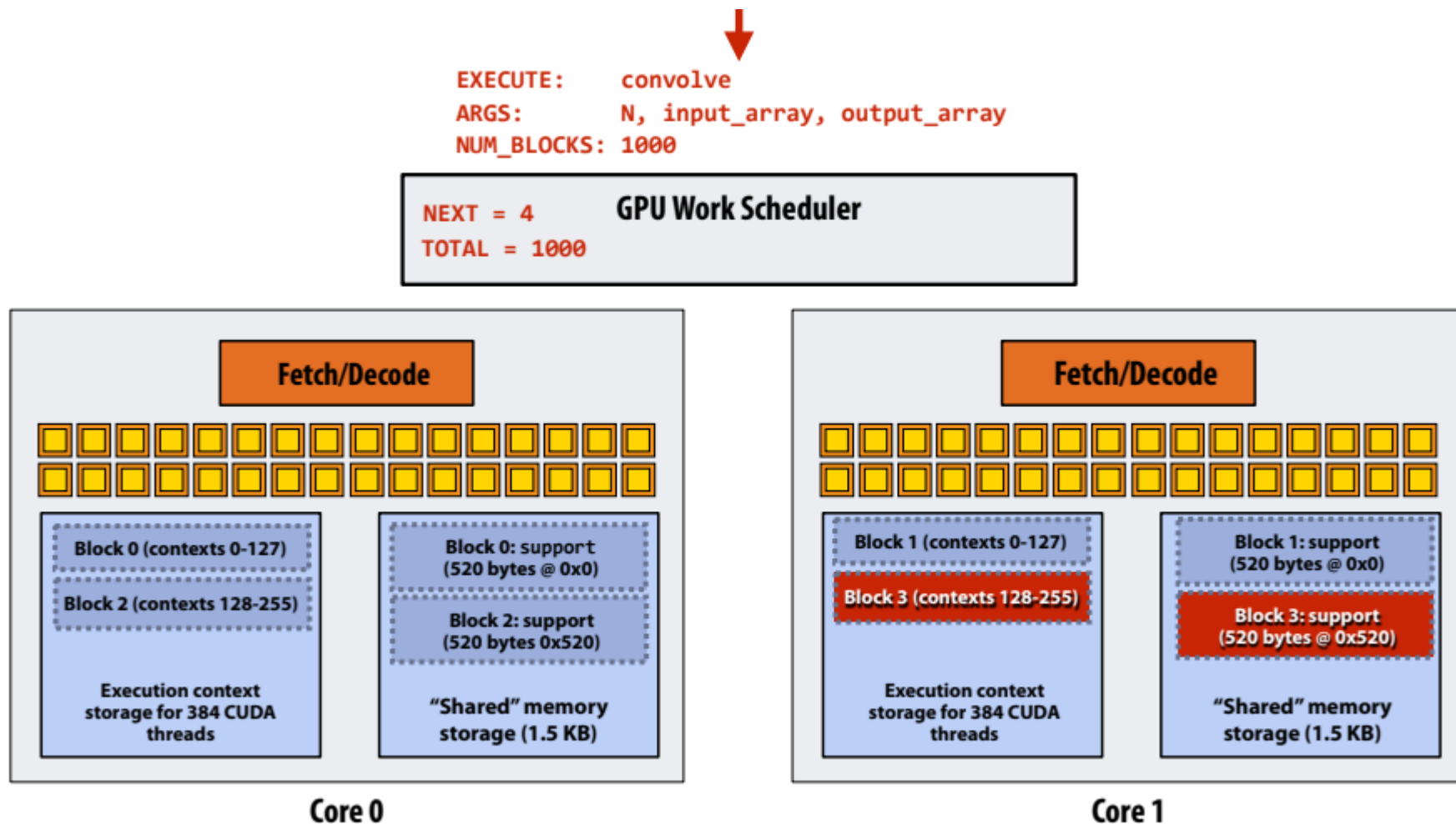
커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

3단계: 스케줄러가 블록을 사용 가능한 실행 컨텍스트에 계속 매핑(인터리브 매핑 표시).

코어에 두 개의 스레드 블록만 적합(세 번째 블록은 공유 스토리지가 부족하여 $3 \times 520\text{바이트} > 1.5\text{KB}$ 로 맞지 않음).



Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

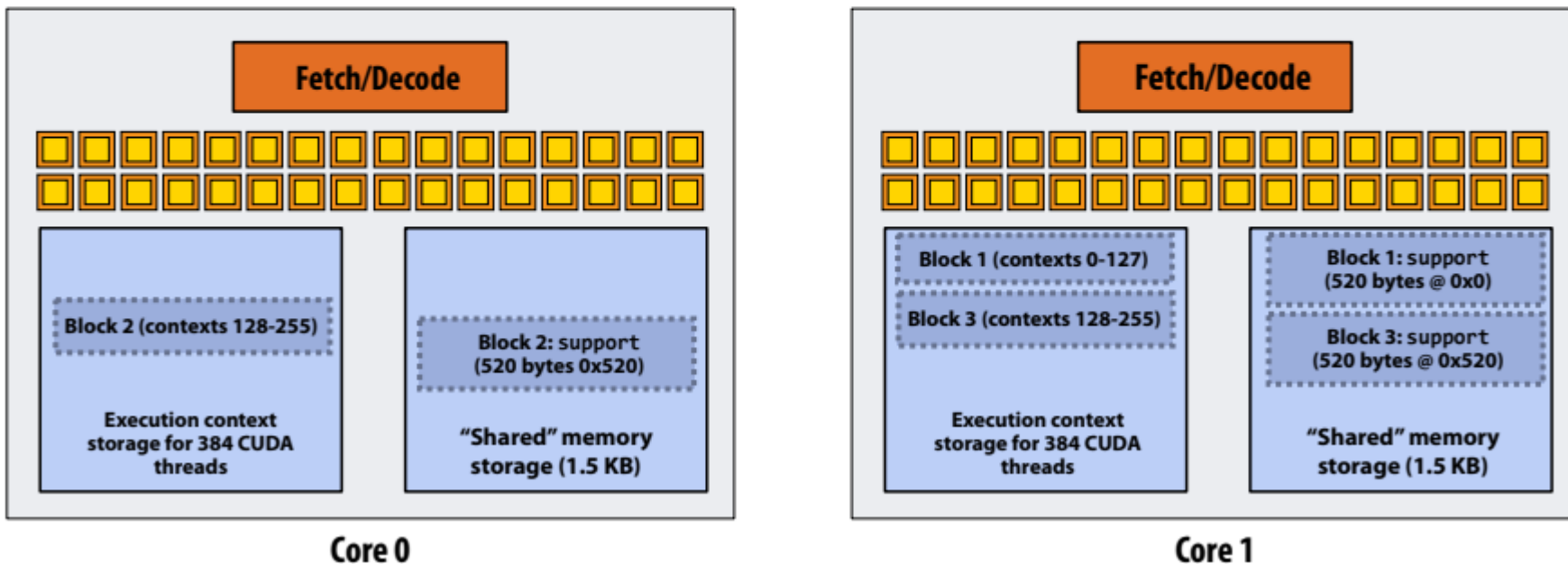
각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

4단계: 코어 0에서 스레드 블록 0이 완료됨



EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

NEXT = 4 GPU Work Scheduler
TOTAL = 1000



Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

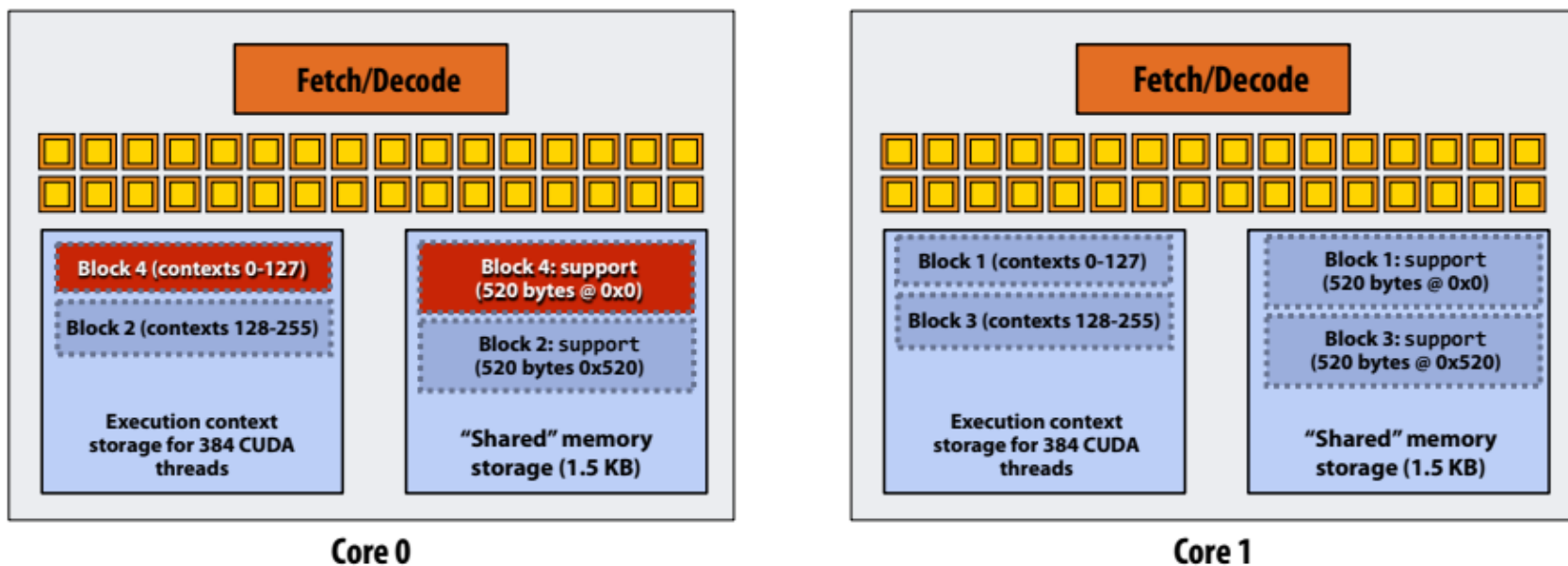
각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

5단계: 블록 4가 코어 0에 예약됨(실행 컨텍스트 0-127에 매핑됨)



EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

NEXT = 5 GPU Work Scheduler
TOTAL = 1000



Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

각 스레드 블록은 $130 \times \text{sizeof(float)} = 520$ 바이트의 공유 메모리를 할당

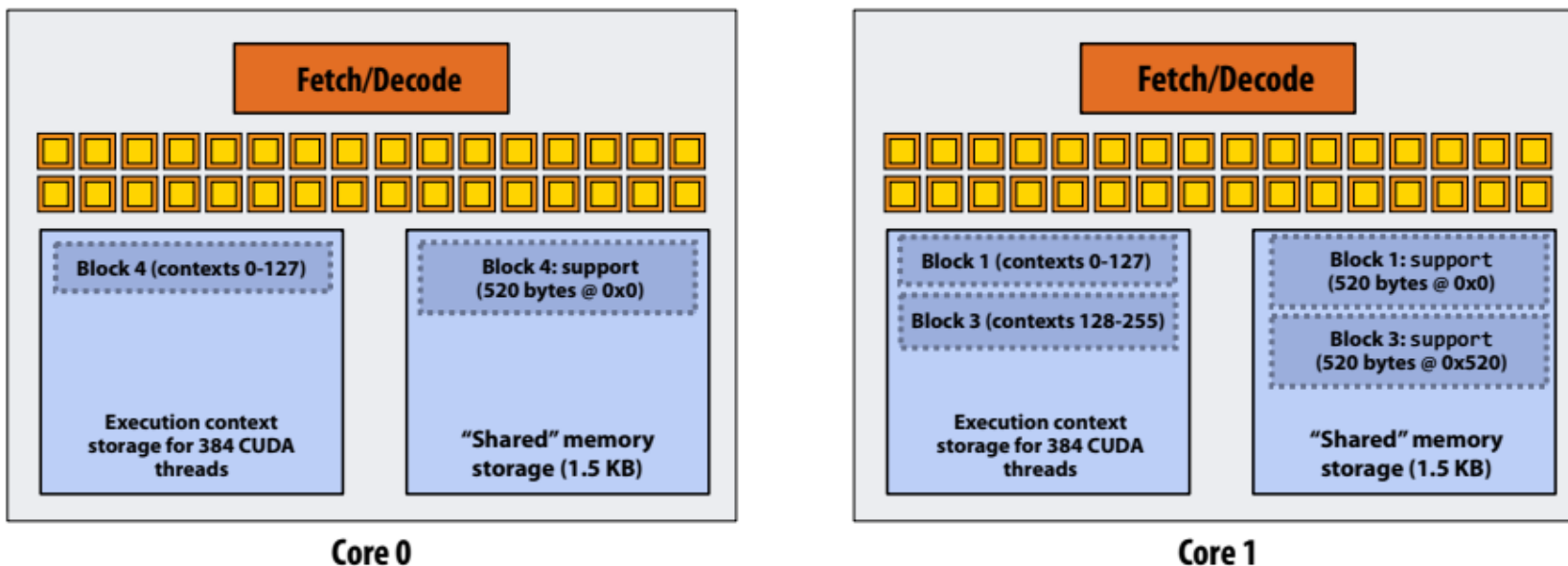
6단계: 코어 0에서 스레드 블록 2 완료



EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

NEXT = 5
TOTAL = 1000

GPU Work Scheduler



Running the CUDA kernel

커널의 실행 요구 사항:

각 스레드 블록은 128개의 CUDA 스레드를 실행.

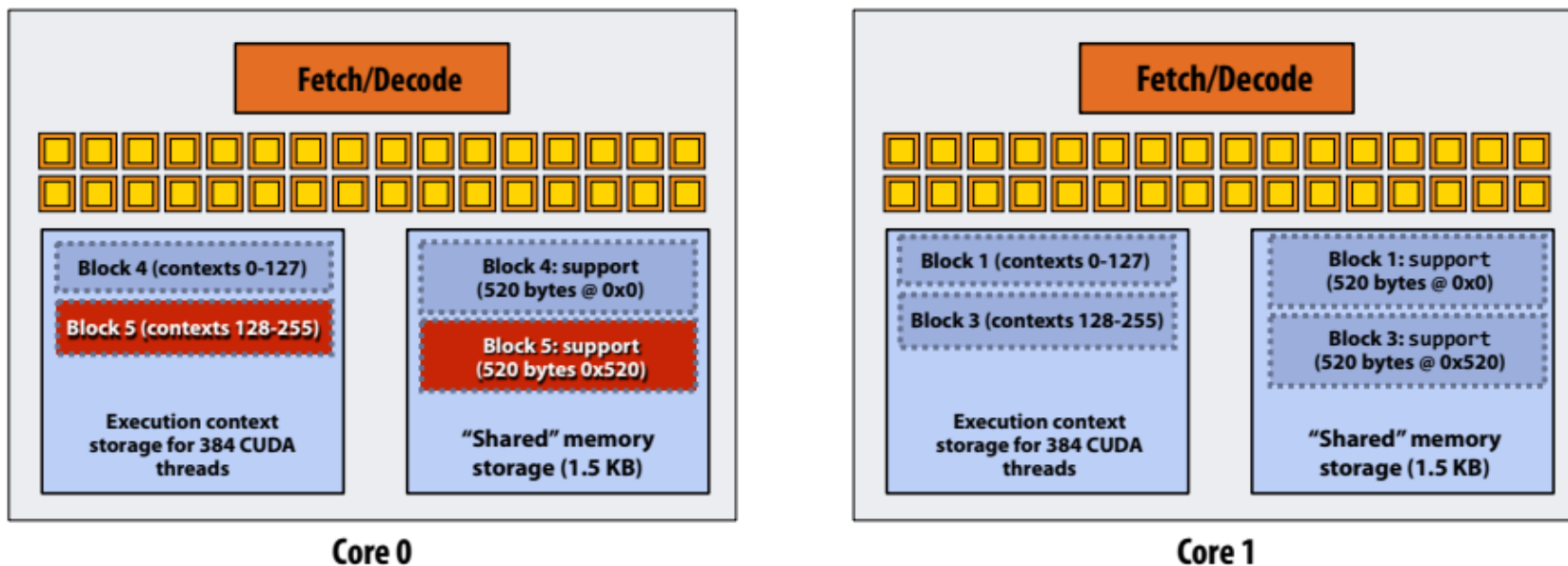
각 스레드 블록은 $130 \times \text{sizeof}(\text{float}) = 520$ 바이트의 공유 메모리를 할당

7단계: 스레드 블록 5가 코어 0에 예약됨(실행 컨텍스트 128-255에 매핑됨)



EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

NEXT = 6 GPU Work Scheduler
TOTAL = 1000



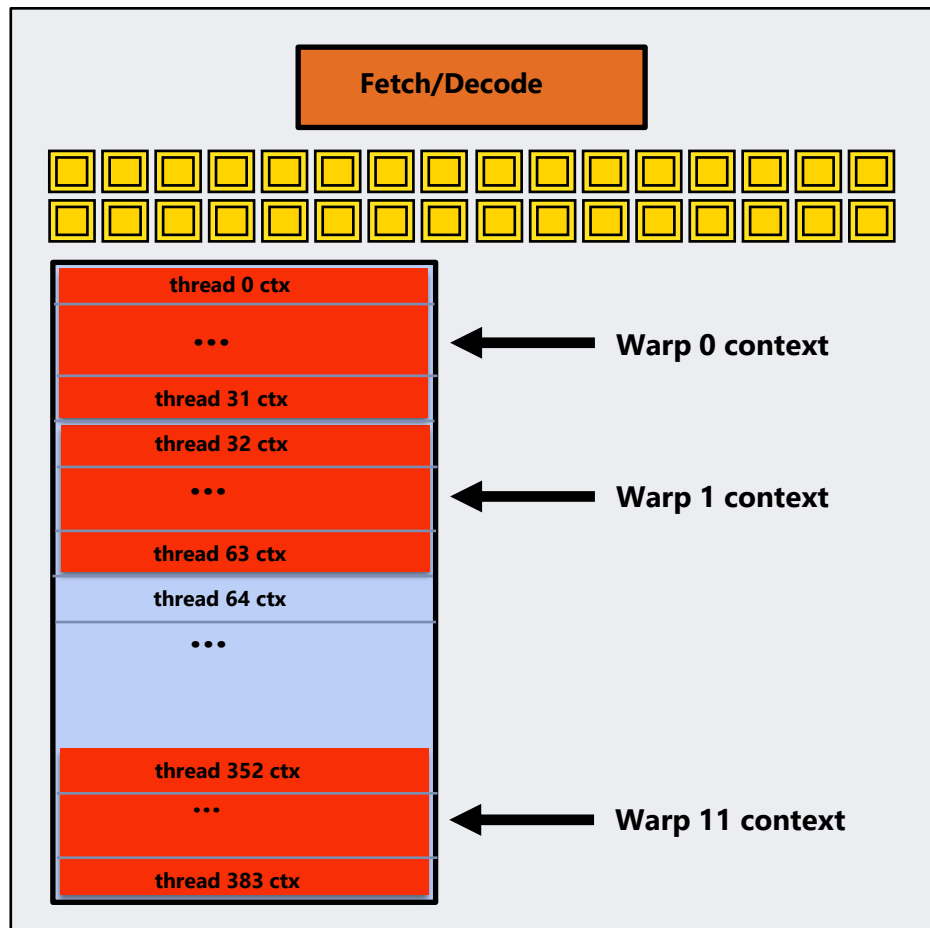
고급 스케줄링 질문:

(다음 예제를 이해했다면 **CUDA** 프로그램이 **GPU**에서 어떻게 실행되는지 잘 이해하고 있으며,
지금까지 강좌에서 다룬 작업 스케줄링 문제도 잘 이해하고 있는 것입니다.)

검토: "워프"란 무엇인가요?

워프는 NVIDIA GPU의 CUDA 구현 세부 사항이다.

최신 NVIDIA 하드웨어에서는 스레드 블록의 32개 CUDA 스레드 그룹이 32쪽 SIMD 실행을 사용하여 동시에 실행.



이 가상의 NVIDIA GPU 예시에서:

- 코어는 12개의 워프에 대한 컨텍스트를 유지.
- 각 클럭을 실행할 하나의 워프를 선택.

검토: "워프"란 무엇인가요?

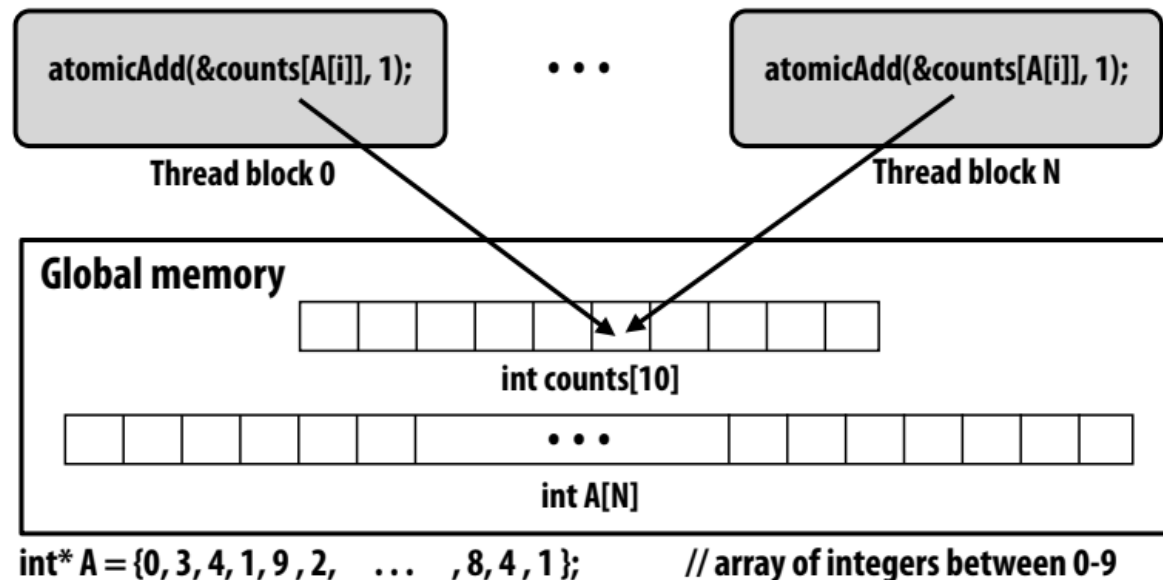
- 워프는 NVIDIA GPU의 CUDA 구현 세부 사항.
- 최신 NVIDIA 하드웨어에서 스레드 블록의 32개 CUDA 스레드 그룹은 32폭 SIMD 실행을 사용하여 동시에 실행.
 - 이러한 32개의 논리적 CUDA 스레드는 하나의 명령 스트림을 공유하므로 분산 실행으로 인해 성능이 저하될 수 있다.
- 명령어 스트림을 공유하는 32개의 스레드 그룹을 워프라고 함.
 - 스레드 블록에서 스레드 0-31은 동일한 워프에 속함(스레드 32-63 등도 마찬가지).
 - 따라서 256개의 CUDA 스레드가 있는 스레드 블록은 8개의 워프에 매핑.
 - 지난번에 설명한 GTX 980의 각 "SMM" 코어는 최대 64개의 워프 실행을 스케줄링하고 인터리빙할 수 있음.
 - 따라서 " SMM " 코어는 여러 CUDA 스레드 블록을 동시에 실행할 수 있음

CUDA 추상화 구현

- **스레드 블록은 시스템에서 원하는 순서대로 예약할 수 있음**
 - 시스템은 블록 간에 종속성이 없다고 가정.
 - 논리적으로는 동시적이다.
 - ISPC 작업과 매우 비슷?
- **동일한 블록의 CUDA 스레드가 동시에 실행됨(동시에 라이브).**
 - 블록 실행이 시작되면 모든 스레드가 존재하고 레지스터 상태가 할당됨.
(이러한 의미는 시스템에 스케줄링 제약을 가함)
 - CUDA 스레드 블록은 그 자체로 SPMD 프로그램(프로그램 인스턴스의 ISPC 갱과 같음)
 - 스레드 블록의 스레드는 동시적이며 협력하는 "workers"임.
- **CUDA 구현:**
 - NVIDIA GPU 워프는 ISPC instances gang과 유사한 성능 특성을 가짐(단, ISPC gang과 달리 워프 개념은 프로그래밍 모델*에 존재하지 않음).
 - 스레드 블록의 모든 워프는 동일한 SM에 예약되므로 공유 메모리 변수를 통해 높은 BW/저지연 통신이 가능.
 - 블록의 모든 스레드가 완료되면 블록 리소스(공유 메모리 할당, 워프 실행 컨텍스트)를 다음 블록에서 사용할 수 있게 됨.

히스토그램을 만드는 프로그램을 생각해 보자

- 이 예제: 배열에 있는 값의 히스토그램을 작성합니다.
 - 모든 CUDA 스레드는 전역 메모리의 공유 변수를 원자적으로 업데이트함.
- CUDA 스레드 블록이 독립성을 보장한다고 주장한 적이 없음. CUDA가 어떤 순서로든 스케줄링할 수 있는 권리를 보유한다고만 언급함.
- 이것은 유효한 코드이다! 이러한 아토믹스 사용은 어떤 순서로든 블록을 스케줄링하는 구현의 기능에 영향을 미치지 않는다. (아토믹스는 상호 배제를 위해 사용되며, 그 이상은 아님).



Bonus slide: “persistent thread” CUDA 프로그래밍 스타일

```
#define THREADS_PER_BLK 128
#define BLOCKS_PER_CHIP 80 * (32*64/128) // specific to V100 GPU

__device__ int workCounter = 0;          // global mem variable

__global__ void convolve(int N, float* input, float* output) {
    __shared__ int startingIndex;
    __shared__ float support[THREADS_PER_BLK*2]; // shared across block
    while (1) {

        if (threadIdx.x == 0)
            startingIndex = atomicInc(workCounter, THREADS_PER_BLK);
        __syncthreads();
        if (startingIndex >= N)
            break;

        int index = startingIndex + threadIdx.x; // thread local
        support[threadIdx.x] = input[index];
        if (threadIdx.x < 2)
            support[THREADS_PER_BLK+threadIdx.x] = input[index+THREADS_PER_BLK];

        __syncthreads();

        float result = 0.0f; // thread-local variable
        for (int i=0; i<3; i++)
            result += support[threadIdx.x + i];
        output[index] = result;

        __syncthreads();
    }
}

// host code ////////////////////////////////////////
int N = 1024 * 1024;
cudaMalloc(&devInput, N*2); // allocate array in device memory
cudaMalloc(&devOutput, N); // allocate array in device memory
// properly initialize contents of devInput here ...

convolve<<<BLOCKS_PER_CHIP, THREADS_PER_BLK>>>>(N, devInput, devOutput);
```

- 아이디어: 기본 GPU 구현에서 지원하는 코어 수와 코어당 블록 수에 대한 지식이 필요한 CUDA 코드를 작성함.
- 프로그래머는 GPU를 채울 수 있는 만큼의 스레드 블록을 정확히 실행함.

(프로그램은 GPU 구현에 대해 GPU가 실제로 모든 블록을 동시에 실행할 것이라는 가정을 함. 욕!)

- 이제 블록에 대한 작업 할당은 전적으로 애플리케이션에 의해 구현됨. (GPU의 스레드 블록 스케줄러 우회)
- 이제 프로그래머의 정신 모델은 *모든* CUDA 스레드가 한 번에 GPU에서 동시에 실행되고 있다는 것임.

CUDA summary

- 실행 의미론(Execution semantics).
 - 문제를 스레드 블록으로 분할하는 것은 데이터 병렬 모델의 정신에 따른 것 (머신 독립적이어야 함: 시스템이 코어 수에 관계없이 블록을 스케줄링함).
 - 스레드 블록의 스레드는 실제로 동시에 실행됨(협력하기 때문에 그래야만 함).
 - 단일 스레드 블록 내부: SPMD 공유 주소 공간 프로그래밍.
 - 이러한 실행 모델 간에는 미묘하지만 주목할 만한 차이점이 있음. 반드시 이해해야 함. (그리고 병렬 프로그래밍 시스템을 접할 때마다 어떤 시맨틱이 사용되고 있는지 스스로에게 물어보세요).
- 메모리 시맨틱(Memory semantics)
 - 분산 주소 공간: 호스트/장치 메모리.
 - 디바이스 메모리 내에서 로컬/블록 공유/글로벌 변수를 스레딩함
 - 로드/저장은 이들 간에 데이터를 이동함(따라서 로컬/공유/글로벌 메모리는 별개의 주소 공간으로 생각하는 것이 정확함).
- 주요 구현 세부 사항(Key implementation details):
 - 스레드 블록의 스레드는 공유 메모리를 통해 빠르게 통신할 수 있도록 동일한 GPU “SM”에 스케줄링됨.
 - 스레드 블록의 스레드는 GPU 하드웨어에서 SIMT 실행을 위해 워프로 그룹화됨.

One last point...

- 이 강의에서는 GPU의 프로그래머블 코어를 위한 CUDA 프로그램 작성에 대해 언급.
 - 작업(CUDA 커널 실행으로 설명됨)은 하드웨어 작업 스케줄러를 통해 코어에 매핑.
- GPU 실행을 구동하기 위한 그래픽 파이프라인 인터페이스도 있습니다.
 - 그리고 GPU의 흥미로운 비프로그래밍 기능은 대부분 그래픽 파이프라인 작업의 실행을 가속화하기 위해 존재.
 - CUDA 프로그램을 실행할 때는 거의 “꺼져 있는” 상태.
- GPU가 그래픽 파이프라인을 효율적으로 구현하는 방법은 그래픽 클래스의 주제... *.